

Anne Baanen
Alexander Bentkamp
Jasmin Blanchette
Xavier Génèreux
Johannes Hölzl
Jannis Limperg

Lean-zh 项目组 译

逻辑验证漫游指南

2026 桌面版

(2026 年 5 月 16 日)



lean-forward.github.io/

[hitchhikers-guide/2026](https://lean-forward.github.io/hitchhikers-guide/2026)

目录

目录	iii
前言	vii
一 基础	1
1 类型与项	3
1.1 类型	3
1.2 项	4
1.3 类型检查与类型推断	6
1.4 类型居留	8
1.5 新引入的 Lean 结构小结	9
2 程序与定理	11
2.1 类型定义	11
2.2 函数定义	16
2.3 定理陈述	20
2.4 新引入的 Lean 结构小结	22
3 逆向证明	23
3.1 策略模式	24
3.2 基本策略	26
3.3 关于连结词和量词的推理	28
3.4 关于相等的推理	32
3.5 重写策略	33
3.6 数学归纳法证明	35
3.7 归纳策略	36
3.8 清理策略	37
3.9 新引入的 Lean 结构小结	37
	iii

4	正向证明	39
4.1	结构化证明	39
4.2	结构化构造	42
4.3	关于联结词和量词的正向推理	44
4.4	计算式证明	50
4.5	使用策略进行正向推理	51
4.6	依值类型	52
4.7	PAT 原理	54
4.8	通过模式匹配与递归进行归纳	57
4.9	新引入的 Lean 结构小结	58
二	函数式与逻辑编程	61
5	函数式编程	63
5.1	归纳类型	63
5.2	结构归纳	64
5.3	结构递归	66
5.4	模式匹配表达式	67
5.5	结构体	69
5.6	类型类	71
5.7	列表	74
5.8	二叉树	80
5.9	Cases 策略	82
5.10	依值归纳类型	83
5.11	新引入的 Lean 结构小结	85
6	归纳谓词	87
6.1	入门示例	87
6.2	逻辑符号	94
6.3	规则归纳	95
6.4	线性算术策略	98
6.5	消去	98
6.6	更多示例	101
6.7	归纳中的陷阱	106
6.8	新引入的 Lean 结构小结	108
7	带作用的编程	109
7.1	引入示例	109

7.2	两个操作和三个定律	112
7.3	一种类型类	114
7.4	无作用	114
7.5	基本异常	115
7.6	可变状态	117
7.7	非确定性	120
7.8	通用证明策略	122
7.9	一个通用算法：在列表上迭代	122
7.10	新引入的 Lean 结构小结	124
8	元编程	125
8.1	策略组合子	126
8.2	宏	129
8.3	元编程单子	130
8.4	第一个示例：一个 Assumption 策略	132
8.5	表达式	133
8.6	第二个示例：一个合取式分解策略	135
8.7	第三个示例：一个直接证明查找器	138
8.8	杂项策略	141
8.9	新引入的 Lean 结构小结	141
三	程序语义	143
四	数学	145
	参考文献	147

前言

形式证明助手是一些旨在帮助用户进行计算机验证证明的软件。我们通常称之为**证明助手**或**交互式定理证明器**，但一位沮丧的学生创造了「**阻止证明的野兽**」一词来形容它，而语音识别软件则偶尔会将「**定理证明器**」误识别为「**恐惧改进器**」，敬请留意。

严格证明与形式证明 交互式定理证明有其自己的术语，始于「证明」的概念。**形式证明**是用逻辑形式主义表达的逻辑论证。在本文中，「形式」表示「逻辑的」或「基于逻辑的」。逻辑学家，也就是研究逻辑学的数学家，在计算机出现之前的几十年里就在纸上进行形式证明。但如今，形式证明几乎总是使用证明助手来完成。

相比之下，**非形式证明**就是数学家通常所说的证明。这类证明通常用 $\text{\LaTeX}$ 或黑板完成，也被称为「纸笔证明」。其详细程度可能差异很大，诸如「显然」、「清楚地」和「不失一般性」之类的短语将部分证明负担转移给了读者。**严格证明**则是一种非常详细的非形式证明。

证明助手的主要优势在于，它们能够运用精确的逻辑，帮助学生构建高度可靠、无歧义的数学陈述证明。它们可以用来证明任意高级的结果，远远超越了玩具示例和逻辑谜题。形式证明还能帮助学生理解有效定义或有效证明的构成。引用 Scott Aaronson 的话：¹

我还记得自己曾经批改过数百份试卷，学生们一开始只是假设需要证明的内容，或者一页一页地写满胡言乱语，希望在一片混乱中，他们可能会偶然说出一些正确的话。

当我们发展一个新理论时，形式证明可以帮助我们探索它。当我们推广、重构或以其他方式修改现有证明时，形式证明非常有用，就像编译器帮助我们开发正确的程序一样。形式证明提供了高度的可信度，使其他人更容易对其进行审查。此外，形式证明可以构成经过验证的计算工具（例如，经过验证的计算机代数系统）的基础。

¹<https://www.scottaaronson.com/teaching.pdf>

成功案例 在数学和计算机科学领域，辅助证明已取得诸多成功。数学形式化领域的一些里程碑式成果包括：Gonthier 等人证明四色定理 [5]，Gonthier 等人证明奇数阶定理 [6]，Hales 等人证明开普勒猜想 [9]，以及 Buzzard 等人定义完美拟态空间 [4]。该领域最早的研究始于 20 世纪 60 年代，由 Nicolaas de Bruijn 及其同事在一个名为 AUTOMATH 的系统中开展。²

包括 AMD [24] 和英特尔 [10] 在内的一些公司一直在使用证明助手来验证其设计。在学术界，一些里程碑式的成果包括操作系统内核 seL4 [13] 和 CertiKOS [8] 的验证，以及经过验证的编译器 CompCert [16]、JinjaThreads [18] 和 CakeML [15] 的开发。

证明助手 全球范围内，有许多正在开发或使用的证明助手。以下列出了一些主要的助手，按其逻辑基础进行分类：

- Set Theory **集合论**：Isabelle/ZF、Metamath、Mizar；
- Simple Type Theory **简单类型论**：HOL4、HOL Light、Isabelle/HOL、PVS；
- Dependent Type Theory **依值类型论**：Agda、Lean、Matita、Rocq（以前叫做 Coq）；
- First-Order Logic 类 Lisp 的**一阶逻辑**：ACL2。

有关证明助手和交互式定理证明的历史，我们参考了 Harrison、Urban 和 Wiedijk 撰写的资料丰富的章节 [11]。

Lean Lean 是一个证明助手，主要由 Leonardo de Moura（亚马逊网络服务，AWS）自 2012 年以来开发。其数学库 mathlib 最初由 Jeremy Avigad（卡内基梅隆大学，CMU）领导开发，现由 Lean 用户社区维护并进一步扩展 [19]。³

本指南使用 Lean 版本 v4.24.0，mathlib 修订版 f897ebcf72cd16f8，以及一些扩展，收集在名为 LoVeLib 的小型库中。⁴ 虽然这只是一个研究项目，还存在一些不完善的地方，但 Lean 非常适合用于交互式定理证明教学，原因如下：

- 它具有表达能力很强且非常有趣的，基于 Calculus of Inductive Construction **归纳构造演算**（一种 Dependent Type Theory **依值类型论**）的逻辑。
- 它扩展了经典公理和 Quotient Type **商类型**，使其可用于验证数学。
- 它包含一个便捷的元编程框架，可用于编写自定义证明的自动化程序。
- 它通过 Visual Studio Code 插件包含一个现代化的用户界面。
- 它具有非常易读且完备的文档。
- 它是开源的。

²<https://www.win.tue.nl/automath/>

³<https://github.com/leanprover-community/mathlib4>

⁴https://github.com/lean-forward/logical_verification_2026/raw/main/lean/LoVe/LoVelib.lean

Lean 的核心库仅包含基本定义和工具。mathlib 则提供了更多功能。尽管名为 mathlib，但它不仅仅是一个数学库，它还在 Lean 的核心库之上提供了大量的自动化功能，满足了人们对现代证明助手的需要。

关于本指南 本指南最初是作为阿姆斯特丹自由大学理学硕士课程《逻辑验证》(LoVe) 的配套课程而设计的。其主要目的是教授交互式定理证明。Lean 是手段，而非目的本身。因此，本指南并非旨在成为全面的 Lean 教程——为此，我们推荐 *Lean 4 定理证明* [?]。本指南也不能替代练习和作业。定理证明不是用来看的，它只能通过实践来学习。

具体来说，你的目标是：

- 学习交互式定理证明的基本理论和技术；
- 学习如何使用逻辑作为一种精确的语言来建模系统并描述它们的性质；
- 熟悉一些证明助手可以成功应用的领域，例如函数式编程、命令式编程语言的语义以及数学；
- 培养可应用于大型项目的实践技能（无论是个人项目、硕士、博士项目还是工业界）；
- 能够转换到其他证明助手并应用所学的知识；
- 对该领域有充分的了解，可以开始阅读在国际会议（例如《认证程序与证明》(CPP) 和《交互式定理证明》(ITP)）或期刊（例如《自动推理杂志》(JAR)）上发表的相关科学论文。

具备良好的 Lean 知识后，应该可以轻松迁移到其他基于依值类型论的证明助手，例如 Agda 或 Rocq，或者基于简单类型论的系统，例如 HOL4 或 Isabelle/HOL。

本指南的一个重要特点是，它与 Knuth 的 *T_EXbook* [14] 一样，并非总是说实话。为了简化阐述，本书提出了一些关于 Lean 的基本但错误的论断。大多数这些论断都会在后续章节中得到纠正。与 Knuth 一样，我们认为「这种故意撒谎的技巧实际上会让你更容易学习这些理念。一旦你理解了一个简单但错误的规则，用它的例外来补充它就不难了。」

本指南附带的 Lean 文件可在公共源码库中找到。⁵ 文件的命名方案遵循本指南的章节名，因此，LoVe07_EffectiveProgramming_Demo.lean 是与课堂上复习的第 7 章「副作用编程」对应的主文件，LoVe07_EffectiveProgramming_ExerciseSheet.lean 是习题表，而 LoVe07_EffectiveProgramming_HomeworkSheet.lean 是家庭作业表。

我们非常感谢《Lean 4 定理证明》[?] 和《具体语义: Isabelle/HOL 描述》[21] 的作者，他们教会了我们 Lean 和编程语言语义学。他们的许多想法都体现在了本指南中。

⁵<https://github.com/Lean-zh/LoVe2026-zh/>

我们感谢 Robert Lewis 和 Assia Mahboubi 对本指南的组织 and 重点提出的宝贵意见。我们感谢 Kiran Gopinathan 和 Ilya Sergey 分享了第 2 章脚注 3 中提到的轶事，并允许我们进一步分享。我们感谢 Daniel Fabian 设计了本指南的第一个平板电脑优化版本。我们感谢 Paul Chisholm 撰写了相关 Lean 文件中的一些评论。我们感谢 Pietro Monticone 随着 Lean 和 mathlib 的发展而帮助更新 Lean 文件。We thank Moritz Roos for completing a proof about the real numbers, which is now included in the main Lean file associated with Chapter ???. We thank Henrik Böving for his comments on the Lean files associated with the guide. 我们感谢 Moritz Roos 完成了关于实数的证明，该证明现在包含在第 ??? 章对应的主 Lean 文件中。我们感谢 Henrik Böving 对与本指南相关的 Lean 文件提出的意见。我们感谢 Mark Summerfield 提出的许多文本建议。最后，我们感谢 Fabian Axer Avila、Chris Bailey、Kevin Buzzard、Paul Chisholm、Dominique Danco、Raufs Dunamalijevs、Wan Fokkink、Lina Gerlach、Alexandra Graß、Robert Lewis、Alexandre Rademaker、Antonius Danny Reyes、Robert Schütz、Kristina Sojakova、Patrick Thomas、Balazs Toth、Huub Vromen、Floris Westerman、Wijnand van Woerkom 和 Yiming Xu 报告了他们在本指南早期版本中发现的错别字和一些更严重的错误。如果您发现本文存在任何错误，请告知对应的作者。⁶

特殊符号 在本指南中，我们假设你使用 Visual Studio Code 及其「lean4」扩展来编辑 .lean 文件。Visual Studio Code 允许你通过输入反斜杠 \ 后跟 ASCII 标识符来输入 Unicode 符号。例如， $\rightarrow$ 、 $\forall$ 或 $\in$ 可以通过输入 \->、\fo 或 \in 并按下 Tab 键或空格键来输入。我们将自由使用这些符号。作为参考，我们提供了本指南中使用的主要非 ASCII 符号列表，并为每个符号提供了其对应的 ASCII 表示形式。在 Visual Studio Code 中，按住 Control 键或 Command 键的同时将鼠标悬停在某个符号上，就能看到该符号的不同输入方式。

⁶Jasmin Blanchette (jasmin.blanchette@ifi.lmu.de)

$\neg$	<code>\not</code>	$\wedge$	<code>\and</code>	$\vee$	<code>\or</code>
$\rightarrow$	<code>\-></code>	$\leftrightarrow$	<code>\<-></code>	$\forall$	<code>\fo</code>
$\exists$	<code>\ex</code>	$\leq$	<code>\<=</code>	$\geq$	<code>\>=</code>
$\neq$	<code>\neq</code>	$\approx$	<code>\sim</code>	$\times$	<code>\x</code>
$\circ$	<code>\circ</code>	$\emptyset$	<code>\empty</code>	$\cup$	<code>\union</code>
$\cap$	<code>\intersect</code>	$\in$	<code>\in</code>	$\Downarrow$	<code>\downlefttharpoon</code>
$\bigcirc$	<code>\bigcirc</code>	$\leftarrow$	<code>\<-</code>	$\mapsto$	<code>\mapsto</code>
$\Rightarrow$	<code>\Rightarrow</code>	$\Rightarrow$	<code>\Rightarrow</code>	$\llbracket$	<code>\llbracket</code>
$\rrbracket$	<code>\rrbracket</code>	$\cdot$	<code>\cdot</code>	α	<code>\a</code>
β	<code>\b</code>	γ	<code>\g</code>	ε	<code>\e</code>
σ	<code>\s</code>	$\circ$	<code>\0</code>	$\subscript{1}$	<code>\1</code>
$\subscript{2}$	<code>\2</code>	$\subscript{3}$	<code>\3</code>	$\subscript{4}$	<code>\4</code>
$\subscript{5}$	<code>\5</code>	$\subscript{6}$	<code>\6</code>	$\subscript{7}$	<code>\7</code>
$\subscript{8}$	<code>\8</code>	$\subscript{9}$	<code>\9</code>		

第一部分

基础

类型与项

我们从学习 Lean 的基础知识开始，首先了解^{Term}项（也称为^{Expression}表达式）及其^{Type}类型。

Lean 的逻辑基础是一种丰富的逻辑，叫做^{Calculus of Inductive Construction}归纳构造演算，它支持^{Dependent Types}值类型。Lean 的逻辑灵感源自 λ -演算，类似于 Haskell、OCaml 和 Standard ML 等类型化函数式编程语言。即使您从未使用过这些语言，也能从现代语言（例如 C++、Java、Python）中找到许多类似的概念。大致来说：

Lean = 函数式编程 + 逻辑

如果你有数学背景，那么你可能已经了解过函数式编程的大部分关键概念了，有时它们有不同的名称。例如，Haskell 程序：

```
fib 0 = 0
fib 1 = 1
fib n = fib (n - 2) + fib (n - 1)
```

与数学定义非常吻合：

$$fib(n) = \begin{cases} 0 & \text{若 } n = 0 \\ 1 & \text{若 } n = 1 \\ fib(n-2) + fib(n-1) & \text{若 } n \geq 2 \end{cases}$$

在本章中，我们将关注 Lean 语言中一个不包含依值类型的部分，称为^{Simple Type Theory}简单类型论（或^{Higher-Order Logic}高阶逻辑）。这大致相当于简单类型的 λ -演算 [3]，并添加了一个相等运算符 (=)。它是一种抽象的、极简版的编程语言，其函数调用机制预示了函数式编程的出现。

1.1 类型

^{Type}类型可以是基本类型，例如 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 `Bool`，也可以是^{Total Function}全函数 $\sigma \rightarrow \tau$ ，其中 σ 和 τ 本身就是类型。类型指明了表达式可以求出何种值，它们引入了一个在数学中隐

式遵循的规则。原则上来说，没有什么可以阻止数学家陈述 $1 \in 2$ ，但类型规则会将其标记为可能有误。

从语义上讲，类型可以被视为集合。我们通常会定义类型 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 Bool ，以便它们忠实地刻画数学家的 $\mathbb{Z}$ 和 $\mathbb{Q}$ ，以及计算机科学家的布尔值，函数箭头 ($\rightarrow$) 也是如此。然而，尽管它们有相似之处，Lean 和数学却是截然不同的语言。Lean 的类型可以被^{Interpreted}解释为集合，但它们本身并非集合。

^{Higher-Order Type}
高阶类型是箭头 $\rightarrow$ 左侧包含嵌套的 $\rightarrow$ 的类型，例如类型 $(\mathbb{Z} \rightarrow \mathbb{Z}) \rightarrow \mathbb{Q}$ 。此类类型的值是接受其他函数作为参数的函数。因此， $(\mathbb{Z} \rightarrow \mathbb{Z}) \rightarrow \mathbb{Q}$ 是一个一元函数类型，它接受一个 $\mathbb{Z} \rightarrow \mathbb{Z}$ 类型的函数作为参数，并返回一个 $\mathbb{Q}$ 类型的值。

1.2 项

^{Term}
简单类型论的**项**，也称为表达式，包括以下几种：

- ^{Constant}**常量** c ;
- ^{Variable}**变量** x ;
- ^{Application}**应用** $t\ u$;
- ^{Anonymous Function}**匿名函数** $\text{fun } x \mapsto t$ (也称为 λ -表达式)。

上面的 t 和 u 表示任意项。我们也可以写作 $t : \sigma$ 来表示 t 具有类型 σ 。

每一种项的具体解释如下：

- 常量 $c : \sigma$ 是一个类型为 σ 的符号，它的含义在当前的全局^{Context}**语境**下是固定的。例如，一个算术理论可能包含常量 $0 : \mathbb{Z}$ 、 $1 : \mathbb{Z}$ 、 $\text{abs} : \mathbb{Z} \rightarrow \mathbb{N}$ 、 $\text{square} : \mathbb{N} \rightarrow \mathbb{N}$ 和 $\text{prime} : \mathbb{N} \rightarrow \text{Bool}$ 。常量包括函数（例如 abs ）和谓词（例如 prime ）。
- 变量 $x : \sigma$ 要么是^{Bound}**绑定的**，要么是^{Free}**自由的**。一个绑定的变量会引用一个匿名函数 $\text{fun } x : \sigma \mapsto t$ 中的输入 x 。反之，一个自由变量是在局部^{Context}**语境**下声明的，这一概念将在后面解释。
- 应用 $t\ u$ ，其中 $t : \sigma \rightarrow \tau$ 和 $u : \sigma$ ，是一个类型为 τ 的项，它表示将函数 t 应用于参数 u 的结果。例如，如果 t 是 abs ， u 是 0 ，则应用是 $\text{abs } 0$ 。除非它是一个复杂的项，否则不需要在参数周围加上括号，例如 $\text{prime } (\text{abs } 0)$ 。
- 给定一个项 $t : \tau$ ，一个匿名函数 $\text{fun } x : \sigma \mapsto t$ 表示一个类型为 $\sigma \rightarrow \tau$ 的^{Total Function}**全函数**，它将每个类型为 σ 的输入值 x 映射到函数体 t ，其中 x 可能出现在 t 中。因此， $\text{fun } x : \mathbb{Z} \mapsto \text{square } (\text{abs } x)$ 表示一个函数，它将 0 映射到 $\text{square } (\text{abs } 0)$ ，将 1 映射到 $\text{square } (\text{abs } 1)$ ，依此类推。变量 x 称为被 fun 所^{Binder}**绑定**。因此， fun 被称为**绑定器**。对于数学家来说，一个更熟悉的语法可能是 $x \mapsto \text{square } (\text{abs } x)$ ，没有 fun 关键字。

常量和变量在语法上看起来很相似，但它们被视为不同的。常量是全局声明的，而变量则通过 `fun` 或其他绑定器在局部引入。

应用和匿名函数互为镜像：匿名函数「构造」一个函数；应用则「解构」一个函数。如果我们将两者结合起来会发生什么？如果我们将一个类型为 σ 的参数 u 应用到匿名函数 $\text{fun } x : \sigma \mapsto t[x]$ ，其中 $t[x]$ 表示某个可能包含 x 的项，那么我们就得到项 $t[u]$ 。（这是一个稍微简化的视角。实际上，如果 $t[x]$ 包含一些绑定器，则可能需要重命名绑定变量以避免捕获 u 的自由变量。在 Lean 中，这种重命名会自动进行。）以下是一些应用匿名函数的示例：

```
(fun n : ℕ ↦ n) 4 产生 4
(fun n : ℕ ↦ square (square n)) 5 产生 square (square 5)
(fun y : ℤ ↦ 1) 0 产生 1
(fun x : ℤ ↦ (fun y : ℤ ↦ x)) 1 产生 fun y : ℤ ↦ 1
```

虽然我们的函数是一元函数（即它们只接受一个参数），但我们可以通过嵌套 `fun` 来构建 n 元函数，这需要使用一种名为^{Currying}柯里化的巧妙技巧。例如， $\text{fun } x : \sigma \mapsto (\text{fun } y : \tau \mapsto x)$ 表示类型为 $\sigma \rightarrow (\tau \rightarrow \sigma)$ 的函数，它接受两个参数并返回第一个参数。严格来说， $\sigma \rightarrow (\tau \rightarrow \sigma)$ 接受一个参数并返回一个函数，而该函数又接受一个参数。

应用的方式与之类似：若 $K := (\text{fun } x : \mathbb{Z} \mapsto (\text{fun } y : \mathbb{Z} \mapsto x))$ ，则 $K\ 1 = (\text{fun } y : \mathbb{Z} \mapsto 1)$ 且 $(K\ 1)\ 0 = 1$ 。函数 K 在 $K\ 1$ 中，如果只应用于一个参数，则被称为^{Partially Applied}部分应用，因为它可以接受一个额外的参数。

柯里化是一个非常有用的概念，我们可以省略大多数的括号：

```
 $\sigma \rightarrow \tau \rightarrow u$  表示  $\sigma \rightarrow (\tau \rightarrow u)$ 
 $t\ u\ v$  表示  $(t\ u)\ v$ 
 $\text{fun } x : \sigma \mapsto \text{fun } y : \tau \mapsto t$  表示  $\text{fun } x : \sigma \mapsto (\text{fun } y : \tau \mapsto t)$ 
```

我们还将使用以下缩写：

```
 $\text{fun } (x : \sigma) (y : \tau) \mapsto t$  表示  $\text{fun } x : \sigma \mapsto \text{fun } y : \tau \mapsto t$ 
 $\text{fun } x\ y : \sigma \mapsto t$  表示  $\text{fun } (x : \sigma) (y : \sigma) \mapsto t$ 
```

此外，在匿名函数 $\text{fun } x : \sigma \mapsto t$ 中，我们通常可以省略类型注解 $:\sigma$ ：

```
 $\text{fun } x \mapsto t$  表示  $\text{fun } x : \sigma \mapsto t$ 
```

然后，Lean 会尝试根据函数体 t 和匿名函数周围的语境推断其类型。类型推断可以简化符号，并节省一些按键。

在数学中，通常用中缀语法来书写二元运算符（例如， $x + y$ ）。这样的记法在 Lean 中也可以，它其实是柯里化函数应用的语法糖（例如，`add x y`）。

使用 Lean 的一种方法是用 `opaque` 命令声明我们需要的类型和常量。考虑以下声明：

```

opaque a : ℤ
opaque b : ℤ
opaque f : ℤ → ℤ
opaque g : ℤ → ℤ → ℤ

#check fun x : ℤ ↦ g (f (g a x)) (g x b)
#check fun x ↦ g (f (g a x)) (g x b)

```

前四行声明了四个常量 (a、b、f、g)，它们可用于构成项。最后两行使用 #check 命令对项进行类型检查并显示其类型。使用传统的数学符号，最后一行的项应写作 $x \mapsto g(f(g(a, x)), g(x, b))$ 。# 前缀表示诊断命令：这些命令可用于调试，但我们通常不会将它们保存在 Lean 文件中。

1.3 类型检查与类型推断

当 Lean 在解析一个项时，它会检查该项的类型是否正确。在此过程中，如果省略了绑定变量的类型，它就会尝试推断这些变量的类型，例如 `fun x ↦ square (square x)` 中 `x` 的类型。

对于简单类型论来说，类型检查和类型推断是可判定的问题。诸如重载（多个常量可以重用相同的名称，例如 $0 : \mathbb{N}$ 和 $0 : \mathbb{R}$ ）等高级特性可能会导致不可判定性 [23]。Lean 采用务实的、面向计算机科学的方法，并假设如果存在多种可能的类型，则数字 0、1、2、... 属于 $\mathbb{N}$ 类型。

Lean 的类型系统可以表示为一个形式系统。一个 ^{Formal System Judgment} **形式系统** 由 ^{Derivation Rule} **判断** 和用于生成判断的 **推导规则** 组成。类型判断的形式为 $C \vdash t : \sigma$ ，表示项 `t` 在局部语境 `C` 中具有类型 σ 。例如，判断 $\vdash \text{abs} : \mathbb{Z} \rightarrow \mathbb{N}$ 表示常数 `abs` 在空的局部语境中具有类型 $\mathbb{Z} \rightarrow \mathbb{N}$ 。

局部语境给出了 `t` 中未受 `fun` 绑定的变量的类型。它用于跟踪由 `t` **外部** 的绑定器绑定的变量。如果同一个变量 `x` 被绑定多次，则最后一次出现的变量会覆盖其他变量。例如，判断 $x : \mathbb{Z}, y : \mathbb{N}, y : \mathbb{Z} \vdash y : \mathbb{Z}$ 表示变量 `y` 在局部语境 $x : \mathbb{Z}, y : \mathbb{N}, y : \mathbb{Z}$ 中具有类型 $\mathbb{Z}$ 。

对于简单类型论而言，类型判断由四条类型规则产生，每种项对应一条：

$$\begin{array}{c}
 \frac{}{C \vdash c : \sigma} \text{CST} \quad \text{当 } c \text{ 是全局声明的，类型为 } \sigma \text{ 的常量} \\
 \\
 \frac{}{C \vdash x : \sigma} \text{VAR} \quad \text{当 } x : \sigma \text{ 是出现在 } C \text{ 中最右边的 } x \\
 \\
 \frac{C \vdash t : \sigma \rightarrow \tau \quad C \vdash u : \sigma}{C \vdash t u : \tau} \text{APP}
 \end{array}$$

$$\frac{C, x : \sigma \vdash t : \tau}{C \vdash (\text{fun } x : \sigma \mapsto t) : \sigma \rightarrow \tau} \text{FUN}$$

每条规则都有零个或多个前提（水平线上方）、一个结论（水平线下方），以及可能的一个附加条件（水平线右侧）。前提是类型判断，而附加条件则是规则中出现的数学变量的任意数学条件。为了证明前提，需要继续向上推导，我们稍后就会看到。至于附加条件，我们可以运用所有数学知识来证明它们为真。

前两条规则分别标记为 CST 和 VAR，没有前提，但它们有附加条件，这些条件必须满足才能应用规则。后两条规则以一或两个判断作为前提，并产生一个新的判断。FUN 是唯一修改局部上下文的规则：当我们进入一个匿名函数的函数体 t 时，我们需要记录绑定变量 x 的存在，以及它的类型，以便在 t 中遇到 x 时做好准备。

我们可以使用此规则系统来证明给定项的类型正确，方法是：通过逆向（即向上）^{Formal Derivation} 推导并应用规则，构建一个类型判断的**形式推导**。与现实中的树一样，推导树的根节点位于底部。推导的判断出现在根节点处，每个分支都以应用一个无前提的规则结束。规则的应用由水平线和标签表示。以下类型推导确定项 $\text{fun } x : \mathbb{Z} \mapsto \text{abs } x$ 在任意局部上下文 C 中具有类型 $\mathbb{Z} \rightarrow \mathbb{N}$ ：

$$\frac{\frac{}{C, x : \mathbb{Z} \vdash \text{abs} : \mathbb{Z} \rightarrow \mathbb{N}} \text{CST} \quad \frac{}{C, x : \mathbb{Z} \vdash x : \mathbb{Z}} \text{VAR}}{C, x : \mathbb{Z} \vdash \text{abs } x : \mathbb{N}} \text{APP} \quad \frac{}{C \vdash (\text{fun } x : \mathbb{Z} \mapsto \text{abs } x) : \mathbb{Z} \rightarrow \mathbb{N}} \text{FUN}$$

从根节点向上阅读证明，注意局部语境是如何贯穿始终的，以及它是如何被 FUN 规则扩展的。该规则将 fun 绑定的变量移动到局部语境，使得 VAR 的应用可以在树的上层进行。如果变量 x 已经在 C 中声明，则在进入 fun 表达式后，它将被 $x : \mathbb{Z}$ 覆盖。

总而言之，类型系统由推导规则组成，这些规则可以 (1) 用其数学变量的任意值进行实例化，以及 (2) 连接起来形成推导树。

下面是第二个示例，这次从一个空的局部语境开始：

$$\frac{\frac{\frac{}{x : \mathbb{Z}, y : \mathbb{Z} \vdash x : \mathbb{Z}} \text{VAR}}{x : \mathbb{Z} \vdash (\text{fun } y : \mathbb{Z} \mapsto x) : \mathbb{Z} \rightarrow \mathbb{Z}} \text{FUN}}{\vdash (\text{fun } x : \mathbb{Z} \mapsto \text{fun } y : \mathbb{Z} \mapsto x) : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}} \text{FUN}$$

到目前为止，两个例子都是**Well-Typed** **类型良好的**。如果我们从一个类型错误的项开始，或者在推导树根部的判断中指定了错误的类型或语境，我们就会发现无法完成推导。推导构成了对项而言类型良好的证明，并且在给定的语境中具有指定的类型。

上述类型系统仅检查项是否类型良好，并不检查类型是否^{Well-formed}形式良好。例如，给定一元类型构造子 `List`, `List ℤ`（整数列表的类型）是形式良好的，而 `ℤ List` 和 `List List` 是^{Ill-formed}形式不良的。对于简单类型理论，形式良好性检查很容易：只应使用声明的类型构造子，并且每个 n 元类型构造子都应传递恰好为 n 类型的参数。

1.4 类型居留

给定一个类型 σ , ^{Type Inhabitation}类型居留问题是指在空的局部语境中寻找该类型的一个^{Inhabitant}居留元，即一个类型为 σ 的项。这练习看似毫无意义，但正如我们将在第 4 章中看到的那样，这个问题与寻找命题的证明密切相关。类似「寻找一个类型为 σ 的项」这种看似简单的练习，其实是掌握定理证明的好方法。

虽然这个问题总体上是不可判定的，但只要策略得当，我们就能取得很大进展。要创建给定类型的项，请从占位符 `_` 开始，并递归地应用以下两个步骤的组合：

1. 如果类型的形式为 $\sigma \rightarrow \tau$ ，则可能的居留项是一个匿名函数，形式为 `fun x :  $\sigma \mapsto \_$` ，其中 `_` 是一个类型为 τ 的缺失项的占位符。Lean 会将 `_` 标记为错误；如果在 Visual Studio Code 中将鼠标悬停在该项上，提示将显示缺失项的类型以及在局部语境中声明的任何变量。
2. 给定一个类型 σ （可以是函数类型），你可以使用任何常量 c 或类型为 $\tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow \sigma$ 的变量 x 来构建一个类型为 σ 的项。对于每个参数，你需要放置一个占位符，从而得到 `$c \_ \dots \_$` 或 `$x \_ \dots \_$` 。

占位符可以递归地使用相同的策略来消除。

作为示例，我们将应用该策略来查找以下类型的项：

$$(\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow ((\beta \rightarrow \alpha) \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$$

最初，只有步骤 1 适用，其中：

$$\sigma := \alpha \rightarrow \beta \rightarrow \gamma \quad \text{且} \quad \tau := ((\beta \rightarrow \alpha) \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$$

（回想一下， $\rightarrow$ 是右结合的。）这得到了项 `fun f  $\mapsto \_$` ，它拥有正确的类型，但左边有一个占位符。由于参数 f 的类型为 σ ，它是一个函数类型，因此使用名称 f 来表示它是合理的。然后，我们继续递归地处理类型为 τ 的占位符。同样，只有步骤 1 可行，因此我们最终得到项 `fun f g  $\mapsto \_$` ，其中 g 的类型为 $(\beta \rightarrow \alpha) \rightarrow \beta$ ，而占位符的类型为 $\alpha \rightarrow \gamma$ 。步骤 1 的第三次应用得到 `fun f g a  $\mapsto \_$` ，其中 a 的类型为 α ，占位符的类型为 γ 。

此时，步骤 1 不再适用。让我们看看步骤 2 是否可行。占位符周围的语境包含以下变量：

$$f : \alpha \rightarrow \beta \rightarrow \gamma, \quad g : (\beta \rightarrow \alpha) \rightarrow \beta, \quad a : \alpha$$

回想一下，我们正在尝试构建一个 γ 类型的项。我们唯一能用来实现这一点的变量是 f ：它接受两个参数并返回一个 γ 类型的值。因此，我们用项 $f _ _$ 替换占位符，其中，两个新的占位符代表两个缺失的参数。将所有内容放在一起，我们现在得到了项 $\text{fun } f \text{ } g \text{ } a \mapsto f _ _$ 。

根据 f 的类型，占位符的类型分别为 α 和 β 。第一个占位符很容易填充，同样使用步骤 2，只需提供 α 类型的 a ，不带任何参数即可。对于第二个占位符，我们将步骤 2 与变量 g 结合使用，它是 β 的唯一来源。由于 g 接受一个参数，因此我们必须提供一个占位符。这意味着我们当前的项是 $\text{fun } f \text{ } g \text{ } a \mapsto f \text{ } a \text{ } (g _)$ 。

我们快完成了。剩下的唯一占位符的类型是 $\beta \rightarrow \alpha$ ，它是 g 的参数类型。应用步骤 1，我们将占位符替换为 $\text{fun } b \mapsto _$ ，其中 $_$ 的类型为 α 。在这里，我们可以简单地提供 a 。我们的最终项是 $\text{fun } f \text{ } g \text{ } a \mapsto f \text{ } a \text{ } (g \text{ } (\text{fun } b \mapsto a))$ 。

上述推导过程冗长乏味，但却具有确定性：在每个点，步骤 1 或步骤 2 中的其中一项适用，但并非同时适用。一般来说，情况并非总是如此。对于其他一些类型，我们可能会遇到死胡同，需要回溯。我们也可能会完全失败，无处可回溯。

关键在于，项应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发软件的一个好的原则是，从可以编译的程序开始，进行尽可能小的更改，以获得新的可以编译的程序，并重复此过程，直到程序完成。

1.5 新引入的 Lean 结构小结

在本章和大多数其他章节的末尾，我们都会简要概述本章引入的构造。某些语法具有多重含义，我们将逐步介绍。有关详细信息，请参阅 *Lean 4 参考手册* [1]、*Lean 4 定理证明* [?] 教程以及 *mathlib* 文档¹。

诊断命令

`#check` 检查并打印一个项的类型

声明

`opaque` 声明一个未指定的新常量或类型

¹https://leanprover-community.github.io/mathlib4_docs/

程序与定理

我们继续学习 Lean 的基础知识，重点关注程序和定理，但暂不进行任何证明。具体来说，我们将回顾如何定义新的类型和函数，以及如何将其预期的性质表述为定理。

2.1 类型定义

Lean 归纳构造演算的一个显著特点是它内置了对归纳类型的支持。^{Inductive Type}归纳类型是一种类型，其值是通过应用称为^{Constructor}构造子的特殊常量来构建的。归纳类型是在程序中表示非循环数据的一种简洁方式。你可能还知道一些其他的、大致相同的名称，包括^{Algebraic Data Type}代数数据类型、^{Inductive Data Type}归纳数据类型、^{Freely Generated Data Type}自由生成的数据类型、^{Recursive Data Type}递归数据类型和^{Data Type}数据类型。

2.1.1 自然数的定义

归纳类型的「Hello, World!」示例是自然数类型 `Nat` ($\mathbb{N}$)。在 Lean 中，它可以定义如下：

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

第一行向世界声明我们引入了一个名为 `Nat` 的新类型，用于表示自然数。第二行和第三行声明了两个新的构造子，`Nat.zero : Nat` 以及 `Nat.succ : Nat → Nat`，它们可用于构造类型为 `Nat` 的值。按照计算机科学和逻辑学中既定的惯例，计数从 0 开始。第二个构造子让这个归纳定义变得有趣——它需要一个类型为 `Nat` 的参数来产生一个类型为 `Nat` 的值。以下这些项

`Nat.zero`

```
Nat.succ Nat.zero
Nat.succ (Nat.succ Nat.zero)
⋮
```

表示类型为 `Nat` 的不同值——`o`、`o` 的后继数、`o` 的后继数的后继数，以此类推。这种记法被称为 ^{Unary}一进制记法，也叫 ^{Peano}皮亚诺记法，以逻辑学家 ^{Giuseppe Peano}朱塞佩·皮亚诺的名字命名。若想了解在 Lean 中对皮亚诺数的另一种解释（还有一些令人惊叹的电子游戏画面），请参阅凯文·巴扎德（Kevin Buzzard）的文章《计算机能证明定理吗？》。¹

类型声明的通用格式为

```
inductive 类型名 (参数1 : 类型1) ... (参数k : 类型k) : Type
where
  | 构造子名称1 : 构造子类型1
  |
  | 构造子名称n : 构造子类型n
```

对于自然数，我们可以让 Lean 使用大家熟悉的名称 `0`、`1`、`2` 等等，实际上，预定义的 `ℕ` 类型就提供了这样的语法糖。不过，使用更详细的表示法能让我们更清楚地了解 Lean 的类型定义。

在本章配套的 Lean 文件中，`Nat` 的定义被放在一个命名空间内，通过 `namespace MyNat` 和 `end MyNat` 进行界定，以将其作用域限制在文件的某一部分。在 `end MyNat` 之后，出现的所有 `Nat`、`Nat.zero` 或 `Nat.succ` 都将指向 Lean 预定义的自然数类型。同样，整个文件被放在 `LoVe` 命名空间下，以避免与 Lean 现有库中的名称冲突。

我们可以在 Lean 的任何位置使用 `#print` 命令来查看之前的定义。例如，在 `MyNat` 命名空间内输入 `#print Nat`，会显示如下信息：

```
inductive LoVe.MyNat.Nat : Type
number of parameters: 0
constructors:
LoVe.MyNat.Nat.zero : Nat
LoVe.MyNat.Nat.succ : Nat → Nat
```

本书侧重于自然数，这也是本书许多内容偏向计算机科学的体现之一。对于数论学家来说，他们可能更关注整数 $\mathbb{Z}$ 和有理数 $\mathbb{Q}$ ；分析学家则更希望处理实数 $\mathbb{R}$ 和复数 $\mathbb{C}$ 。然而，自然数在计算机科学中无处不在，并且作为归纳类型有着非常简单的定义。正如我们将在第 ?? 章中看到的，自然数还可以用来构建其他类型。

¹<http://chalkdustmagazine.com/features/can-computers-prove-theorems/>

2.1.2 算数表达式的定义

下一个例子会用到整数。如果我们要实现一个计算器程序或一种编程语言，那么就需要定义一种类型，用来将算术表达式表示为抽象语法树。下面的例子展示了如何在 Lean 中实现这一点：

```
inductive AExp : Type where
  | num : ℤ → AExp
  | var : String → AExp
  | add : AExp → AExp → AExp
  | sub : AExp → AExp → AExp
  | mul : AExp → AExp → AExp
  | div : AExp → AExp → AExp
```

从数学上讲，该定义等价于用以下生成规则来归纳地定义类型 AExp：

1. 对于每个整数 i ，项 $\text{AExp.num } i$ 都是一个 AExp 值。（直观上，构造子 AExp.num 就是把一个整数「包装」为一个算术表达式。）
2. 对于每个字符串 x ，项 $\text{AExp.var } x$ 都是一个 AExp 值。
3. 若 e_1 和 e_2 都是 AExp 值，那么 $\text{AExp.add } e_1 \ e_2$ 、 $\text{AExp.sub } e_1 \ e_2$ 、 $\text{AExp.mul } e_1 \ e_2$ 和 $\text{AExp.div } e_1 \ e_2$ 也都是 AExp 值。

上述定义穷尽了所有情况。AExp 所有可能的取值都只能通过生成规则 1 到 3 构造得到。此外，使用不同生成规则构造出的 AExp 值彼此不同。这两条归纳类型的性质可以用 Joseph Goguen 提出的口号来概括：「无垃圾，无混淆」。

将上面 Lean 中简洁的 AExp 定义与实现相同功能的 Java 程序作对比也许会很有启发意义。该 Java 程序包含一个接口和六个实现该接口的类，分别对应于 AExp 类型及其六个构造子：

```
public interface AExp { }

public class Num implements AExp {
    public int num;

    public Num(int num) { this.num = num; }
}

public class Var implements AExp {
    public String var;

    public Var(String var) { this.var = var; }
}
```

```
public class Add implements AExp {
    public AExp left;
    public AExp right;

    public Add(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Sub implements AExp {
    public AExp left;
    public AExp right;

    public Sub(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Mul implements AExp {
    public AExp left;
    public AExp right;

    public Mul(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Div implements AExp {
    public AExp left;
    public AExp right;

    public Div(AExp left, AExp right)
    { this.left = left; this.right = right; }
}
```

在 Python 中，同样的类声明如下：

```
class AExp:
    pass

class Num(AExp):
```

```

def __init__(self, num):
    self.num = num

class Var(AExp):
    def __init__(self, var):
        self.var = var

class Add(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Sub(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Mul(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Div(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

```

2.1.3 列表的定义

我们考虑的下一个类型是有限列表：

```

inductive List ( $\alpha$  : Type) where
| nil   : List  $\alpha$ 
| cons  :  $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$ 

```

Polymorphism

该类型是**多态的**：它由一个类型参数 α 参数化，我们可以用具体类型对其进行实例化。例如， $\text{List } \mathbb{Z}$ 表示整数列表的类型， $\text{List } (\text{List } \mathbb{R})$ 表示实数列表的列表类型。类型构造子 `List` 以一个类型作为参数并返回一个类型。类似于 Java 的泛型和 C++ 的模板，多态性是一种提供参数化类型的机制。

以下命令显示了构造子的类型：

```
#check List.nil
#check List.cons
```

Lean 的内置列表提供了语法糖来编写列表：

```
[] 表示 List.nil
x :: xs 表示 List.cons x xs
[x1, ..., xn] 表示 x1 :: ... :: xn :: []
```

和所有其他二元运算符一样，`::` 运算符的结合优先级低于函数应用。因此，`f x :: List.reverse ys` 会被解析为 `(f x) :: (List.reverse ys)`。避免不必要的括号是一种良好的编程习惯，因为括号会降低可读性。此外，在中缀运算符两侧加上空格，有助于表达正确的优先级。

函数式程序员通常会用 `xs`、`ys`、`zs` 这样的名字来表示列表，虽然在 Lean 中 `l` 也很常见。由于列表包含多个元素，使用复数形式是很自然的。例如，猫的列表可以命名为 `cats`，猫的列表的列表可以命名为 `catss`。当我们将一个非空列表表示为表头和表尾时，通常会写作 `x :: xs` 或 `cat :: cats`。

2.2 函数定义

如果我们只想声明一个函数，那么可以使用 `opaque` 命令（章节 1.2）。但通常来说，我们想要**定义**函数的行为，那么可以使用 `def` 命令。由于 Lean 的根基是函数式变成，函数会定义为可求值的数学表达式，而非修改某些状态的指令时程序。与此相应，Lean 并不使用 `while` 或 `for` 来迭代，而是使用递归作为遍历数据的主要机制。

2.2.1 在自然数上递归

我们来看一个简单的例子，看看递归是如何工作的。斐波那契数的 Lean 定义如下：

```
def fib : ℕ → ℕ
| 0      => 0
| 1      => 1
| n + 2 => fib (n + 1) + fib n
```

左侧的模式对应于函数的参数。在这里，`fib` 的声明中只有一个类型为 `ℕ` 的参数，因此我们只需对一个自然数进行**模式匹配**。当输入具有给定的形式时，相应的模式匹配就会被触发。例如，如果输入是 1，那么模式 1 就会匹配它，并计算对应的右

侧表达式。如果输入是 5，则会触发模式 $n + 2$ ，此时 $n := 3$ 。然后将 n 取值为 3 计算右侧表达式，得到 $\text{fib } 4 + \text{fib } 3$ 。

递归定义的一般形式为：

```
def 名称(参数1 : 类型1) ... (参数m : 类型m) : 类型
  | 模式1 => 结果1
    .....
  | 模式n => 结果n
```

参数 参数₁ 到 参数_m 不能用于模式匹配，只有在 类型 中声明的剩余参数才能进行模式匹配。如果一行中提供了多个模式，它们之间用逗号分隔。模式中 can 包含变量，这些变量在对应的右侧表达式中是可见的，也可以包含构造子。

基本的自然数算术运算，如加法、乘法和幂运算，都可以通过递归来定义。当然，这些运算在 Lean 中已经预定义了（分别为 $+$ 、 $*$ 和 $^$ ），但自己实现一遍也是很有意义的练习。我们先从加法开始：

```
def add : ℕ → ℕ → ℕ
  | m, Nat.zero    => m
  | m, Nat.succ n => Nat.succ (add m n)
```

我们对两个参数同时进行模式匹配，区分第二个参数为零和非零的情况。每一次对 `add` 的递归调用，都会从第二个参数中剥离出一个 `Nat.succ` 构造子。Lean 允许我们用 `0` 和 `n + 1` 这样的语法糖来代替 `Nat.zero` 和 `Nat.succ n`。

我们可以使用 `#eval` 或 `#reduce` 来计算 `add` 应用于数字的结果：

```
#eval add 2 7
#reduce add 2 7
```

两个命令都会在 Visual Studio Code 中打印出 9。`#eval` 会使用一个经过优化的解释器，而 `#reduce` 则会使用 Lean 的推理内核，它的效率更低。

当定一个函数时，最佳实践是每次提供一些测试来确保函数的行为符合预期。你甚至可以在 Lean 文件中保留 `#eval` 或 `#reduce` 作为文档。

乘法的定义与加法类似：

```
def mul : ℕ → ℕ → ℕ
  | _, Nat.zero    => Nat.zero
  | m, Nat.succ n => add m (mul m n)
```

下划线 (`_`) 表示未使用的变量。我们本可以使用一个名字（例如 `m`），但 `_` 能更好地记录我们的意图。

下面的 `#eval` 命令会打印 14，符合预期：

```
#eval mul 2 7
```

指数（「 m 的 n 次方」）有多种定义的方式。我们的第一个方案在结构上与乘法的定义完全相同：

```
def power : ℕ → ℕ → ℕ
| _, Nat.zero    => 1
| m, Nat.succ n => mul m (power m n)
```

我们可以将第一个参数 m 提取出来，并作为 参数 放在函数名旁边，位于引入函数类型的冒号之前（排除该参数 m ）：

```
def powerParam (m : ℕ) : ℕ → ℕ
| Nat.zero    => 1
| Nat.succ n => mul m (powerParam m n)
```

从外部看，这两个定义没有区别。事实上，我们已经在 List 构造子的类型参数 α 中见过这种语法了（第 2.1 节）。

还有另一种定义方式，即先引入一个通用的迭代器，然后使用正确的参数来调用它：

```
def iter (α : Type) (z : α) (f : α → α) : ℕ → α
| Nat.zero    => z
| Nat.succ n => f (iter α z f n)

def powerIter (m n : ℕ) : ℕ :=
  iter ℕ 1 (mul m) n
```

`iter` 函数接收一个类型 α ，一个「零」值 z ，一个一元函数 f ，以及一个自然数 n ，并计算出

$$\underbrace{f(f(\dots(f z) \dots))}_{n \text{ 次}}$$

它是高阶函数的一个例子：一种将另一个函数（此处为 f ）作为参数的函数。在函数式编程中，函数被视为与其他对象一样的普通对象，可以用作参数或函数的返回结果。在这里，我们使用 `iter` 来计算 m 的 n 次幂：

$$\underbrace{\text{mul } m (\text{mul } m (\dots (\text{mul } m 1) \dots))}_{n \text{ 次}}$$

最后，请注意 `powerIter` 的定义不是递归的。对于不使用模式匹配的非递归函数，其语法简单如下：

```
def name (参数1 : 类型1) ... (参数m : 类型m) : 类型 :=
  结果
```

2.2.2 算术表达式上的递归

回到算术表达式类型，如果我们想在 Java 中实现一个 `eval` 函数，我们可能会将其作为 `AExp` 接口的一部分，并在每个子类中实现它。对于 `Add`、`Sub`、`Mul` 和 `Div`，我们会递归地在 `left` 和 `right` 对象上调用 `eval`。

在 Lean 中，其语法非常简洁。我们定义一个函数，并使用模式匹配来区分这六种情况：

```
def eval (env : String → ℤ) : AExp → ℤ
| AExp.num i      => i
| AExp.var x      => env x
| AExp.add e1 e2 => eval env e1 + eval env e2
| AExp.sub e1 e2 => eval env e1 - eval env e2
| AExp.mul e1 e2 => eval env e1 * eval env e2
| AExp.div e1 e2 => eval env e1 / eval env e2
```

请注意，此函数是高阶函数：它接收一个函数 `env`，该函数表示一个将值分配给变量的「Environment环境」。在 `AExp.var` 情况下，环境用于求值变量，并在递归情况 (`AExp.add`、`AExp.sub`、`AExp.mul` 和 `AExp.div`) 中传递。

也许你会担心 `AExp.div` 情况下的除以零问题。让我们看看 `#eval` 对此有何评价，使用一个将 7 分配给所有变量的环境：

```
#eval eval (fun x ↦ 7) (AExp.div (AExp.var "y") (
  AExp.num 0))
```

输出结果是 0。在 Lean 中，除法被方便地定义为一个全函数，当分母为零时返回零。关于为什么这并不危险的清晰解释，请参阅 Buzzard 的博客。²

2.2.3 列表上的递归

列表上的递归函数可以用类似的方式定义：

```
def append (α : Type) : List α → List α → List α
| List.nil,      ys => ys
| List.cons x xs, ys => List.cons x (append α xs ys)
```

`append` 函数接受三个参数：一个类型 α 和两个类型为 `List  $\alpha$` 的列表。

```
#eval append ℕ [3, 1] [4, 1, 5]
#eval append _ [3, 1] [4, 1, 5]
```

通过传入占位符 `_`，我们让 Lean 从其他两个参数的类型中推断出类型 $\mathbb{N}$ 。

为了使类型参数 α 成为隐式参数，我们可以将其放在花括号 `{ }` 中：

²<https://xenaproject.wordpress.com/2020/07/05/division-by-zero-in-type-theory-a-faq/>

```
def appendImplicit {α : Type} : List α → List α → List
  α
| List.nil,      ys => ys
| List.cons x xs, ys => List.cons x (appendImplicit xs
  ys)
```

```
#eval appendImplicit [3, 1] [4, 1, 5]
```

@ 符号可用于使隐式参数显式化。有时这对于指导 Lean 的解析器是必要的。

```
#eval @appendImplicit N [3, 1] [4, 1, 5]
```

```
#eval @appendImplicit _ [3, 1] [4, 1, 5]
```

我们可以在定义中使用语法糖，无论是在 => 左侧的模式中，还是在右侧：

```
def appendPretty {α : Type} : List α → List α → List α
| [],      ys => ys
| x :: xs, ys => x :: appendPretty xs ys
```

在 Lean 的标准库中，追加函数是一个名为 ++ 的中缀运算符。我们可以用它来定义一个反转列表的函数：

```
def reverse {α : Type} : List α → List α
| []      => []
| x :: xs => reverse xs ++ [x]
```

2.3 定理陈述

Lean 既是证明助手又是编程语言的原因在于，我们可以对定义的类型和常量陈述定理，并证明它们成立。在交互式定理证明中，术语**定理**、**引理**、**推论**、**事实**、**性质**以及**真陈述**的使用或多或少是可以互换的。类似地，**命题**、**逻辑公式**和**陈述**的意思也都是一样的。

在 Lean 中，命题仅仅是 Prop 类型的项。这与一阶逻辑形成对比，在一阶逻辑中，传统上在语法中区分项和公式。可以证明的命题被称为定理（或引理、推论等）；否则，它就是一个非定理或假陈述。数学家有时将术语「命题」作为定理的同义词（例如，「命题 3.14」），但在形式逻辑中，命题也可以是假的。

以下是关于第 2.2 节中定义的增加、乘法和列表反转操作的真陈述示例：

```
theorem add_comm (m n : N) :
  add m n = add n m :=
  sorry
```

```

theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
  sorry

theorem mul_comm (m n : ℕ) :
  mul m n = mul n m :=
  sorry

theorem mul_assoc (l m n : ℕ) :
  mul (mul l m) n = mul l (mul m n) :=
  sorry

theorem mul_add (l m n : ℕ) :
  mul l (add m n) = add (mul l m) (mul l n) :=
  sorry

theorem reverse_reverse {α : Type} (xs : List α) :
  reverse (reverse xs) = xs :=
  sorry

```

其一般形式为

```

theorem name (参数1 : 类型1) ... (参数m : 类型m) :
  陈述 :=
  证明

```

`:=` 符号将定理的陈述与其证明隔开。`theorem` 的语法与没有模式匹配的 `def` 命令非常相似，用 `statement` 代替 `type`，用 `proof` 代替 `result`。在上面的例子中，我们放置了标记 `sorry` 作为实际证明的占位符。这个标记字面上是向未来的读者和 Lean 表示道歉，因为缺乏证明。这也是值得担心的一个理由，直到我们设法消除它。在第 3 章和第 4 章中，我们将看到如何实现这一点。

带有 `sorry` 证明的 `theorem` 命令的直观语义是：「这个命题应该是可以证明的，但我还没有进行证明——抱歉。」有时，我们想表达一个相关的想法，即：「让我们假设这个命题成立。」Lean 为此提供了 `axiom` 命令，通常与 `opaque` 结合使用。例如：

```

opaque a : ℤ
opaque b : ℤ

axiom a_less_b :
  a < b

```

在 `opaque` 命令之后，除了它们的类型之外，我们没有关于 `a` 和 `b` 的任何信息。公理指定了它们应该具有的性质。该命令的通用格式为：

```
axiom name (参数1 : 类型1) ... (参数m : 类型m) :  
    陈述
```

公理是危险的，因为它们可能导致逻辑不一致，从而使我们能够推导出 `False`。例如，如果我们添加第二个公理陈述 `a > b`，我们就可以很容易地推导出 `False`。交互式定理证明的历史充满了不一致的公理。这里有一个轶事：在 2020 年的「认证程序与证明」会议上，一篇提交的论文因为一个有缺陷的公理而被拒绝，其中一位同行评审员从中推导出了 `False`。³因此，我们通常会避免使用公理。

从 Lean 的角度来看，带有 `sorry` 证明的定理实际上就是一个公理，可能会破坏逻辑的一致性。为了防止误解，最好只在开发证明时将 `sorry` 作为临时措施，而不是将其作为更明确且诚实的 `axiom` 的替代品。

2.4 新引入的 Lean 结构小结

诊断命令

<code>#eval</code>	使用优化的解释器执行一个项
<code>#print</code>	打印一个常量的定义
<code>#reduce</code>	使用 Lean 的 Inference Kernel 推断内核执行一个项

声明

<code>axiom</code>	陈述一个公理
<code>def</code>	定义一个新的常量
<code>inductive</code>	引入一个类型及其构造子
<code>namespace ... end</code>	Named Scope 在命名作用域内收集声明
<code>theorem</code>	陈述一个定理及其证明

证明命令

<code>sorry</code>	代表缺失的证明或定义
--------------------	------------

³该论文的一位作者报告说，该公理现在已经经过修订并作为定理推导出来。整个形式化开发现在已不含公理。修订后的论文已在「计算机辅助验证 2020」会议上被接受 [7]。

逆向证明

Tactic **策略**是在^{Goal}**目标**(即要证明的^{Proposition}**命题**)上操作的过程, 要么完全证明它, 要么产生新的子目标, 或者失败。当我们陈述一个定理时, 定理的陈述就是初始目标。一旦使用合适的策略消除了所有(子)目标, 证明就完成了。此处介绍的大多数策略在^{Theorem Proving in Lean 4}**《Lean 4 定理证明》**第 5 章中有更详细的说明 [12]。

Tactic **策略**是一种^{Backward}**逆向**证明机制。它们从^{Goal}**目标**出发, 向前推进到已证明的定理。考虑定理 a 、 $a \rightarrow b$ 、 $b \rightarrow c$ 以及目标 $\vdash c$ 。与类型判断类似, 目标用符号 $\vdash$ 标识。一个非形式的逆向证明如下:

为了证明 c , 由 $b \rightarrow c$ 可知, 只需证明 b 即可。
为了证明 b , 由 $a \rightarrow b$ 可知, 只需证明 a 即可。
为了证明 a , 我们直接使用 a 。 □

^{Backward Proof}**逆向证明**的显著特征是使用了「^{it suffices to}**只需证明**」这一短语。请注意我们是如何从一个^{Goal}**目标**变换到另一个目标 ($\vdash c$ 、 $\vdash b$ 、 $\vdash a$), 直到没有目标需要证明为止的。相比之下, ^{Forward Proof}**正向证明**会从定理 a 开始, 每次处理一个定理, 最终推导出所需的定理 c :

由 a 与 $a \rightarrow b$ 可得 b 。
由 b 与 $b \rightarrow c$ 可得 c , 正如预期。 □

^{Forward Proof}**正向证明**只操作^{Theorem}**定理**, 不操作^{Goal}**目标**。我们将在第 4 章深入学习正向证明。
非形式化的证明有时会混合使用这两种风格。只要能通过「为了证明....., 只需证明.....」之类的措辞清晰地识别出^{Backward}**逆向**步骤, 这种混合就是可行的。在目标前面加上 $\vdash$ 符号有助于提醒这些断言尚未被证明。

你可能熟悉的另一种表达证明的格式是^{Natural Deduction}**自然演绎**。下面给出了对应于上述证明的自然演绎推导:

$$\begin{array}{c}
 \frac{\frac{\frac{}{\vdash b \rightarrow c}}{\vdash b \rightarrow c} \quad \frac{\frac{\vdash a \rightarrow b \quad \vdash a}{\vdash b} \rightarrow E}{\vdash c} \rightarrow E
 \end{array}$$

从上往下读时，该推导对应于^{Forward Proof}**正向证明**；从下往上读时，它对应于^{Backward Proof}**逆向证明**。

3.1 策略模式

在第 2 章中，每当需要证明时，我们只是简单地使用了 `sorry` 占位符。对于^{Tactical Proof}**策略证明**，我们现在将写下 `by` 来进入^{Tactic Mode}**策略模式**。在此模式下，我们可以调用一系列^{Tactic}**策略**。

策略在目标上操作，目标由我们想要证明的命题 Q 和^{Local Context}**局部语境** C 组成。局部语境由形如 $x : \sigma$ 的变量声明和形如 $h : P$ 的^{Hypothesis}**假设**组成。我们用 $C \vdash Q$ 表示一个目标，其中 C 是变量和假设的列表，而 Q 是^{Goal}**目标**的^{Target}**目的**。

为了让事情更具体，请考虑以下 Lean 示例：

```

theorem fst_of_two_props :
  ∀ a b : Prop, a → b → a :=
  by
    intro a b
    intro ha hb
    apply ha
  
```

请注意，蕴含箭头 $\rightarrow$ 是右结合的；这意味着 $a \rightarrow b \rightarrow a$ 与 $a \rightarrow (b \rightarrow a)$ 相同。直观地说，该陈述的意思是「 a 蕴含了 b 蕴含 a 」，或者等价地，「 a 和 b 蕴含 a 」。

我们将逐行审阅这个证明。

`by`

^{Keyword}**关键字** `by` 表示我们进入了^{Tactic Mode}**策略模式**，在该模式下我们可以通过^{Tactic}**策略**来指定证明。随后是策略，每行一个，并且所有策略的缩进相同。

如果我们将光标置于 `by` 关键字上，Visual Studio Code 会报告当前的目标仅仅是原始定理的陈述：

$\vdash \forall a b : \text{Prop}, a \rightarrow b \rightarrow a$

接下来的策略

`intro a b`

告诉 Lean 固定两个^{Free Variable}自由变量 a 和 b ，它们对应于同名的两个^{Bound Variable}绑定变量。该策略模仿了数学家在纸上的工作方式：为了证明一个 $\forall$ 量化的命题，只需证明该命题对于绑定变量的某些任意但固定的值成立即可。目标变为：

$$a\ b : \text{Prop} \vdash a \rightarrow b \rightarrow a$$

其中命题 a 和 b 在目标的^{Local Context}局部语境(即 $\vdash$ 符号左侧)中声明。我们通常为自由变量使用与绑定变量相同的名称，正如这里所做的，但这并不是强制性的。通常，使用唯一的名称并避免遮蔽现有变量是良好的做法。

除了使用两个变量名调用一次 `intro` 之外，我们也可以调用两次 `intro`，即 `intro a` 后面跟着 `intro b`。通常，每个带有 n 个参数的 `intro` 调用都会提取接下来的 n 个量化的变量。

`intro` 策略不仅限于量化变量。在我们的证明脚本中，

```
intro ha hb
```

告诉 Lean 将 (来自 $a \rightarrow b \rightarrow$ 的) 前提 a 和 b 移至局部语境，并称这些假设为 ha 和 hb 。

事实上，为了证明一个蕴含式，只需将其左侧作为假设，并证明其右侧即可。于是目标变为：

$$a\ b : \text{Prop},\ ha : a,\ hb : b \vdash a$$

习惯上会在假设名称前加前缀 h 。

最后，策略

```
apply ha
```

告诉 Lean 将名为 ha 的假设 a 与目标 $\vdash a$ 进行匹配。由于 a 在语法上等价于 a ，匹配成功。这就完成了证明。

非形式化地，我们可以用类似纸笔数学的风格，将证明写成如下形式：

令 a 和 b 为命题。

假设 $(ha)\ a$ 和 $(hb)\ b$ 为真。

为了证明 a ，我们需要使用假设 ha 。 □

(数学家可能会使用数字标签，如 (1) 和 (2)，而非具有描述性的假设名称。)

回到 Lean 证明，我们可以通过将变量和假设声明为定理的参数，来避免调用 `intro`，如下所示：

```
theorem fst_of_two_props_params (a b : Prop) (ha : a) (
  hb : b) :
  a :=
by apply ha
```

目标是：

$$a\ b : \text{Prop},\ ha : a,\ hb : b \vdash a$$

定理的所有参数在语境中立即生效。就像前面的例子一样，该目标通过 `apply ha` 即可证明。

下面是一个连续使用多个 `apply` 的例子：

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
  → c) :
  a → c :=
by
  intro ha
  apply hbc
  apply hab
  apply ha
```

站在数学家的角度，我们可以将最后的证明表述如下：

假设 $(ha)\ a$ 为真。

为了证明 c ，根据假设 hbc ，只需证明 b 即可。

为了证明 b ，根据假设 hab ，只需证明 a 即可。

为了证明 a ，我们使用假设 ha 。

□

3.2 基本策略

`intro` 和 `apply` 策略是策略证明的基础。其他基本策略包括 `exact`、`assumption`、`rfl` 和 `ac_rfl`。如果我们有足够的耐心使用这些策略进行推理，而不依赖更强大的 Proof Automation 的**证明自动化**，它们可以发挥很大作用。它们也可以用来解决各种逻辑谜题。

下面，大而细的方括号 `[]` 表示可选的语法。

`intro`

```
intro [名字1 ... 名字n]
```

`intro` 策略将最前面的 $\forall$ 量化变量或最前面的^{Assumption}前提 $a \rightarrow$ 从目标的^{Target}目的移动到局部语境中。该策略接受一个可选参数，用于指定语境中变量或前提的名称，从而覆盖默认名称。如果提供了多个名称（即 $n > 1$ ），则会移动多个变量或前提。

给定一个可证明的目标，`intro` 总是会产生一个可证明的目标。因此，它被称为是^{Safe}**安全的**。

apply

`apply` 定理或假设

`apply` 策略将目标的^{Target}目的与指定定理或假设的结论进行匹配，并将该定理或假设的前提作为新的目标加入。匹配是在^{Up to Computation}计算等价的意义下进行的。

我们必须谨慎调用 `apply`，因为它可能将一个可证明的目标转换为不可证明的子目标。例如，如果目标是 $\vdash \text{True}$ ，而我们 `apply` 了定理 $\text{False} \rightarrow \text{True}$ ，则结论 True 会与目标的目的 True 匹配，最后我们得到了不可证明的子目标 $\vdash \text{False}$ 。我们称 `apply` 是^{Unsafe}不安全的。

exact

`exact` 定理或假设

`exact` 策略将目标的目的与指定的定理或假设进行匹配，从而证明该目标。在这种情况下，我们通常也可以使用 `apply`，但 `exact` 能更好地传达我们的意图。

assumption

`assumption` 策略从局部语境中寻找一个与目标的目的匹配的假设，并将其应用以证明目标。

rfl

`rfl` 策略证明形如 $\vdash l = r$ 的目标，其中左右两边 l 和 r 在^{Up to Computation}计算等价的意义下在语法上是相等的。计算首先是指定义的^{Expansion}展开，但也包括将匿名函数应用于参数的^{Reduction}归约等等。这些^{Conversion}转换有传统的名称。下面列出了主要转换及其示例，其中全局语境包含 `def double (n : ℕ) : ℕ := n + n`：

α -转换 $(\text{fun } x \mapsto f\ x) = (\text{fun } y \mapsto f\ y)$
 β -转换 $(\text{fun } x \mapsto f\ x)\ a = f\ a$
 δ -转换 `double 5 = 5 + 5`
 ζ -转换 $(\text{let } n : \mathbb{N} := 2; n + n) = 4$
 η -转换 $(\text{fun } x \mapsto f\ x) = f$
 ι -转换 `Prod.fst (a, b) = a`

将这些转换作为从左向右的重写规则来重复应用的过程称为^{Reduction}归约；将转换反向应用一次称为^{Expansion}展开。

简而言之，为了证明 $\vdash l = r$ ，`rfl` 策略会展开 `l` 和 `r` 中的定义，并执行 β -归约及其他归约。如果在归约过程中，`l` 和 `r` 在某一点变得语法上完全相同，则证明成功。通常，`rfl` 在数学家会说「根据定义」的地方起作用。

ac_rfl

`ac_rfl` 与 `rfl` 类似，但它可以用于在^{Associativity}结合律（例如 $(a + b) + c = a + (b + c)$ ）和^{Commutativity}交换律（例如 $a + b = b + a$ ）的意义下进行推理。这适用于已注册为具有结合律和交换律的二元运算，例如算术类型上的 $+$ 和 $*$ ，以及集合上的 $\cup$ 和 $\cap$ 。我们将在第 3.6 节中看到一个例子。

sorry

我们在第 2 章中遇到的 `sorry` 证明命令可以作为策略在策略证明中的任何位置使用。它「证明」了当前目标，但实际上并未进行证明。请谨慎使用。

3.3 关于连结词和量词的推理

在学习对自然数、列表或其他数据类型进行推理之前，我们必须首先学习如何对 Lean 的逻辑连结词和量词进行推理。让我们从一个简单的例子开始：^{Conjunction}合取 ($\wedge$) 的^{Commutativity}交换律。

```
theorem And_swap (a b : Prop) :
  a ∧ b → b ∧ a :=
by
  intro hab
  apply And.intro
  apply And.right
  exact hab
  apply And.left
  exact hab
```

此时，我们建议您将光标置于 Visual Studio Code 中的示例之上，以查看证明状态的序列。通过将光标放在每个命令之上或紧随其后，您可以查看该行命令的效果。对于最后一行，Lean 只会报告「No goals」。

该证明是典型的 `intro-apply-exact` 组合。它使用了以下定理：

```
And.intro : ?a → ?b → ?a ∧ ?b
And.left  : ?a ∧ ?b → ?a
And.right : ?a ∧ ?b → ?b
```

其中问号 (?) 表示可以被**实例化**的变量 —— 例如, 通过将目标的**目的**与定理的**结论**进行匹配。这些变量被称为**元变量**。

上述三个定理是与**合取**相关的**引入规则**和两个**消去规则**。逻辑符号 (例如 $\wedge$) 的引入规则是结论以该符号作为最外层符号的定理。对偶地, 消去规则在其前提中包含该符号。对于每个逻辑符号, 引入规则告诉我们**如何证明**一个以该符号为最外层位置的命题。相比之下, 消去规则告诉我们该命题**必然是如何被证明的**。

在上述证明中, 我们应用 $\wedge$ 的引入规则来证明目标 $\vdash b \wedge a$, 并应用两个消去规则从假设 $a \wedge b$ 中提取 b 和 a 。我们在 $\vdash b \wedge a$ 中通过使用所谓的引入规则来「消去」 $\wedge$, 这听起来可能很奇怪。该术语之所以是「逆向」的, 是因为我们的证明是**逆向的**。

问号也可能出现在目标中。它们表示可以任意**实例化**的变量。在上面的证明过程中, 在 `apply And.right` 策略之后, 我们得到了目标:

`a b : Prop, hab : a  $\wedge$  b  $\vdash$  ?left.a  $\wedge$  b`

其中 `?left.a` 是一个**元变量**。策略 `exact hab` 将 (目的中的) `?left.a` 与 (hab 中的) `a` 进行匹配。这种通过实例化变量使两个项在语法上等价的过程称为**合一**。**匹配**是合一的一种特殊情况, 即其中一个项不包含变量, 如本例所示。

顺便提一下, 每当元变量出现在目标中时, 还会出现额外的子目标, 每个元变量的类型也会作为一个子目标 (例如 $\vdash \text{Prop}$)。这些令人困惑的子目标仅仅是一个提醒, 说明我们需要用正确类型的项来实例化这些元变量。我们通常可以忽略这些子目标。一旦元变量被实例化 (通常是在我们解决另一个子目标时), 其相关的子目标也会随之消失。

在 Lean 中, **合一**是在**计算等价**的意义下进行的。例如, 项 $(\text{fun } x \mapsto ?m) a$ 和 b 可以通过取 `?m := b` 来合一, 因为 $(\text{fun } x \mapsto b) a$ 和 b 在 β -转换的意义下是语法相等的。

下面是定理 `And_swap` 的另一种证明:

```
theorem And_swap_braces :
   $\forall a b : \text{Prop}, a \wedge b \rightarrow b \wedge a :=$ 
  by
    intro a b hab
    apply And.intro
    · exact And.right hab
    · exact And.left hab
```

该定理的陈述方式有所不同, 将 a 和 b 作为 $\forall$ 量化变量, 而不是定理的参数。从逻辑上讲, 这是等价的, 但在证明中我们必须在 `hab` 之外额外**引入** a 和 b 。

另一个区别是在子证明前面使用了项目符号 ($\cdot$)。当我们面临两个或多个目标需要证明时, 通常良好的风格是在每个子证明前面加上一个点。**策略组合子** $\cdot$

专注于第一个子目标；其后的策略必须证明该子目标。在我们的例子中，`apply And.intro` 策略创建了两个子目标： $\vdash b$ 和 $\vdash a$ 。

第三个区别是，我们现在通过 ^{Juxtaposition}并置，直接将 `And.right` 和 `And.left` 应用于 ^{Hypothesis}假设 $a \wedge b$ ，分别获得 b 和 a ，而不是等待定理的前作为新的子目标出现。这是在 ^{Backward Proof}逆向证明中迈出的微小 ^{Forward}正向步骤。同样的语法既可以用于 ^{Discharge}解除（即证明）一个假设，也可以用于 ^{Instantiate}实例化一个 $\forall$ 量词。这种方法的一个好处是，我们避免了可能令人困惑的 `?left.a` 元变量。

在下一个例子中，我们使用并置来实例化 $\forall$ 量词：

```
theorem f5_if (h :  $\forall n : \mathbb{N}, f\ n = n$ ) :  
  f 5 = 5 :=  
  by exact h 5
```

如果 h 是定理 $\forall n, f\ n = n$ ，那么 $h\ 5$ 就是定理 $f\ 5 = 5$ 。

^{Disjunction}析取 ($\vee$) 的引入规则和消去规则如下：

```
Or.inl :  $\forall b : \text{Prop}, ?a \rightarrow ?a \vee b$   
Or.inr :  $\forall b : \text{Prop}, ?a \rightarrow b \vee ?a$   
Or.elim :  $?a \vee ?b \rightarrow (?a \rightarrow ?c) \rightarrow (?b \rightarrow ?c) \rightarrow ?c$ 
```

^{Juxtaposition}并置，将定理名称应用于我们要实例化的值来直接实例化。因此，`Or.inl False` 对应于定理 $?a \rightarrow ?a \vee \text{False}$ 。这就是 ^{Forward}正向风格。

或者，我们可以在形如 $\dots \vdash c \vee d$ 的目标上调用 `apply Or.inl`。这会将定理中的 $?a$ 设为 c ， $?b$ 设为 d 。新的子目标是 $\dots \vdash c$ 。这就是 ^{Backward}逆向风格。

^{Unsafe}`Or.inl` 和 `Or.inr` 都是不安全的：如果您应用了两者中错误的一个，或者在证明中过早地应用了其中任何一个，您可能会得到一个不可证明的子目标。如果您考虑可证明的目标 $\vdash \text{True} \vee \text{False}$ ，就可以很容易地看到这一点：`apply Or.inr` 会产生不可证明的子目标 $\vdash \text{False}$ 。

`Or.elim` 规则乍一看可能违反直觉。本质上，它指出如果我们有 $a \vee b$ ，那么为了证明任意的 c ，只需证明在 a 成立时 c 成立，以及在 b 成立时 c 成立即可。您可以将 $(?a \rightarrow ?c) \rightarrow (?b \rightarrow ?c) \rightarrow ?c$ 看作是一个巧妙的技巧，仅使用蕴含来表达析取的含义。

^{Equivalence}等价 ($\leftrightarrow$) 的引入规则和消去规则如下：

```
Iff.intro :  $(?a \rightarrow ?b) \rightarrow (?b \rightarrow ?a) \rightarrow (?a \leftrightarrow ?b)$   
Iff.mp :  $(?a \leftrightarrow ?b) \rightarrow ?a \rightarrow ?b$   
Iff.mpr :  $(?a \leftrightarrow ?b) \rightarrow ?b \rightarrow ?a$ 
```

^{Existential Quantification}存在量化 ($\exists$) 的引入规则和消去规则如下：

存在量化 ($\exists$) 的引入规则和消去规则如下：

Exists.intro : $\forall w, (?P\ w \rightarrow (\exists x, ?P\ x))$
 Exists.elim : $(\exists x, ?P\ x) \rightarrow (\forall a, ?P\ a \rightarrow ?c) \rightarrow ?c$

$\exists$ 的引入规则可以用于为一个存在量词提供一个^{Witness}证据进行^{Instantiate}实例化——证据是使量词的主体为真的值。例如：

```
theorem Exists_double_iden :
   $\exists n : \mathbb{N}, \text{double } n = n :=$ 
  by
    apply Exists.intro 0
    rfl
```

同样地，我们以^{Forward}正向方式实例化一个 $\forall$ 量词：Exists.intro 0 是定理 $?P\ 0 \rightarrow (\exists x, ?P\ x)$ 。该规则是^{Unsafe}不安全的：为 x 选择错误的证体会导致不可证明的目标。例如，如果目标是 $\vdash \exists n, n > 5$ ，而我们选择 3 作为证体，最后会得到不可证明的子目标 $\vdash 3 > 5$ 。

$\exists$ 的消去规则让人联想到 $\vee$ 的消去规则。事实上，一种富有成效的思考量化 $\exists n, ?P\ n$ 的方式是，将其看作是一个可能^{Infinitary}无穷的析取 $?P\ 0 \vee ?P\ 1 \vee \dots$ 。类似地， $\forall n, ?P\ n$ 可以被看作是 $?P\ 0 \wedge ?P\ 1 \wedge \dots$ 。

对于真 (True)，只有一个引入规则：

True.intro : True

「真」不包含任何信息。如果它作为^{Hypothesis}假设出现，它是完全没用的，并且没有消去规则可以从中提取任何信息。下面第 3.8 节中描述的 clear 策略可以用来移除此类无用的假设。

对偶地，对于假 (False)，只有一个消去规则：

False.elim : False $\rightarrow$?a

无法证明「假」，但如果我们以某种方式从某处（例如从假设中）得到了它，那么我们可以推导出任何东西 (?a)。

事实上，^{Negation}否定 (Not) 是根据蕴含和「假」来定义的： $\neg a$ 是 $a \rightarrow \text{False}$ 的缩写。直观上，它们的含义相同。我们可以把「非 a 」看作是「 a 会导致荒谬的事情（因此非 a ）」。

Lean 的逻辑是^{Classical}经典的，支持 ^{Law of Excluded Middle}排中律 和 ^{Proof by Contradiction}反证法：

Classical.em : $\forall a : \text{Prop}, a \vee \neg a$
 Classical.byContradiction : $(\neg ?a \rightarrow \text{False}) \rightarrow ?a$

蕴含 ($\rightarrow$) 和 ^{Universal Quantification}全称量化 ($\forall$) 就像是谚语中那种「^{dogs that did not bark}不叫的狗」。它们没有引入规则或消去规则。相反，对于这两者，`intro` 策略就是引入原理，而 ^{Application}应用 (如 λ -演算中那样) 就是消去原理。例如，给定定理 $hab : a \rightarrow b$ 和 $ha : a$ ，应用 $hab\ ha$ 就是一个陈述 b 的定理。

为了证明涉及连结词和量词的逻辑谜题，我们提倡一种「盲目」的、「电子游戏」式的推理风格，它主要依赖于 `intro` 和 `apply` 等基本策略。以下是一些通常奏效的策略：

- 如果目标的目的是一个蕴含式 $P \rightarrow Q$ ，调用 `intro hP` 将 P 移动到你的假设中： $\dots, hP : P \vdash Q$ 。
- 如果目标的目的是一个全称量化 $\forall x : \sigma, Q$ ，调用 `intro x` 将 x 移动到局部语境中： $\dots, x : \sigma \vdash Q$ 。
- 寻找一个结论与目标的形状相同（可能包含可以匹配的变量）的定理或假设，并对其调用 `apply`。例如，如果目标的目的是 Q ，而你有一个形如 $hPQ : P \rightarrow Q$ 的定理或假设，请尝试 `apply hPQ`。
- 一个被否定的目标 $\vdash \neg P$ 在计算上等价于 $\vdash P \rightarrow \text{False}$ ，因此你可以调用 `intro hP` 来产生子目标 $hP : P \vdash \text{False}$ 。通过调用 `rw [Not]`（在第 3.5 节中描述）来展开否定的定义通常是一个好策略。
- 有时你可以通过输入 `apply False.elim` 将目标替换为 `False` 来取得进展。作为下一步，你通常会 `apply` 一个形如 $P \rightarrow \text{False}$ 或 $\neg P$ 的定理或假设。
- 当你面临多个选择时（例如在 `Or.inl` 和 `Or.inr` 之间选择），请记住你所做的选择，并在遇到 ^{Dead End}死胡同或感觉没有进展时进行 ^{Backtrack}回溯。
- 如果你怀疑自己可能遇到了死胡同，请检查在给定的前提下目标是否确实是可证明的。即使你最初陈述的是一个可证明的定理，当前的目标也可能是不可证明的（例如，如果你应用了不安全的规则）。

3.4 关于相等的推理

^{Equality}

相等 ($=$) 也是一个基本的逻辑常量。它的特征由以下引入规则和消去规则给出：

```
Eq.refl : ∀ a, a = a
Eq.symm : ?a = ?b → ?b = ?a
Eq.trans : ?a = ?b → ?b = ?c → ?a = ?c
Eq.subst : ?a = ?b → ?P ?a → ?P ?b
```

^{Equivalence Relation}

前三个定理是指定 $=$ 为 ^{Equivalence Relation}等价关系的引入规则。第四个定理是一个消去规则，它允许我们在任意语境（由元变量 $?P$ 表示）中进行等量替换。

下面的例子展示了其中一些规则的应用。我们应用 `Eq.trans` 和 `Eq.symm`，利用等式 $a = b$ 和 $c = b$ 来证明 $a = c$ ：

```
theorem Eq_trans_symm {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  apply Eq.trans
  • exact hab
  • apply Eq.symm
    exact hcb
```

由于这种重写操作非常常见，Lean 提供了一个 `rw` 策略来实现相同的结果。如果适用，该策略还会自动应用 `rfl`：

```
theorem Eq_trans_symm_rw {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  rw [hab]
  rw [hcb]
```

关于解析的说明：相等的结合优先级高于逻辑连结词。因此， $a = b \wedge c = d$ 被解读为 $(a = b) \wedge (c = d)$ 。

3.5 重写策略

重写策略 `rw` 及其相关的 `simp` 进行等量替换。它们将等式作为自左向右的重写规则，将出现的左侧项替换为右侧项。

默认情况下，它们在目标的目的上操作，但也可以通过 `at` 关键字来重写指定的假设：

```
at h1 ... hn  重写指定的假设
at *           重写所有假设和目标
```

`rw`

```
rw [定理或常量1, ..., 定理或常量n] [at 位置]
```

`rw` 策略使用一个或多个等式作为自左向右的重写规则来重写目标。它会搜索与第一个等式左侧匹配的的第一个子项；一旦找到，该子项的所有出现都会被替换为右侧项。如果等式包含变量，则会根据需要进行实例化。要反向使用定理（即作为自右

向左的重写规则)，请在定理名称前加上一个短左箭头 ($\leftarrow$)。如果指定了多个等式，它们将依次被应用。

因此，给定定理 $hg : \forall x, g\ x = f\ x$ 和目标 $\vdash h\ (f\ a)\ (g\ b)\ (g\ c)$ ，策略 $rw\ [hg]$ 会产生子目标 $\vdash h\ (f\ a)\ (f\ b)\ (g\ c)$ ，而 $rw\ [\leftarrow hg]$ 则产生子目标 $\vdash h\ (g\ a)\ (g\ b)\ (g\ c)$ 。

除了定理之外，我们还可以指定一个常数的名称。这将尝试使用该常数的定义等式之一作为重写规则。

simp

`simp [at 位置]`

`simp` 策略穷尽地使用一组标准的重写规则（称为 ^{Simp Set}**simp 集合**）来重写目标。`simp` 集合中的每个等式都用作自左向右的重写规则。`simp` 集合包含关于预定义符号（例如算术和列表运算符）的各种规则，并且可以通过在适当的定理上添加 `@[simp]` 属性来扩展。

`simp [定理或常量1, ..., 定理或常量n] [at 位置]`

对于上述 `simp` 变体，指定的定理会被临时添加到 `simp` 集合中。在定理列表中，星号 (*) 可以用来代表所有假设。定理名称前的减号 (-) 会临时将该定理从 `simp` 集合中移除。一个既能简化假设又能利用结果简化目标的目的的强大命令是 `simp [*] at *`。

给定定理 $hg : \forall x, g\ x = f\ x$ 和目标 $\vdash h\ (f\ a)\ (g\ b)\ (g\ c)$ ，策略 `simp [hg]` 产生子目标 $\vdash h\ (f\ a)\ (f\ b)\ (f\ c)$ ，其中 $g\ b$ 和 $g\ c$ 都已被重写。除了定理之外，我们还可以指定常数的名称。这会临时将常数的定义等式添加到 `simp` 集合中。

说到这里，你可能会想，「那么 `simp` 到底做了什么呢？」当然，你可以去研究源代码或查阅科学文献。但这可能不是对你时间最有效的利用。事实上，即使是证明助手的专家用户也并不完全理解他们每天使用的策略的行为。最成功的用户会采取一种轻松、随性的态度，依次尝试各种策略，并研究出现的子目标（如果有的话），看看自己是否走在正确的道路上。

随着你不断地使用 `simp` 和其他策略，你会对它们擅长处理什么样的目标产生一些直觉。这是交互式定理证明只能通过实践来学习的众多原因之一。通常，你不会确切地理解 Lean 做了什么——为什么一个策略成功了，或者失败了。定理证明有时会非常令人沮丧。^{The Hitchhiker's Guide to the Galaxy}《银河系漫游指南》封面上的那些醒目而友好的大字所给出的建议在这里同样适用：**不要恐慌**。

3.6 数学归纳法证明

^{Structural induction}
induction 归纳策略对一个归纳类型的值执行结构归纳。**结构归纳**意味着归纳遵循归纳类型的结构。对于由 `Nat.zero` 和 `Nat.succ` 构造的自然数，结构归纳对应于标准的数学归纳法：要证明 $p\ n$ ，只需证明 $p\ 0$ 和 $\forall k, p\ k \rightarrow p\ (k + 1)$ 。借助 `induction`，我们可以推理 2.2 一节中通过递归定义的加法和乘法运算。

加法是通过对其第二个参数进行递归定义的。我们将证明两个定理，`add_zero` 和 `add_succ`，它们为我们提供了在**第一个**参数上递归的替代等式。我们从 `add_zero` 开始：

```
theorem add_zero (n : ℕ) :
  add 0 n = n :=
by
  induction n with
  | zero      => rfl
  | succ n' ih => simp [add, ih]
```

^{Induction}
归纳策略后面跟着两个由它们对应的构造子标识的子证明。此外，任何特定于子证明的变量或假设都可以明确地在构造子名称之后命名。

第一个情况，标记为 `zero`，对应于基本情况 $\vdash \text{add } 0\ 0 = 0$ 。第二个情况，标记为 `succ`，对应于归纳步骤

```
n' : ℕ, ih : add 0 n' = n' ⊢ add 0 (Nat.succ n') = Nat.succ n'
```

`succ` 后面指定的名称 `n'` 和 `ih` 分别是我们给 `Nat.succ` 的参数和归纳假设所起的名称。用其他名称也可以，但通常将归纳假设称为 `ih`。

我们可以继续通过结构归纳证明定理：

```
theorem add_succ (m n : ℕ) :
  add (Nat.succ m) n = Nat.succ (add m n) :=
by
  induction n with
  | zero      => rfl
  | succ n' ih => simp [add, ih]
```

```
theorem add_comm (m n : ℕ) :
  add m n = add n m :=
by
  induction n with
  | zero      => simp [add, add_zero]
```

```

| succ n' ih => simp [add, add_succ, ih]

theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
  by
    induction n with
    | zero      => rfl
    | succ n' ih => simp [add, ih]

```

一旦我们证明了一个二元运算满足交换律和结合律，就应该让 Lean 的自动化机制，尤其是 `ac_rfl` 知道这一点。下面的命令即为 `add` 实现了这一点：

```

instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }

```

(instance 机制将在第 5 章讲解。) 下面的例子使用 `ac_rfl` 策略，在 `add` 的结合律与交换律意义下进行推理：

```

theorem mul_add (l m n : ℕ) :
  mul l (add m n) = add (mul l m) (mul l n) :=
  by
    induction n with
    | zero      => rfl
    | succ n' ih =>
      simp [add, mul, ih]
      ac_rfl

```

下面给出一些关于如何使用数学归纳法进行证明的提示：

- 按照目标中出现的某个函数的定义结构，来进行归纳证明，通常是有益的。特别地，如果一个函数是通过对其第 n 个参数进行递归来定义的，那么对该参数进行归纳通常是合理的。
- 如果归纳的基本情况难以证明，这通常表明选错了变量，或者需要先证明一些辅助引理。

3.7 归纳策略

我们已经在上面看到了 `induction` 策略的实际用法。为方便查阅，下面给出它的完整语法。一般而言，本指南正文中介绍的每一种策略，都会在名为「X 策略」的小节中附带此类语法描述。

induction

```
induction 项 [with
| 构造子1 名字1 => 策略1
  :
| 构造子n 名字n => 策略n]
```

`induction` 策略对指定项执行结构归纳。这会生成与该项类型定义中构造子数量相同的子目标。在对应递归构造子（如 `Nat.succ` 或 `List.cons`）的子目标中，归纳假设将作为可用假设出现。可选的名字 `名字1`、...和 `名字n` 用于命名生成的变量或假设。

3.8 清理策略

下面这些策略有助于简化证明目标。目前我们尚未用到它们，但在探索证明思路的过程中，它们会很有帮助。

clear

```
clear 变量或假设1 ... 变量或假设n
```

只要指定的变量和假设没有在目标的其他任何地方使用，`clear` 就会移除它们。

rename

```
rename 变量的类型或假设的命题 => 新名字
```

`rename` 策略会修改变量或假设的名字。

3.9 新引入的 Lean 结构小结**属性**

`@[simp]` 将定理添加到 `simp` 集

证明命令

`by` 开始一个策略式的证明

策略

<code>ac_refl</code>	证明 $l = r$ 满足交换律和结合律
<code>apply</code>	将目标的目的与定理的结论进行匹配
<code>assumption</code>	使用假设证明目标
<code>clear</code>	从目标中移除变量或假设
<code>exact</code>	使用指定的定理证明目标
<code>induction</code>	对归纳类型的变量进行结构归纳
<code>intro</code>	将 $\forall$ -量化的变量移入到目标的假设中
<code>rename</code>	重命名变量或假设
<code>rfl</code>	证明 $l = r$ 计算等价
<code>rw</code>	使用给定的定理作为重写规则重写一次
<code>simp</code>	使用一组预先注册的重写规则穷举重写
<code>sorry</code>	表示缺少证明

策略组合子

- 专注于第一个子目标；需要证明该子目标。

正向证明

策略证明是逆向进行的。它们从目标开始，将其归约到已有的定理和假设。通常，以正向方式进行证明也是合理的：从已有的定理和假设出发，逐步向我们的目标推进。

Structured Proofs
结构化证明就是一种支持这种推理方式的风格。策略证明往往易于编写但难于阅读。大多数用户会结合这两种风格，根据具体情况使用最合适的一种。结构化证明较高的可读性使其深受某些用户的喜爱。

Proof Terms
结构化证明是撒在 Lean **证明项**之上的一层语法糖。它们使用 `assume`、`have`、`let` 和 `show` 等关键字来构建，模仿了纸笔形式的证明。所有 Lean 的证明，无论是策略证明还是结构化证明，在系统内部都会被归约成证明项。在第 3 章中，我们已经看到了一些示例：给定假设 $ha : a$ 和 $hab : a \rightarrow b$ ，项 $hab\ ha$ 即是命题 b 的证明项，因此我们可以写成 $hab\ ha : b$ 。定理和假设的名称（例如 ha 和 hab ）也是证明项。沿着这一思路继续，给定 $hbc : b \rightarrow c$ ，我们可以为命题 c 构造证明项 $hbc\ (hab\ ha)$ 。我们可以将 hab 视为一个函数，它可将 a 的证明转换为 b 的证明，对于 hbc 也同理。

Tactic Mode
结构化证明是 Lean 中的默认方式。它们可以在**策略模式**之外使用。要进入策略模式，我们必须使用 `by` 命令。

Theorem Proving in Lean 4
此处涵盖的概念在 **Lean 4 定理证明**[12] 的第 2 至 4 章中有更详细的描述。Nederpelt 和 Geuvers 的教科书 [20] 是另一个有用的参考资料。

4.1 结构化证明

作为第一个例子，考虑以下结构化证明：

```
theorem fst_of_two_props :  
  ∀ a b : Prop, a → b → a :=
```

```
fix a b : Prop
assume ha : a
assume hb : b
show a from
  ha
```

每个由 $\forall$ ^{Quantifier} 量词绑定的变量以及 ^{Implication} 蕴涵的每个假设, 都在证明中通过使用 `fix` 和 `assume` 命令显式地引入。可以同时引入多个变量。我们经常会省略变量的类型, 特别是当它们可以从名称中猜出时; 然而, 我们总是会写出完整的 ^{Proposition} 命题, 并且每行放置一个, 以提高可读性 —— 毕竟, 这是 ^{Structured Style} 结构化风格的主要潜在优势之一。结尾处的 `show ... from` 命令为了可读性重复了要证明的命题, 并在关键字 `from` 之后给出证明。此时的 ^{Goal} 目标是 $a b : \text{Prop}, ha : a, hb : b \vdash a$ 。

非正式地, 我们可以将证明写为如下形式:

固定一些命题 a 和 b 。

假设 $(ha) a$ 且 $(hb) b$ 为真。

我们必须证明 a 。这根据 ha 平凡地得出。

□

一些作者会在「命题」之前插入诸如「^{Arbitrary but Fixed}任意但固定」之类的限定词, 或者他们会写「令 a 和 b 是某些命题」。所有这些变体都是等价的。此外, 我们也可以用「QED」来代替「□」以结束证明。

上述 Lean 证明是非典型的, 因为目标的目的出现在 ^{Hypothesis} 假设之中。通常, 我们必须执行一些中间推理步骤, 其形式基本上是「由某某得到某某」。在 Lean 中, 每个中间步骤都采用 `have` 命令的形式, 如下例所示:

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
  → c) :
  a → c :=
  assume ha : a
  have hb : b :=
    hab ha
  have hc : c :=
    hbc hb
  show c from
    hc
```

非正式地:

假设 $(ha) a$ 为真。

由 ha 和 hab , 我们得到 $(hb) b$ 。

由 hb 和 hbc , 我们得到 $(hc) c$ 。

我们必须证明 c 。这根据 hc 平凡地得出。

□

注意这是一个^{Forward Proof}**正向证明**：它从假设 a 开始，逐个定理地向目标定理 c 推进。
^{Modus Ponens}**肯定前件**($\lceil$ 从 a 和 $a \rightarrow b$ 推导出 b $\rceil$) 通过简单的并置来表达 (例如 $hab\ ha$)。

一般而言, `fix-assume-show` 骨架重复了定理的陈述。我们经常会根据目标中的绑定变量来命名固定变量, 正如我们在这里所做的那样, 但这并不是强制性的。
^{Shadowing}此外, 避免遮蔽已有变量是一个良好的实践。在最后一个 `fix` 或 `assume` 与 `show` 之间, 我们可以根据需要的论证详细程度, 放入任意数量的 `have` 命令。细节可以增加可读性, 但提供过多细节可能会令读者难以忍受。

`have` 命令的语法与 `theorem` 相似, 但它出现在结构化证明之中。我们也可以将 `have` 视为一个定义。在 `have hb : b := hab ha` 中, 右侧的 $hab\ ha$ 是 b 的证明项, 而左侧的 hb 被定义为该证明项的同义词。从此以后, hb 和 $hab\ ha$ 即可互换使用。由于 hb 和 hc 只被使用了一次, 且它们的证明非常简短, 专家们往往倾向于将它们^{Inline}**内联**, 把 hc 替换为 $hbc\ hb$, 然后把 hb 替换为 $hab\ ha$, 从而得到:

```
theorem prop_comp_inline (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
  assume ha : a
  show c from
    hbc (hab ha)
```

一个典型的结构化证明具有如下的 `fix-assume-have-show` 格式:

```
theorem hr :
  ∀(c₁ : σ₁) ... (c₁ : σ₁), P₁ → ... → Pₘ → R :=
  fix (c₁ : σ₁) ... (c₁ : σ₁)
  assume h₁ : P₁
  ⋮
  assume hₘ : Pₘ
  have k₁ : Q₁ := ...
  ⋮
  have kₙ : Qₙ := ...
  show R from ...
```

4.2 结构化构造

上一节介绍了用于编写结构化证明的主要命令：fix、assume、have 和 show。现在我们更系统地回顾结构化证明的各组成部分。

定理或假设

除了 sorry 之外，最简单的结构化证明是定理或假设的名称。如果我们有：

```
theorem two_add_two_Eq_four :
  2 + 2 = 4 :=
  by ...
```

那么在此之后，定理名称 two_add_two_Eq_four 即可被用作 $2 + 2 = 4$ 的证明。例如：

```
theorem this_time_with_feeling :
  2 + 2 = 4 :=
  two_add_two_Eq_four
```

我们可以将参数传递给定理，以^{Instantiate}实例化 $\forall$ 量词并^{Discharge}解除假设。假设定理 add_comm ^{Instance}($m : \mathbb{N}$) : $\text{add } m \ n = \text{add } n \ m$ 可用，并且我们想要证明它的^{Instance}实例 $\text{add } 0 \ n = \text{add } n \ 0$ 。这可以通过使用定理名称和两个参数来巧妙地完成：

```
theorem add_comm_zero_left (n : ℕ) :
  add 0 n = add n 0 :=
  add_comm 0 n
```

这等价于策略证明 by exact add_comm 0 n，但更为简洁。exact 策略可以被看作是 by 的逆操作。为何要进入策略模式又立刻离开它呢？

与 exact 和 apply 一样，定理或假设的陈述会在计算等价的意义上与当前目标进行匹配。这提供了一些灵活性。

fix

```
fix names : type
```

fix 命令将由 $\forall$ 绑定的量化变量从目标的目的移动到^{Local Context}局部语境。它是 intro 策略的结构化版本。

请注意，Lean 中用于固定变量的标准策略是 fun。我们倾向于使用由 LoVelib 提供的 fix，作为一个更具可读性的替代方案。我们将把 fun 保留用于指定匿名函数。

assume

```
assume name : proposition
```

assume 命令将最前面的假设从目标的目的移动到局部语境中。它可以被看作是 intro 策略的结构化版本。

请注意，Lean 中用于陈述假设的标准策略是 fun。我们倾向于使用由 LoVelib 提供的 assume，作为一个更具可读性的替代方案。我们将把 fun 保留用于指定匿名函数。

have

```
have 名称 : 命题 :=
  证明
```

have 命令允许我们陈述并证明一个中间定理，该定理可以引用之前由 fix、assume 和 have 引入的名称。证明过程可以是策略式的^{Tactical}，也可以是结构化的。通常，我们倾向于使用结构化证明来勾勒主要论证过程，而在证明子目标或无关紧要的中间步骤时则求助于策略式证明。

当我们将参数传递给定理名称时，会出现另一种混合情况。例如，给定 $hab : a \rightarrow b$ 和 $ha : a$ ，策略 `exact hab ha` 将证明目标 $\vdash b$ 。在这里，`hab ha` 是一个嵌套在策略内部的证明项。

let

```
let 名称 [ : 类型 ] := 项
```

let 命令引入一个新的局部定义^{Local Definition}。它可以用于为证明后续部分多次出现的复杂对象命名。它与 have 类似，但它是为可计算数据^{Computable Data}而不是证明设计的。展开或引入一个 let 对应于 λ -变换 ^{λ -conversion} (第 3.2 节)。

构造「`let 名称 [ : 类型 ] := 项`」之后必须跟随换行符或分号 (;)。

show

```
show 命题 from
  证明
```

show 命令允许我们重复要证明的目标，这可以起到文档的作用。它还允许我们在计算等价的意义上改写目标。如果我们不想重复目标且不需要改写它，可以直接写 证明，而不必使用「`show 命题 from 证明`」的语法。证明可以是策略式的，也可以是结构化的。

4.3 关于联结词和量词的正向推理

以正向方式对^{Logical Connectives Quantifier}逻辑联结词和量词进行推理，使用的是与策略模式（第 3.3 节）相同的^{Introduction and Elimination Rules}引入和消去规则。通过几个例子可以初步体会其风格。让我们从^{Conjunction}合取开始：

```
theorem And_swap (a b : Prop) :
  a ∧ b → b ∧ a :=
  assume hab : a ∧ b
  have ha : a :=
    And.left hab
  have hb : b :=
    And.right hab
  show b ∧ a from
    And.intro hb ha
```

即便是不了解 And.left 等含义的读者，也能理解我们是从 $a \wedge b$ 中提取出 a 和 b ，然后将它们重新组合为 $b \wedge a$ 。数学家们理解这一证明可能比理解其对应的策略证明要容易得多：

```
theorem And_swap_tactical (a b : Prop) :
  a ∧ b → b ∧ a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

一般来说，逆向证明更容易「不假思索地」推导出来，而且证明助手提供的大多数自动化功能也是逆向工作的。这很有道理：假设你是在调查谋杀案的马普尔小姐或赫尔克里·波洛。逆向调查会从犯罪现场开始，尝试提取线索，将少数嫌疑人与犯罪联系起来。相比之下，正向调查可能会从询问多达 80 亿人开始，以确定他们是否有不在场证明。哪种方法更有可能成功？

接下来的例子涉及到^{One-Point Rules}单点规则。在绑定变量实际上只能取一个值时，这些定理可以用来消去量词。例如，命题 $\forall n, n = 666 \rightarrow \text{beast} \geq n$ 可以简化为 $\text{beast} \geq 666$ 。下面的定理证明了这种简化的合理性：

```
theorem Forall.one_point {α : Type} (t : α) (P : α →
  Prop) :
  (∀ x, x = t → P x) ↔ P t :=
```

```

Iff.intro
  (assume hall :  $\forall x, x = t \rightarrow P\ x$ 
    show  $P\ t$  from
      by
        apply hall t
        rfl)
  (assume hp :  $P\ t$ 
    fix x :  $\alpha$ 
    assume heq :  $x = t$ 
    show  $P\ x$  from
      by
        rw [heq]
        exact hp)

```

证明过程看起来可能有些令人生畏，但其实并不难。关键在于一步步来。起初，我们观察到目标是一个^{Equivalence}等价式，于是我们写下：

```
Iff.intro ( _ ) ( _ )
```

其中 `Iff.intro` 是 $\leftrightarrow$ 的引入规则（第 3.3 节）。

两个^{Placeholders}占位符对于使证明格式良好非常重要。我们使用圆括号是因为我们强烈怀疑子证明是非平凡的。由于证明基本上就是项，也就是程序，我们在第 1.4 节给出的建议在此同样适用，只需作相应调整（*mutatis mutandis*）：

关键在于，proof 应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发 proof 的一个好的原则是，从可以编译的 proof 开始，进行尽可能小的更改，以获得新的可以编译的 proof，并重复此过程，直到 proof 完成。

将鼠标悬停在第一个占位符上，会显示对应的^{Subgoal}子目标。我们可以看到 Lean 期望得到 $\vdash (\forall x, x = t \rightarrow P\ x) \rightarrow P\ t$ 的证明，于是我们提供了一个合适的^{Skeleton}骨架。蕴涵的结构化证明由 `assume` 后跟 `show` 组成：

```

Iff.intro
  (assume hall :  $\forall x, x = t \rightarrow P\ x$ 
    show  $P\ t$  from
      _ )
  ( _ )

```

剩余的每一个占位符都可以用结构化证明或策略证明来替代。为了消除第一个占位符，我们进入策略模式，并将假设 $\forall x, x = t \rightarrow P\ x$ 的^{Instance}实例 $t = t \rightarrow P\ t$ 应用于目标 $\vdash P\ t$ 。这留下了^{Proof Obligation}证明义务 $t = t$ ，我们可以使用 `rfl` 策略来完成它：

```
Iff.intro
  (assume hall : ∀x, x = t → P x
   show P t from
     by
       apply hall t
       rfl))
  (_)
```

第二个占位符代表 $\vdash P t \rightarrow \forall x, x = t \rightarrow P x$ 。为了消除它，我们根据第一个蕴涵的假设编写 `assume`，为 $\forall$ 编写 `fix`，为第二个蕴涵的假设编写 `assume`，并为结论编写 `show`：

```
(assume hp : P t
 fix x : α
 assume heq : x = t
 show P x from
  _)
```

`show` 的证明是策略式的：我们将 $\vdash P x$ 中的 x 重写为 t ，得到 $\vdash P t$ ，这与假设 `hp` 相符：

```
(assume hp : P t
 fix x : α
 assume heq : x = t
 show P x from
  by
    rw [heq]
    exact hp)
```

让我们检查一下单点规则在我们的引导示例上是否真的有效：

```
theorem beast_666 (beast : ℕ) :
  (∀n, n = 666 → beast ≥ n) ↔ beast ≥ 666 :=
  Forall.one_point _ _
```

它确实有效。除了 `Forall.one_point _ _`，我们也可以写成 `Forall.one_point 666 (fun m ↦ beast ≥ m)`。

最后， $\exists$ 的单点规则演示了如何在结构化证明中使用 $\exists$ 的引入和消去规则：

```
theorem Exists.one_point {α : Type} (t : α) (P : α →
  Prop) :
  (∃x : α, x = t ∧ P x) ↔ P t :=
  Iff.intro
```

```

(assume hex :  $\exists x, x = t \wedge P x$ 
 show P t from
   Exists.elim hex
   (fix x :  $\alpha$ 
    assume hand :  $x = t \wedge P x$ 
    have hxt :  $x = t$  :=
      And.left hand
    have hpx :  $P x$  :=
      And.right hand
    show P t from
      by
        rw [←hxt]
        exact hpx))
(assume hp : P t
 show  $\exists x : \alpha, x = t \wedge P x$  from
   Exists.intro t
   (have tt :  $t = t$  :=
    by rfl
    show  $t = t \wedge P t$  from
      And.intro tt hp))

```

虽然证明看起来可能有些复杂，但它的开发过程基本上是不假思索的，使用了与 $\forall$ 单点规则证明相同的步骤。

首先，我们使用 $\leftrightarrow$ 的引入规则：

```
Iff.intro (_) ( _ )
```

对于第一个占位符，我们需要在假设 $\exists x : \alpha, x = t \wedge P x$ 下提供 $P t$ 的证明。因此，我们将证明骨架展开为：

```

Iff.intro
  (assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
   show P t from
     _ )
  ( _ )

```

此时，我们想要从假设 hex 中提取 x 的^{Witness}见证。为了达到这个目的，我们使用 $\exists$ 的消去规则：

```

Iff.intro
  (assume hex :  $\exists x : \alpha, x = t \wedge P x$ 

```

```

show P t from
  Exists.elim hex ( )
( )

```

第一个占位符需要子目标 $\vdash \forall x, x = t \wedge P x \rightarrow P t$ 的证明。我们先把它勾勒出来：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
   Exists.elim hex
    (fix x :  $\alpha$ 
     assume hand :  $x = t \wedge P x$ 
     show P t from
       _))
( )

```

请注意子证明是如何镜像子目标的：fix 对应 $\forall$ ，assume 对应 $\rightarrow$ 的假设，而 show 对应 $\rightarrow$ 的结论。现在我们可以访问见证 x 及其特征属性 $x = t \wedge P x$ 。

接下来，我们使用 have 命令和 $\wedge$ 的消去规则 (And.left 和 And.right) 提取 $x = t \wedge P x$ 的两个 ^{Conjuncts}合取项：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
   Exists.elim hex
    (fix x :  $\alpha$ 
     assume hand :  $x = t \wedge P x$ 
     have hxt :  $x = t$  :=
       And.left hand
     have hpx :  $P x$  :=
       And.right hand
     show P t from
       _))
( )

```

现在我们可以尝试证明 $P t$ 。我们将 t 重写为 x ，并应用假设 $P x$ ：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
   Exists.elim hex

```

```

(fix x :  $\alpha$ 
  assume hand :  $x = t \wedge P\ x$ 
  have hxt :  $x = t$  :=
    And.left hand
  have hpx :  $P\ x$  :=
    And.right hand
  show  $P\ t$  from
    by
      rw [←hxt]
      exact hpx))
( )

```

对于剩余的占位符，我们首先根据子目标 $\vdash P\ t \rightarrow \exists x : \alpha, x = t \wedge P\ x$ 提供一个证明骨架：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    _ )

```

然后我们使用 $\exists$ 的引入规则提供见证 t ：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t ( ))

```

接下来是更多的样板代码：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t
      (show  $t = t \wedge P\ t$  from
        _))

```

最后的合取项 $t = t \wedge P\ t$ 很容易证明：其左侧通过^{Reflexivity}自反性得出，而右侧仅仅是假设 hp 。最后，我们得到：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t
      (have tt :  $t = t$  :=
        by rfl
      show  $t = t \wedge P\ t$  from
        And.intro tt hp))

```

为了开发上述结构化证明，我们使用了与展示类型的^{Inhabitant}居留元（第 1.4 节）时大致相同的步骤，将函数箭头理解为蕴涵。这两个步骤之间的相似性并非巧合，正如我们将在第 4.7 节中看到的那样。

4.4 计算式证明

在非正式数学中，我们经常将证明表示为等式、不等式或等价式的^{Transitive Chain}传递链（例如 $a = b = c$ 、 $a \geq b \geq c$ 或 $a \leftrightarrow b \leftrightarrow c$ ）。在 Lean 中，此类^{Calculational Proofs}计算式证明由 `calc` 命令提供支持，该命令提供了一种轻量级语法，并负责为等式及算术比较运算符等^{Preregistered Relations}预注册关系应用^{Transitivity Theorems}传递性定理。

其通用语法如下：

```
calc
  项0 运算符1 项1 :=
    证明1
  _ 运算符2 项2 :=
    证明2
    ⋮
  _ 运算符n 项n :=
    证明n
```

下划线（`_`）是语法的一部分。每个^{Compatible}证明_{*i*} 证明了陈述 项_{*i-1*} 运算符_{*i*} 项_{*i*}。运算符_{*i*} 不必完全相同，但它们必须彼此兼容。例如，`=`、`<` 和 `≤` 是兼容的，而 `>` 和 `<` 则不兼容。

下面是一个简单的例子：

```
theorem two_mul_example (m n : ℕ) :
  2 * m + n = m + n + m :=
calc
  2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  _ = m + n + m :=
    by ac_rfl
```

数学家们（假设他们愿意为这样一个平凡的结论提供证明）可能会将上述证明大致写成如下形式：

$$\begin{aligned}
 2 * m + n &= (m + m) + n && \text{(因为 } 2 * m = m + m \text{)} \\
 &= m + n + m && \text{(通过 } + \text{ 的} \text{ } \overset{\text{Associativity Commutativity}}{\text{结合律和交换律}} \text{)}
 \end{aligned}$$

□

在 Lean 证明中，下划线代表项 $(m + m) + n$ ，如果我们不使用 `calc` 来编写证明，我们就不得不重复写出该项：

```
theorem two_mul_example_have (m n : ℕ) :
  2 * m + n = m + n + m :=
  have hmul : 2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  have hcomm : m + m + n = m + n + m :=
    by ac_rfl
  show _ from
    Eq.trans hmul hcomm
```

请注意，使用 `have` 还需要我们显式地调用 `Eq.trans`，并为这两个中间步骤命名。

4.5 使用策略进行正向推理

许多用户更喜欢^{Tactic Mode}**策略模式**而非结构化证明。但即使在策略模式下，以正向方式进行推理、混合正向和逆向推理步骤也是非常有用的。结构化证明命令 `have`、`let` 和 `calc` 也可以作为策略使用，这使得上述操作成为可能。

下面的例子在一个我们已经见过好多次的定理上演示了 `have` 和 `let` 策略：

```
theorem prop_comp_tactical (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
  by
    intro ha
    have hb : b :=
      hab ha
    let c' := c
    have hc : c' :=
      hbc hb
    exact hc
```

have

```
have 名称 : 命题 :=
  证明
```

`have` 策略允许我们在策略模式下陈述并证明一个中间定理。之后，该定理在^{Goal State}**目标状态**中可用作一个假设。

let

`let 名称 [ : 类型 ] := 项`

`let` 策略允许我们在策略模式下引入一个局部定义。之后，所定义的符号及其定义在目标状态中可用。

calc

`calc`

```
项0 运算符1 项1 :=
  证明1
_ 运算符2 项2 :=
  证明2
  ⋮
_ 运算符n 项n :=
  证明n
```

`calc` 策略允许我们在策略模式下进入计算式证明（第 4.4 节）。该策略与同名的结构化证明命令具有相同的语法。我们可以将作为策略的 `calc ...` 看作是作为上述结构化证明命令的 `apply (exact calc ...)` 的别名。

4.6 依值类型

^{Dependent Type} **依值类型** 是 ^{Dependent Type Theory} **依值类型论** 这一系列逻辑系统的定义性特征。虽然你可能不熟悉这个术语，但你很可能以某种形式熟悉这个概念。

考虑一个函数 `pick`，它接受一个 ^{Natural Number} **自然数** n （即来自 $\mathbb{N} = \{0, 1, 2, \dots\}$ 的一个值），并返回一个介于 0 和 n 之间的自然数。直观上，`pick n` 的类型应该是 $\{0, 1, \dots, n\}$ （即由所有满足 $i \leq n$ 的自然数组成的类型）。在 Lean 中，这个类型写为 $\{i : \mathbb{N} // i \leq n\}$ 。这就是 `pick n` 的类型。具有数学背景的读者可能会倾向于将 `pick` 看作一个由 $\mathbb{N}$ ^{Index} **索引** 的 ^{Indexed Family of Terms} **索引项族**：

$$(\text{pick } n : \{i : \mathbb{N} // i \leq n\})_{n:\mathbb{N}}$$

其中每个项的类型都依赖于索引——例如，`pick 5 : {i :  $\mathbb{N}$  //  $i \leq 5$ }`。但是 `pick` 本身的类型是什么呢？我们希望表达 `pick` 是一个函数，它接受一个参数 $n : \mathbb{N}$ ，并返回一个类型为 $\{i : \mathbb{N} // i \leq n\}$ 的值。为了体现这一点，我们可以写作：

$$\text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} // i \leq n\}$$

这是一个依值类型：结果的类型依赖于参数 n 的值。（变量名 n 本身是无关紧要的；我们也可以写成 m 或 x_0 。）

除非另有说明，否则「依值类型」指的是依赖于一个（非类型）项的类型，如上所示，其中 $n : \mathbb{N}$ 是项，而 $\{i : \mathbb{N} // i \leq n\}$ 是依赖于它的类型。但是：

- 类型也可以依赖于另一个类型——例如，类型构造子 `List`，其 ^{η -expanded} **η -展开**的变体 ^{Domain and Codomain} `fun  $\alpha : \text{Type} \mapsto \text{List } \alpha$` ，或者具有相同**定义域和陪域**的函数的**多态类型** ^{Polymorphic Type} `fun  $\alpha : \text{Type} \mapsto \alpha \rightarrow \alpha$` 。
- 项可以依赖于类型——例如，^{Polymorphic Identity Function} **多态恒等函数** `fun  $\alpha : \text{Type} \mapsto \text{fun } x : \alpha \mapsto x$` 。
- 当然，项也可以依赖于项——例如，`fun  $n : \mathbb{N} \mapsto n + 2$` 。

综上所述， $\text{fun } x \mapsto t$ 共有四种情况：

函数体 (t)	参数 (x)	描述
项	依赖于 项	^{Simply Typed λ-expression} 简单类型 λ-表达式
类型	依赖于 项	狭义的依值类型
项	依赖于 类型	^{Polymorphic Term} 多态项
类型	依赖于 类型	^{Polymorphic Type Constructor} 多态类型构造子

最后三行对应于 Henk Barendregt 的 ^{λ -cube} **λ -立方**的三条**轴**。¹

第 1.3 节中介绍的 APP 和 FUN 规则必须经过 ^{Generalized} **泛化**，才能适用于依值类型：

$$\begin{array}{c}
 \frac{C \vdash t : (x : \sigma) \rightarrow \tau[x] \quad C \vdash u : \sigma}{C \vdash t u : \tau[u]} \text{APP}' \\
 \\
 \frac{C, x : \sigma \vdash t : \tau[x]}{C \vdash (\text{fun } x : \sigma \mapsto t) : (x : \sigma) \rightarrow \tau[x]} \text{FUN}'
 \end{array}$$

记号 $\tau[x]$ 代表一个可能包含 x 的类型，而 $\tau[u]$ 代表将其中所有出现的 x 替换为 u 后得到的相同类型。

当 x 不出现在 $\tau[x]$ 中时，就出现了简单类型的情况。此时，我们可以简单地将 $(x : \sigma) \rightarrow$ 写为 $\sigma \rightarrow$ 。我们所熟悉的记号 $\sigma \rightarrow \tau$ 等价于 $(_ : \sigma) \rightarrow \tau$ 。很容易验证，当 x 不出现在 $\tau[x]$ 中时，APP' 和 FUN' 与 APP 和 FUN 是重合的。

下面的例子演示了 APP'：

$$\frac{\vdash \text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} // i \leq n\} \quad \vdash 5 : \mathbb{N}}{\vdash \text{pick } 5 : \{i : \mathbb{N} // i \leq 5\}} \text{APP}'$$

¹https://en.wikipedia.org/wiki/Lambda_cube

下一个例子演示了 FUN'：

$$\frac{\frac{\alpha : \text{Type}, x : \alpha \vdash x : \alpha}{\alpha : \text{Type} \vdash (\text{fun } x : \alpha \mapsto x) : \alpha \rightarrow \alpha} \text{FUN or FUN'}}{\vdash (\text{fun } \alpha : \text{Type} \mapsto \text{fun } x : \alpha \mapsto x) : (\alpha : \text{Type}) \rightarrow \alpha \rightarrow \alpha} \text{FUN'}$$

这一图景并不完整，因为我们只检查了项——即冒号 (:) 左侧的实体——是否类型正确。冒号右侧的实体——即类型——也应该使用相同的类型系统进行检查。例如，`Nat.succ` 的类型是 $\mathbb{N} \rightarrow \mathbb{N}$ ，其类型是 `Type`。类型的类型（例如 `Type` 和 `Prop`）被称为^{Universe}**宇宙**。我们将在第 ?? 章中更深入地研究它们。

值得注意的是，^{Universal Quantification}**全称量化**仅仅是^{Dependent Type}**依值类型**的一个别名： $\forall x : \sigma, \tau$ 是 $(x : \sigma) \rightarrow \tau$ 的缩写。这一点在下文中会变得更加清晰。

4.7 PAT 原理

你可能已经注意到，同样的符号 $\rightarrow$ 既用于^{Implication}**蕴含**（例如 `False → True`），也作为^{Type Constructor}**类型构造子**（例如 $\mathbb{Z} \rightarrow \mathbb{N}$ ）。如果没有^{Context}**语境**，我们无法分辨 $a \rightarrow b$ 是指一个定义域为 a 且陪域为 b 的函数类型，还是命题「 a 蕴含 b 」。

事实证明，这两个概念不仅在形式上「看起来」相同，它们「确实」是同一个东西。这被称为^{PAT Principle}**PAT 原理**，其中 PAT 是一个双重助记词：

$$\begin{array}{ll} \text{Propositions as Types} & \text{Proofs as Terms} \\ \text{PAT} = \text{命题即类型} & \text{PAT} = \text{证明即项} \end{array}$$

此外，由于类型也是项，这也意味着命题也是项。然而，PAT 并不是一个四重助记词（~~PAT = 证明即类型~~，^{Proofs as Types}**证明即类型**）。另请注意，并非所有类型都是命题，也并非所有项都是证明。

通过使用项和类型来表示证明和命题，依值类型论实现了概念上的极大精简。
问题

「H 是 P 的证明吗？」

变得等同于

「项 H 的类型是 P 吗？」

因此，在 Lean 内部，并没有专门的证明检查器，只有类型检查器。证明由类型检查器进行检查。

让我们逐一回顾这些主要实体。我们使用数学变量 σ 和 τ 表示类型；使用 P 和 Q 表示命题；使用 t 、 u 和 x 表示项；使用 h 、 G 和 H 表示证明。

从「命题即类型」开始，对于类型，我们有：

- $\sigma \rightarrow \tau$ 是从 σ 到 τ 的函数类型。
- $(x : \sigma) \rightarrow \tau[x]$ 是从 $x : \sigma$ 到 $\tau[x]$ 的依值函数类型。

相比之下，对于命题，我们有：

- $P \rightarrow Q$ 可以读作「P 蕴含 Q」，或读作将 P 的证明映射到 Q 的证明的函数类型。
- $\forall x : \sigma, Q[x]$ 可以读作「对于所有的 x, Q[x]」，或读作类型为 $(x : \sigma) \rightarrow Q[x]$ 的函数类型，它将类型为 σ 的值 x 映射到 Q[x] 的证明。

继续看「证明即项」，对于项，我们有：

- 常量是一个项。
- 变量是一个项。
- $t\ u$ 是函数 t 对参数 u 的应用。
- $\text{fun } x \mapsto t[x]$ 是一个将 x 映射到 $t[x]$ 的函数。

相比之下，对于证明（即证明项），我们有：

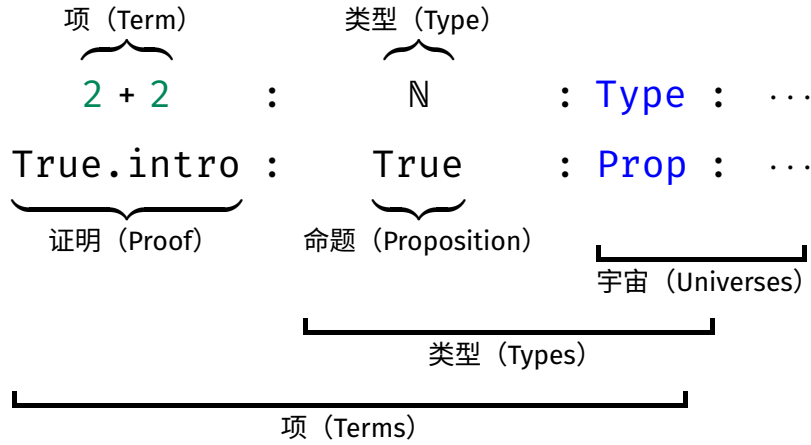
- 定理或假设的名称是一个证明。
- $H\ t$ 是一个证明，它通过项 t 对证明 H 陈述中的首个 $\forall$ 量词进行^{Instantiate}实例化。
- $H\ G$ 是一个证明，它通过证明 G ^{Discharge}解除了 H 陈述中的首个前提。这种操作被称为^{Modus Ponens}肯定前件。
- $\text{fun } h : P \mapsto H[h]$ 是 $P \rightarrow Q$ 的一个证明，前提是对于所有 $h : P$, $H[h]$ 都是 Q 的证明。
- $\text{fun } x : \sigma \mapsto H[x]$ 是 $\forall x : \sigma, Q[x]$ 的一个证明，前提是对于所有 $x : \sigma$, $H[x]$ 都是 Q[x] 的证明。

最后两种情况由 Fun' 规则支持。与原始证明项不同，在结构化证明中，我们会使用 assume 或 fix 来代替 fun，且为了可读性，我们通常还想使用 show 重复一遍结论，如下所示：

```
theorem case_4 :
  P → Q :=
  assume h : P
  show Q from
    H[h]
```

```
theorem case_5 :
  ∀x : σ, Q[x] :=
  fix x : σ
  show Q[x] from
    H[x]
```

依值类型论的术语可能会让人感到相当困惑，因为某些词具有狭义和广义之分。下面的图表展示了重要词汇的各种含义：



从广义上讲，任何表达式都是一个^{Term}项，任何可能出现在^{Typing Judgment}类型判断(:) 右侧的表达式都是一个^{Type}类型，而任何出现在类型判断右侧，且其左侧为一个类型的表达式，都是一个^{Universe}宇宙。这与将 $t : u$ 读作「t 的类型为 u」以及宇宙是类型的类型这一概念是一致的。

根据 PAT 原理，一些 Lean 命令虽然有两个名称，但本质上是相同的。fix 和 assume 就是这种情况；事实上，它们都是 LoVelib 中 fun 的别名。

还有一些成对出现的命令，其行为略有不同，例如 def/theorem 和 let/have。根本区别在于：当我们定义某个函数或数据时，我们不仅关心其类型，还关心其函数体——也就是行为。另一方面，一旦我们证明了一个定理，证明本身就变得^{Irrelevant}无关紧要了。唯一重要的是^{Proof Irrelevance}存在一个证明。我们将在第 ?? 章回到证明无关性这一话题。

下表总结了策略证明、结构化证明和^{Raw Proof Terms}原始证明项之间的区别：

策略证明	结构化证明	原始证明项
<code>intro x</code>	<code>fix x : τ</code>	<code>fun x ↦</code>
<code>intro h</code>	<code>assume h : P</code>	<code>fun h ↦</code>
<code>have k := H</code>	<code>have k := H</code>	<code>(fun k ↦ ...) H</code>
<code>let x := t</code>	<code>let x := t</code>	<code>(fun x ↦ ...) t</code>
<code>exact (H : P)</code>	<code>show P from H</code>	<code>H : P</code>
<code>calc</code>	<code>calc</code>	<code>calc</code>

注意，后跟 u 的 `let x := t` 本质上是编写 `(fun x ↦ u) t` 的一种方式，只不过^{Reduction}β-归约被禁用了。

4.8 通过模式匹配与递归进行归纳

在第 3.6 节中，我们回顾了使用 `induction` 策略来进行归纳证明的方法。另一种更灵活的风格依赖于模式匹配和 ^{PAT Principle}**PAT 原理**。

回想一下在第 2.2.3 节末尾给出的列表 ^{Reverse}**反转** 的定义：

```
def reverse {α : Type} : List α → List α
| []      => []
| x :: xs => reverse xs ++ [x]
```

事实上，`reverse` 在 Lean 的标准库中以 `List.reverse` 的形式存在，其定义更高效，但对于推理来说不那么方便。一个值得证明的有用性质是 `reverse` 是它自身的逆：对于所有列表 `xs`，均有 `reverse (reverse xs) = xs`。然而，如果我们尝试通过归纳法证明它，很快就会遇到障碍。其 ^{Induction Step}**归纳步骤**为：

```
: ∀ xs, reverse (reverse xs) = xs ⊢ reverse (reverse xs ++ [x]) = x ::
xs ih
```

请注意，在这个双重 `reverse` 的「夹层」中出现了令人不悦的 `++ [x]`。我们需要一种方法将外层的 `reverse`「分配」到 `++` 上，以获得一个能与归纳假设左侧匹配的项。诀窍是证明并使用以下定理：

```
theorem reverse_append {α : Type} :
  ∀ xs ys : List α,
    reverse (xs ++ ys) = reverse ys ++ reverse xs
| [],      ys => by simp [reverse]
| x :: xs, ys => by simp [reverse, reverse_append xs]
```

该定理的证明与其说是证明，倒不如说更像是一个递归函数的定义。左侧的模式 `[]` 和 `x :: xs` 对应于 ^{Constructor} $\forall$ 量化变量 `xs` 的两个**构造子**。在每个 `=>` 符号的右侧是对应情况的证明。我们可以进行模式匹配的变量是那些出现在 $\forall$ 量词中的变量，按出现顺序排列（此处为 `xs` 和 `ys`）。在归纳步的证明内部，归纳假设使用的名称与我们正在证明的定理名称相同（`reverse_append`）。

我们显式地将 `xs` 作为参数传递给归纳假设。这限制了该假设，使其仅适用于 `xs` 而不适用于其他列表。特别是，这确保了定理不会应用于 `x :: xs`，否则会导致循环证明：「为了证明 `reverse_append (x :: xs)`，请使用 `reverse_append (x :: xs)`」。Lean 的终止性检查器会发现该证明是非良基的并报错，但我们希望避免这种情况。此外，显式参数 `xs` 也可以作为文档，实际上是在说：「我们需要的该定理的唯一递归实例是在 `xs` 上的实例」。

作为参考，^{Tactical Proof}**策略式证明**如下：

```
theorem reverse_append_tactical {α : Type} (xs ys : List
α) :
```

```
reverse (xs ++ ys) = reverse ys ++ reverse xs :=
by
  induction xs with
  | nil          => simp [reverse]
  | cons x xs' ih => simp [reverse, ih]
```

如果我们用 `[y]` 代替 `ys`，该定理同样是可证明且有用的。但尽可能一般地陈述定理是一个好习惯，这会使库更具可重用性。此外，在归纳证明中，为了获得足够强的归纳假设，这通常是必需的。一般而言，寻找合适的归纳方式和定理需要思考和创造力。

Lean 支持对多个变量同时进行模式匹配（例如上面的 `xs` 和 `ys`）。此时模式之间用逗号分隔。其一般格式为：

```
theorem name (参数1 : 类型1) ... (参数m : 类型m) :
  语句
| 模式1 => 证明1
  :
| 模式n => 证明n
```

注意它与 `def` 的语法（第 2.2 节）有很强的相似性。事实上，这两个命令几乎完全相同，但 `theorem` 认为定义的项或证明是^{Opaque}不透明的，而 `def` 则保持其^{Transparent}透明。由于一旦定理被证明，实际的证明就不再重要了（第 ?? 章），因此稍后无需对其进行展开。`let` 和 `have` 之间也存在类似的区别。

根据 PAT 原理，通过模式匹配和递归进行的归纳证明等同于一个递归^{Proof Term}证明项。当我们调用归纳假设时，实际上只是在递归地调用一个递归函数。这解释了为什么归纳假设与我们证明的定理同名。Lean 的终止性检查器被用来建立归纳证明的良基性。

有了 `reverse_append` 定理，我们可以回到最初的目标：

通过模式匹配和递归进行归纳在 Lean 用户中很受欢迎。它的主要优点是语法方便，且支持^{Well-founded Induction}良基归纳，这比 `induction` 策略（第 3.7 节）提供的^{Structural Induction}结构归纳更强大。然而，在本指南中，我们不需要良基归纳的全部威力。此外，出于微妙的逻辑原因，通过模式匹配和递归进行的归纳不适用于^{Inductive Predicates}归纳谓词，这也是第 6 章的主题。基于这些原因，我们通常更喜欢使用 `induction` 策略。

4.9 新引入的 Lean 结构小结

证明命令

<code>assume</code>	声明假设
<code>calc</code>	通过传递性组合证明

<code>fix</code>	固定变量
<code>have</code>	声明中间定理
<code>let</code>	引入局部定义
<code>show</code>	声明目标

策略

<code>calc</code>	通过传递性组合证明
<code>have</code>	声明中间定理
<code>let</code>	引入局部定义

第二部分

函数式与逻辑编程

函数式编程

我们将深入探讨带类型函数式编程的本质：归纳类型、归纳证明、递归函数、模式匹配、结构体（记录）以及类型类。这里涵盖的概念在《Lean 4 定理证明》[12] 的第 7 到 10 章中有更详细的描述。

5.1 归纳类型

归纳类型模仿了带类型函数式编程语言（如 Haskell、ML、OCaml）中的数据类型。它们也让人联想到 Scala 中的密封类。在第 2 章中，我们看到了一些基本的归纳类型：自然数、算术表达式类型以及有限列表。在本章中，我们将重新审视列表并研究二叉树。我们还将简要了解长度为 n 的向量，这是一种依值类型。

回想一下作为归纳类型的自然数定义：

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

这定义了类型 `Nat` 以及两个常量 `Nat.zero` 和 `Nat.succ`，它们被称为构造子。该定义还断言了构造子的一些属性。此外，它还引入了进一步的常量，用于在内部支持归纳和递归。

正如我们在第 2.1 节中看到的，归纳类型是一种其成员全部由（且仅由）有限次应用其构造子所构建的值组成的类型。格言：

- **无杂质**：类型中不包含除使用构造子可表达的值之外的任何值。
- **无混淆**：使用不同构造子组合构建的值是不同的。

对于自然数,「无杂质」意味着不存在无法使用 `Nat.zero` 和 `Nat.succ` 的有限组合来表达的特殊值 (如 -1 、 ε 、 ∞ 或 `NaN`);「无混淆」则确保了对于所有 n , `Nat.zero` $\neq$ `Nat.succ n`, 且 `Nat.succ` 是^{Injective}单射的。

此外,归纳类型的值总是有限的。无限项

`Nat.succ (Nat.succ (Nat.succ (Nat.succ ...)))`

并不是一个值。同样地,也不存在一个值 n 使得 `Nat.succ n = n`, 这一点我们将在下文证明。

归纳类型使用起来非常方便,因为它们支持归纳和递归,且它们的构造子表现良好;但并非所有类型都能定义为归纳类型。特别地,诸如 $\mathbb{Q}$ (有理数) 和 $\mathbb{R}$ (实数) 等数学类型需要更复杂的构造,这些构造基于^{Quotienting}商构造和^{Subtyping}子类型化。这将在第 ?? 和第 ?? 章中详细说明。

5.2 结构归纳

「结构归纳」是数学归纳法向任意归纳类型的推广。为了通过对 n 的结构归纳来证明目标 $n : \mathbb{N} \vdash P[n]$, 只需证明两个子目标, 传统上称为^{Base Case}基本情况和^{Induction Step}归纳步骤:

$\vdash P[0]$

$k : \mathbb{N}, ih : P[k] \vdash P[k + 1]$

当然,我们也可以写 `Nat.zero` 和 `Nat.succ k` 来代替 0 和 $k + 1$ 。

通常情况下,情况会更复杂。目标可能包含一些不依赖于 n 的额外假设 (例如 Q), 以及一些依赖于 n 的假设 (例如 $R[n]$)。假设我们每种假设各有一个, 这就给出了初始目标:

$hQ : Q, n : \mathbb{N}, hR : R[n] \vdash S[n]$

对 n 进行结构归纳会产生两个子目标:

$hQ : Q, hR : R[0] \vdash S[0]$

$hQ : Q, k : \mathbb{N}, ih : R[k] \rightarrow S[k], hR : R[k + 1] \vdash S[k + 1]$

假设 Q 只是简单地从初始目标原封不动地继承下来, 而 $R[n] \vdash S[n]$ 处理起来几乎就像目标的目的变成了 $R[n] \rightarrow S[n]$ 。在上面的第一个例子中, 令 $P[n] := R[n] \rightarrow S[n]$ 即可轻松验证这一点。由于这种通用格式非常冗长且几乎不包含额外信息 (既然我们已经理解了其工作原理), 从现在起, 我们将以尽可能简单的形式呈现目标, 不带额外假设。

对于列表, 给定目标 $xs : \text{list } \alpha \vdash P[xs]$, 对 xs 进行结构归纳得到:

$\vdash P[[]]$

$y : \alpha, ys : \text{list } \alpha, ih : P[ys] \vdash P[y :: ys]$

当然，我们也可以写 `List.nil` 和 `List.cons y ys`。没有与 `y` 关联的归纳假设，因为 `y` 不是列表类型的。

对于算术表达式，基本情况是：

$$i : \mathbb{Z} \vdash P[\text{AExp.num } i] \qquad x : \text{String} \vdash P[\text{AExp.var } x]$$

归纳步骤是：

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.add } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.sub } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.mul } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.div } e_1 \ e_2]$$

注意关于 e_1 和 e_2 的两个归纳假设。

通常情况下，结构归纳会为每个构造子产生一个子目标。在每个子目标中，对于我们正在进行归纳的类型的构造子参数，归纳假设都是可用的。

给定一个归纳类型 τ ，计算子目标的步骤总是相同的：

1. 将 $P[]$ 中的空缺替换为应用于新变量（例如 $y :: ys$ ）的每个可能的构造子，产生与构造子数量相同的子目标。
2. 将这些新变量（例如 y 、 ys ）添加到局部语境中。
3. 为所有类型为 τ 的新变量添加归纳假设。

作为一个例子，我们将证明对于所有 $n : \mathbb{N}$ ， $\text{Nat.succ } n \neq n$ 。我们从一个非正式证明开始：

证明采用对 n 的结构归纳。

情况 0: 我们必须证明 $\text{Nat.succ } 0 \neq 0$ 。这遵循归纳类型构造子的「无混淆」性质。

情况 $\text{Nat.succ } k$: 归纳假设是 $\text{Nat.succ } k \neq k$ 。我们必须证明 $\text{Nat.succ } (\text{Nat.succ } k) \neq \text{Nat.succ } k$ 。通过 Nat.succ 的单射性，我们得到 $\text{Nat.succ } (\text{Nat.succ } k) = \text{Nat.succ } k$ 等价于 $\text{Nat.succ } k = k$ 。因此，只需证明 $\text{Nat.succ } k \neq k$ ，这恰好对应于归纳假设。 $\square$

请注意此非形式证明的主要特征，你应该在自己的非形式论证中努力复现这些特征：

- 证明以明确声明我们正在进行的证明类型开始（例如，哪种归纳以及对哪个变量进行归纳）。
- 各个情况都被清晰地识别出来，并且对于每个情况，都陈述了目标的目的和假设。

- 明确调用了证明所依赖的关键定理（例如，`Nat.succ` 的单射性）。

现在让我们在 Lean 中完成这个证明：

```
theorem Nat.succ_neq_self (n : ℕ) :
  Nat.succ n ≠ n :=
by
  induction n with
  | zero      => simp
  | succ n' ih =>
    intro hsucc
    apply ih
    apply Nat.succ.inj hsucc
```

这个证明有些刻意，因为它可以用单个 `simp` 调用所取代，但它仍然具有启发意义。

5.3 结构递归

Structural Recursion

结构递归是一种递归形式，它允许我们从进行递归的值中剥离出一个构造子。

下面的阶乘函数就是结构递归的：

```
def fact : ℕ → ℕ
| 0      => 0
| n + 1 => (n + 1) * fact n
```

我们在这里剥离的构造子是 `Nat.succ`（写作 `+ 1`），这样的函数保证在递归停止前只调用自身有限次。例如，`fact 12345` 将调用自身 12345 次。该函数被称为是 ^{Terminate}**终止**的，这一性质有助于确保逻辑一致性。

在结构递归中，等式的数量与构造子的数量相同。初学者往往倾向于提供额外的冗余情况，如下例所示：

```
def factThreeCases : ℕ → ℕ
| 0      => 0
| 1      => 1
| n + 1 => (n + 1) * factThreeCases n
```

抵制这种诱惑对你最有好处。定义中的情况越多，对其进行推理的工作量就越大。请记住那句格言：「一个好的定义抵得上三个定理」。

对于结构递归函数，Lean 可以自动证明其终止性。对于更通用的递归模式，终止性检查可能会失败。有时失败是有充分理由的，如下例所示：

```
-- fails
def illegal : ℕ → ℕ
```

```
| n => illegal n + 1
```

如果 Lean 接受这个定义,我们就可以利用它来证明 $0 = 1$,只需从等式 $\text{illegal } n = \text{illegal } n + 1$ 的两边减去 $\text{illegal } n$ 。从 $0 = 1$, 我们可以推导出 `False`, 再从 `False`, 我们可以推导出任何结论。显然, 我们不要这样的结果。

如果我们使用 `opaque` 和 `axiom`, 那就没有任何办法了:

```
opaque immoral : ℕ → ℕ

axiom immoral_eq (n : ℕ) :
  immoral n = immoral n + 1

theorem proof_of_False :
  False :=
  have hi : immoral 0 = immoral 0 + 1 :=
    immoral_eq 0
  have him :
    immoral 0 - immoral 0 = immoral 0 + 1 - immoral 0 :=
    by rw [←hi]
  have h0eq1 : 0 = 1 :=
    by simp at him
  show False from
    by simp at h0eq1
```

我们更倾向于使用 `def` 而非 `opaque` 和 `axiom` 的另一个原因是, 定义的等式可以用于计算。像 `rfl` 这样在计算意义下进行统一的策略, 在我们每次引入一个定义时都会变得更强大; 而诊断命令 `#eval` 和 `#reduce` 也可以用于已定义的常量。

细心的读者会注意到, 上面关于阶乘的定义在数学上是错误的: 无论参数是什么, `fact` 和 `factThreeCases` 竟然都返回 0。我们确实搞砸了。这些令人尴尬的错误提醒我们要「测试」我们的定义, 并「证明」它们的一些性质。虽然有缺陷的公理偶尔会出现, 但更常见的是定义未能捕获预想的概念。仅仅因为一个函数被命名为 `fact`, 并不意味着它实际上就在计算阶乘。

5.4 模式匹配表达式

模式匹配不仅可以在 `def` 命令的顶层进行, 还可以通过 `match` 表达式深入到项的内部。该构造具有以下通用语法:

```

match 项1, ..., 项m with
  | 模式11, ..., 模式1m => 结果1
  ⋮
  | 模式n1, ..., 模式nm => 结果n

```

该构造让人联想到一些现代编程语言中的 match。上述 match 表达式的含义如下：

考虑项 项₁, ..., 项_m。

如果它们的形式分别为模式₁₁, ..., 模式_{1m},
则得到 结果₁。

⋮

如果它们的形式分别为模式_{n1}, ..., 模式_{nm},
则得到 结果_n。

模式可以包含变量、构造子和匿名占位符 (`_`)。表达式 结果_i 可以引用相应模式中引入的变量。

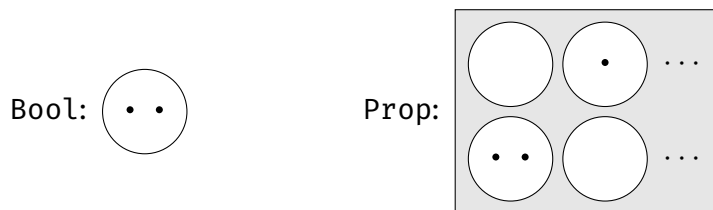
下面的函数定义演示了表达式内部模式匹配的语法：

`bcount` 函数计算列表中满足给定谓词 `p` 的元素数量。该谓词的上域是 `Bool`。作为一般规则，我们将在程序中使用 **布尔值** 类型 `Bool`，而在陈述程序性质时使用 **命题** 类型 `Prop`。 `Bool` 类型的两个值被称为 `false` 和 `true`（以小写字母形式）。

¹

逻辑**联结词**被称为 `or`（中缀：`||`）、`and`（中缀：`&&`）以及 `not`（前缀：`!`）。

下图展示了 `Bool` 和 `Prop` 的解释：



点代表元素，圆圈代表类型，矩形代表类型的类型，即**宇宙**。我们可以看到，`Bool` 被解释为具有两个值的集合，而 `Prop` 由无限多个命题（即类型）组成，每个命题都有零个或多个证明（即元素）。我们将在第 6 章中完善这张图。

我们不能对命题（类型为 `Prop`）进行模式匹配，但我们可以使用 `if-then-else` 来代替。例如，自然数上的 `min` 运算符可以定义如下：

¹Lean 也允许我们使用 `True` 和 `False`，但它们会被隐式地从 `Prop` 转换为 `Bool`。我们通常建议避免这种隐式强制转换。遗憾的是，目前没有办法禁用它们。

```
def min (a b : ℕ) : ℕ :=
  if a ≤ b then a else b
```

这需要一个^{Decidable}**可判定**(即可执行)的命题。对于 $\leq$ 来说确实如此：给定具体的参数(如 35 和 49)，Lean 可以将 $35 \leq 49$ 约化为 True。Lean 使用一种称为^{Type Class}**类型类**的机制来跟踪可判定性，下文将对此进行解释。

5.5 结构体

Lean 提供了定义^{Structure}**结构体**(也称为^{Record}**记录**)的便捷语法。它们本质上是具有一些语法糖的非递归、单构造子归纳类型。

下面的定义引入了一个名为 RGB 的结构体，它有三个类型为 $\mathbb{N}$ 的字段，分别名为 red、green 和 blue：

```
structure RGB where
  red    : ℕ
  green  : ℕ
  blue   : ℕ
```

此定义的效用大致相当于以下命令：

```
inductive RGB : Type where
  | mk : ℕ → ℕ → ℕ → RGB

def RGB.red : RGB → ℕ
  | RGB.mk r _ _ => r

def RGB.green : RGB → ℕ
  | RGB.mk _ g _ => g

def RGB.blue : RGB → ℕ
  | RGB.mk _ _ b => b
```

我们可以将一个新结构体定义为现有结构体的扩展。下面的定义用第四字段 alpha 扩展了 RGB：

```
structure RGBA extends RGB where
  alpha : ℕ
```

定义结构体的通用语法为：

```
structure 结构体名 (参数1 : 类型1) ... (参数k : 类型k)
  [extends 结构体1, \ldots, 结构体m] where
```

```

    字段名1 : 字段类型1
    ⋮
    字段名n : 字段类型n

```

参数 参数₁、...、参数_k 实际上也是额外的字段，但与 字段名₁、...、字段名_n 不同，它们存储在类型中，作为类型构造子 (结构体名) 的参数。

可以使用多种语法来指定值：

```

def pureRed : RGB :=
  RGB.mk 0xff 0x00 0x00

def pureGreen : RGB :=
  { red    := 0x00
    green  := 0xff
    blue   := 0x00 }

def semitransparentGreen : RGBA :=
  { pureGreen with
    alpha := 0x7f }

```

semitransparentGreen 的定义从 pureGreen 中复制了所有值，但显式设置了 alpha 字段。

接下来，我们定义一个名为 shuffle 的操作：

```

def shuffle (c : RGB) : RGB :=
  { red    := RGB.green c
    green  := RGB.blue c
    blue   := RGB.red c }

```

该定义依赖于生成的选择器 RGB.red、RGB.green 和 RGB.blue。除了 RGB.red c，我们也可以写成 c.red，其他字段同理。有时我们会在 Lean 的输出中看到这种记法，即使我们自己不使用它。

连续三次应用 shuffle 就相当于完全没有应用它：

```

theorem shuffle_shuffle_shuffle (c : RGB) :
  shuffle (shuffle (shuffle c)) = c :=
  by rfl

```

5.6 类型类

类型类是由 Haskell 推广的一种机制，目前存在于多个证明助手中。在 Lean 中，²
Type Class
类型类是一种结合了抽象常量及其性质的结构体类型。²

通过为常量提供具体定义并证明其性质成立，可以将一个类型声明为类型类的实例 (instance)。根据类型，Lean 会检索相关的实例。

一个简单的例子是 `Inhabited` 类型类，它只需要一个常量 `Inhabited.default`，而不需要任何性质：

```
class Inhabited (α : Type) : Type where
  default : α
```

类的定义语法与结构体相同，只是使用了关键字 `class` 代替了 `structure`。参数 α 代表可以作为该类成员的任意类型。这个特定的类型类具有单个参数和单个字段，但通常情况下，一个类型类可以具有多个参数和多个字段。

任何具有至少一个元素的类型都可以被注册为 `Inhabited` 类型类的实例。例如，我们可以通过选择一个任意数字作为默认值来注册 `N`：

```
instance Nat.Inhabited : Inhabited N :=
  { default := 0 }
```

这指定了一个名为 `Nat.Inhabited`、类型为 `Inhabited N` 的值。因为我们使用的是关键字 `instance` 而不是 `def`，所以这个结构体的值被注册为「规范实例」，以便在需要类型为 `Inhabited N` 的结构体时使用。在存储类型类实例的全局表中，现在有一个条目：

$$\text{Inhabited } N \mapsto \text{Nat.Inhabited}$$

对于列表，即使 α 不包含任何居留元，空列表也是一个显而易见的默认值：

```
instance List.Inhabited {α : Type} : Inhabited (List α)
:=
  { default := [] }
```

这在全局表中添加了以下条目：

$$\text{Inhabited } (\text{List } ?\alpha) \mapsto \text{List.Inhabited}$$

作为一个例子，观察到类型为 $\alpha \rightarrow \beta$ 的有限函数可以用它们的函数表来表示，函数表的类型为 $\beta \times \dots \times \beta$ (包含 $|\alpha|$ 个 β 的副本)。因此， $|\alpha \rightarrow \beta| = |\beta|^{|\alpha|}$ 。要使结果为 0，必须满足 $|\beta| = 0$ 且 $|\alpha| \neq 0$ 。换句话说，如果 (1) β 有居留元或 (2) α 「没有」居留元，则类型 $\alpha \rightarrow \beta$ 有居留元。我们关注情况 (1)：

²尽管名称如此，Lean 的类型类机制与 Scala 的隐式参数 (implicit arguments) 的关系比与 Haskell 的类型类的关系更密切。

```
instance Fun.Inhabited {α β : Type} [Inhabited β] :
  Inhabited (α → β) :=
  { default := fun a : α ↦ Inhabited.default }
```

该实例本身依赖于相同类型类但在不同类型上的实例。这种情况经常发生。

偶对 ^{Pair} 的类型 $\alpha \times \beta$ ，也称为**乘积类型** ^{Product Type}，包含形式为 (a, b) 的值，其中 $a : \alpha$ 且 $b : \beta$ 。给定一个偶对 $ab : \alpha \times \beta$ ，可以通过编写 `Prod.fst ab` 和 `Prod.snd ab` 来提取其第一和第二分量。要提供 $\alpha \times \beta$ 的居留元，我们需要 α 的居留元和 β 的居留元：

```
instance Prod.Inhabited {α β : Type}
  [Inhabited α] [Inhabited β] :
  Inhabited (α × β) :=
  { default := (Inhabited.default, Inhabited.default) }
```

使用 `Inhabited` 类型类，我们可以定义列表上的 ‘head’ 操作：即返回列表第一个元素的函数。由于空列表不包含任何元素，因此在这种情况下没有可以返回的有意义的值。给定一个属于 `Inhabited` 类型类的类型，我们可以简单地返回其默认值：

```
def head {α : Type} [Inhabited α] : List α → α
| []      => Inhabited.default
| x :: _ => x
```

通过编写 `[Inhabited α]`，我们要求 α 属于 `Inhabited`。这允许我们在定义中访问 `Inhabited.default`。

语法 `[Inhabited α]` 为 `head` 常量添加了一个隐式参数。但与其他隐式参数不同，Lean 会通过所有已声明的实例进行「类型类搜索」，以确定该参数的值。因此，在运行命令

```
#eval head ([] : List ℕ)
```

时，Lean 会寻找 `Inhabited ℕ` 的实例并找到我们上面声明的实例 `Nat.Inhabited`。在那次声明中，我们将 `default` 设置为 `0`，因此这就是 `#eval` 打印出的内容。如果有多个实例适用且 Lean 选择了错误的一个，我们可以使用 `@` 语法将类型类参数转换为显式参数，并提供所需的类型类实例。

让我们仔细看看 `Inhabited.default`：

```
Inhabited.default {α : Type} [Inhabited α] : α
```

注意方括号 `[ ]` 的使用。当我们使用 `Inhabited.default α` 来定义 `head` 时，Lean 在注册实例的全局表和局部语境中寻找 `Inhabited α` 的实例。全局表包含 `Inhabited ℕ` 的条目，但没有匹配 `Inhabited α` 的条目。另一方面，局部语境包含一个类型为 `Inhabited α` 的匿名参数，可以使用它。

Lean 的核心库对 `List.head` 的定义与我们完全相同。在实践中，几乎所有的类型都是非空的（显著的例外是 `False`），所以 `Inhabited` 限制几乎不是问题。

我们可以证明关于 `Inhabited` 类型类的抽象定理，例如：

```
theorem head_head {α : Type} [Inhabited α] (xs : List α)
:
  head [head xs] = head xs
```

为了在类型为 `List α` 的列表上使用运算符 `head`，需要假设 `[Inhabited α]`。如果我们省略这个假设，Lean 会抛出一个错误，告诉我们「类型类合成」失败。

还有更多只有常量但没有性质的类型类，包括：

<code>class Zero (α : Type)</code> <code>where</code> <code>zero : α</code>	<code>class One (α : Type)</code> <code>where</code> <code>one : α</code>
<code>class Neg (α : Type)</code> <code>where</code> <code>neg : α → α</code>	<code>class Inv (α : Type)</code> <code>where</code> <code>inv : α → α</code>
<code>class Add (α : Type)</code> <code>where</code> <code>add : α → α → α</code>	<code>class Mul (α : Type)</code> <code>where</code> <code>mul : α → α → α</code>

Syntactic Type Classes

这些**句法类型类**引入了在不同语境下具有不同语义的常量。例如，`one` 可以代表自然数 1、整数 1、实数 1、单位矩阵以及许多其他「一」的概念。这些类型类的主要目的是为丰富的代数类型类层次结构（群、Monoids幺半群、环、域等）奠定基础，并允许 Overloading重载常见的数学符号，如 `+`、`*`、`0`、`1` 和 `- 1`。

Semantic Type Classes

句法类型类不会对可以声明为实例的类型强加严格的限制。相比之下，**语义类型类**包含约束给定常量行为的性质。

在第 3.6 节中，我们遇到了以下语义类型类：

```
class Std.Commutative (α : Type) (f : α → α → α) where
  comm : ∀ a b, f a b = f b a

class Std.Associative (α : Type) (f : α → α → α) where
  assoc : ∀ a b c, f (f a b) c = f a (f b c)
```

这一次，关联不再是从类型到常量，而是从类型「以及一个函数」到性质。Lean 并不介意这种称谓上的偏差：尽管它们被称为类型类，但 Lean 的类型类非常灵活，可以用来表示各种约束。

从概念上讲, `Std.Commutative` 是一个三元组 (α, f, comm) 的依值类型, `Std.Associative` 也是如此。 f 的类型取决于 α , 而 comm 的类型取决于 α 和 f 。尽管它们是参数, 但 α 和 f 也与 comm 一起存储。

在第 3.6 节中, 我们将 $\mathbb{N}$ 上的 `add` 函数注册为一个可交换且可结合的操作:

```
instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }
```

每当我们尝试访问 `@Std.Commutative.comm N add` 时, 都会得到 `add_comm`, 对于 `@Std.Associative.assoc N add` 也是如此。策略 `ac_refl` 会尝试查找问题中所有二元运算符的 `comm` 和 `assoc` 性质, 并在性质存在时利用它们。

定义类型类的通用语法如下:

```
class 类名 (参数1 : 类型1) ... (参数k : 类型k)
  [extends 结构体1, ..., 结构体m] where
  常量名1 : 常量类型1
  ⋮
  常量名n : 常量类型n
  性质名1 : 命题1
  ⋮
  性质名p : 命题p
```

实例化类型类的通用语法如下:

```
instance 实例名 : 类型类 参数 :=
  { 常量1 := 定义1,
    ⋮
    常量n := 定义n,
    性质1 := 证明1,
    ⋮
    性质p := 证明p }
```

5.7 列表

Lean 提供了丰富的有限列表函数库。在本节中, 我们将回顾其中的一些, 并定义一些我们自己的函数; 这些练习有助于我们熟悉 Lean 中的函数式编程。

在第一个例子中, 我们对一个列表进行分情况讨论:

```

theorem head_head_cases {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
by
  cases xs with
  | nil      => rfl
  | cons x xs' => rfl

```

该证明依赖于 `cases`，它是 `induction` 的近亲。它对其参数进行分情况讨论，但不生成归纳假设。调用 `cases xs` 将目标 $\vdash P[xs]$ 转换为两个子目标： $\vdash P[[]]$ 和 $\vdash P[x :: xs']$ 。我们本也可以使用 `induction xs`。如果你在 `induction` 和 `cases` 之间犹豫不决，你总是可以选择 `induction`，然后看看你是否需要归纳假设——如果你不需要它，通过将 `induction` 替换为 `cases` 来记录这一事实是一种良好的风格。

在结构化证明中，我们可以使用 `match` 表达式来执行分情况讨论：

```

theorem head_head_match {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
match xs with
| List.nil      => by rfl
| List.cons x xs' => by rfl

```

在下一个例子中，我们展示了如何利用构造子的^{Injectivity}单射性。当等式两边都应用了相同的构造子时，`cases` 策略会利用单射性来简化等式。在下面的证明中，简化前的等式是 $x :: xs = y :: ys$ ：

```

theorem injection_example {α : Type} (x y : α) (xs ys :
  List α)
  (h : x :: xs = y :: ys) :
  x = y ∧ xs = ys :=
by
  cases h
  simp

```

`cases` 策略在整个目标中将 `y` 替换为 `x`，将 `ys` 替换为 `xs`，产生子目标

$$\vdash x = x \wedge xs = xs$$

这可以由 `simp` 轻松证明。

当构造子不同时，`cases` 策略也可用于识别不可能的情况：

```

theorem distinctness_example {α : Type} (y : α) (ys :
  List α)
  (h : [] = y :: ys) :
  False :=
  by cases h

```

接下来，我们定义列表上的^{Map Function}映射函数：这种函数将其参数 f （其本身也是一个函数）应用到集合中存储的所有元素上。

```

def map {α β : Type} (f : α → β) : List α → List β
| []      => []
| x :: xs => f x :: map f xs

```

请注意，由于 f 在递归调用中不会改变，我们将其作为整个定义的参数。另一种选择（也是在递归调用中会改变的参数的唯一选择）如下：

```

def mapArgs {α β : Type} : (α → β) → List α → List β
| _, []      => []
| f, x :: xs => f x :: mapArgs f xs

```

映射函数的一个基本性质是，如果其参数是恒等函数 ($\text{fun } x \mapsto x$)，则它们没有效果：

```

theorem map_ident {α : Type} (xs : List α) :
  map (fun x ↦ x) xs = xs :=
  by
    induction xs with
    | nil      => rfl
    | cons x xs' ih => simp [map, ih]

```

另一个基本性质是，连续的映射可以压缩为单个映射，其参数是所涉及函数的复合：

```

theorem map_comp {α β γ : Type} (f : α → β) (g : β → γ)
  (xs : List α) :
  map g (map f xs) = map (fun x ↦ g (f x)) xs :=
  by
    induction xs with
    | nil      => rfl
    | cons x xs' ih => simp [map, ih]

```

在引入新操作时，展示这些操作与其他操作组合使用时的行为是非常有用的。示例如下：

```

theorem map_append {α β : Type} (f : α → β)
  (xs ys : List α) :
  map f (xs ++ ys) = map f xs ++ map f ys :=
by
  induction xs with
  | nil          => rfl
  | cons x xs' ih => simp [map, ih]

```

值得注意的是,最后三个证明在文本上是完全相同的。这些是典型的induction-rfl-simp 证明。

下一个列表操作删除列表的第一个元素,返回尾部:

```

def tail {α : Type} : List α → List α
| []      => []
| _ :: xs => xs

```

对于 [], 我们简单地返回 [] 作为其自身的尾部。

与 tail 对应的是提取列表第一个元素的函数。我们已经在第 5.6 节中回顾了一种使用Inhabited 类型类的方案。另一种可能的定义是使用 Option 包装:

```

def headOpt {α : Type} : List α → Option α
| []      => Option.none
| x :: _ => Option.some x

```

类型 Option α 有两个构造子:Option.none 和 Option.some a,其中 a : α。当没有有意义的值可以返回时,我们使用 Option.none, 否则使用 Option.some。我们可以将 Option.none 视为函数式编程中的空指针,但与空指针(和空引用)不同,类型系统会防止不安全的解引用。要检索存储在 Option 中的值,我们必须进行模式匹配。示意图如下:

```

match headOpt xs with
| Option.none    => handleTheError
| Option.some x => doSomethingWithValue x

```

我们不能简单地写 doSomethingWithValue (headOpt xs), 因为这会产生类型错误。类型系统强迫我们考虑错误处理。

利用依值类型的力量,我们有另一种实现偏函数的方法——即,我们可以指定一个前置条件。调用者必须随后传递一个前置条件得到满足的证明作为参数:

```

def headPre {α : Type} : (xs : List α) → xs ≠ [] → α
| [],      hxs => by simp at *
| x :: _, hxs => x

```

`headPre` 函数接受两个显式参数。第一个参数 `xs` 是一个列表。第二个参数 `hxs` 是 $xs \neq []$ 的证明。由于第二个参数的类型 $xs \neq []$ 取决于第一个参数，我们必须使用依值类型语法 $(xs : \text{List } \alpha) \rightarrow$ 而不是 $\text{List } \alpha \rightarrow$ ，以便我们可以命名第一个参数。函数的结果是类型为 α 的值；由于有了前置条件，不需要 `Option` 包装。

第二个参数用于排除 `xs` 为 `[]` 的情况。在这种情况下，该参数（称为 `hxs`）是 $[] \neq []$ 的证明，而这是不可能的。证明由此导出一个^{Contradiction}矛盾，并利用它导出一个任意的 α 。从矛盾中，我们可以导出任何东西，甚至是 α 的一个居留元。

然后我们可以如下调用该函数：

```
#eval headPre [3, 1, 4] (by simp)
```

这会打印出 3。第二个参数 `by simp` 是 `[3, 1, 4]` 不为 `[]` 的证明。

现在，给定两个长度相同的列表 $[x_1, \dots, x_n]$ 和 $[y_1, \dots, y_n]$ ，‘zip’ 操作构造一个由偶对组成的列表 $[(x_1, y_1), \dots, (x_n, y_n)]$ ：

```
def zip {α β : Type} : List α → List β → List (α × β)
| x :: xs, y :: ys => (x, y) :: zip xs ys
| [], _           => []
| _ :: _, []      => []
```

如果一个列表比另一个短，该函数也是有定义的。例如，`zip [a, b, c] [x, y] = [(a, x), (b, y)]`。请注意，这种具有三个情况的递归偏离了结构递归架构。列表的长度是通过递归定义的：

```
def length {α : Type} : List α → ℕ
| []      => 0
| x :: xs => length xs + 1
```

我们可以关于 ‘zip’ 结果的长度说一些有趣的事情——即，它是两个输入列表长度的最小值：

```
theorem length_zip {α β : Type} (xs : List α) (ys : List β) :
  length (zip xs ys) = min (length xs) (length ys) :=
by
  induction xs generalizing ys with
  | nil      => simp [zip, min, length]
  | cons x xs' ih =>
    cases ys with
    | nil      => rfl
    | cons y ys' => simp [zip, length, ih ys',
      min_add_add]
```

上面的证明教会了我们又一个技巧。归纳假设是

$ih : \forall ys : list \beta, length (zip\ xs\ ys) = \min (length\ xs) (length\ ys)$

为什么这里有一个 $\forall$ 量词? $induction\ xs\ generalizing\ ys$ ^{Generalized} 策略泛化了定理陈述, 使得归纳假设不再局限于证明目标中的某个固定 ys , 而是可以用于 ys 的任意值。这里需要这种灵活性, 因为我们想用 ys 的尾部 (称为 ys') 而不是 ys 本身来实例化该量词。

证明依赖于一个关于 $\min$ 函数的定理, 我们需要自己证明它:

```
theorem min_add_add (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  by
    cases Classical.em (m ≤ n) with
    | inl h => simp [min, h]
    | inr h => simp [min, h]
```

回想定义 $\min a\ b = (if\ a \leq b\ then\ a\ else\ b)$ 。要对 $\min$ 进行推理, 我们经常需要对条件 $a \leq b$ 进行分情况讨论。这可以通过使用 `cases Classical.em (a ≤ b)` 来实现。这会创建两个子目标: 一个以 $a \leq b$ 为假设, 另一个以 $\neg a \leq b$ 为假设。假设在每种情况下都可以作为 h 使用。

在结构化证明中, 有两种方法可以对命题进行分情况讨论:

```
theorem min_add_add_match (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  match Classical.em (m ≤ n) with
  | Or.inl h => by simp [min, h]
  | Or.inr h => by simp [min, h]
```

```
theorem min_add_add_if (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  if h : m ≤ n then
    by simp [min, h]
  else
    by simp [min, h]
```

我们再次看到, 用于编写函数式程序的机制 (如 `match` 和 `if-then-else`) 也可用于编写结构化证明 (毕竟证明也是项)。我们现在可以为第 4.7 节末尾呈现的表格添加几行:

策略式证明	结构化证明	原始证明项
<code>cases t</code>	<code>match t with</code>	<code>match t with</code>
<code>cases Classical.em</code>	<code>if Q then ... else</code>	<code>if Q then ... else</code>
<code>Q</code>		

我们以一个关于 `map` 和 `zip` 的分配律结束：

```
theorem map_zip {α α' β β' : Type} (f : α → α') (g : β
  → β') :
  ∀xs ys, map (fun (a, b) ↦ (f a, g b)) (zip xs ys) =
    zip (map f xs) (map g ys)
| x :: xs, y :: ys => by simp [zip, map, map_zip f g
  xs ys]
| [], _ => by rfl
| _ :: _, [] => by rfl
```

左侧的模式与 `zip` 定义中使用的模式完全对应。这比分别对 `xs` 进行归纳和对 `ys` 进行分情况讨论要简单得多，就像我们在证明 `length_zip` 时所做的那样。好的证明通常遵循它们所基于的定義的结构。

在 `zip` 的定义和 `map_zip` 的证明中，我们小心地指定了三个不重叠的模式。也可以编写具有重叠模式的等式，例如：

```
def f {α : Type} : List α → ...
| [] => ...
| xs => ... xs ...
```

由于模式是按顺序应用的，上述命令定义的函数与以下函数相同：

```
def f {α : Type} : List α → ...
| [] => ...
| x :: xs => ... (x :: xs) ...
```

我们通常建议使用后者，即更显式的风格，因为这样产生的意外更少。

5.8 二叉树

具有接受多个递归参数的构造子的归纳类型定义了树状对象。^{Binary Trees} **二叉树** 的节点最多有两个子节点。Lean 对二叉树的定义如下：

```
inductive Tree (α : Type) : Type
| nil : Tree α
| node : α → Tree α → Tree α → Tree α
```

对于二叉树，结构归纳会产生两个归纳假设，内部节点的每个子树各一个。为了通过对 t 进行结构归纳来证明目标 $t : \text{Tree } \alpha \vdash P[t]$ ，我们需要展示以下子目标：

$$\vdash P[\text{Tree.nil}]$$

$$a : \alpha, l r : \text{Tree } \alpha, ih_l : P[l], ih_r : P[r] \vdash P[\text{Tree.node } a \ l \ r]$$

Mirror Operation

列表翻转的树对应物是**镜像操作**：

```
def mirror {α : Type} : Tree α → Tree α
| Tree.nil           => Tree.nil
| Tree.node a l r => Tree.node a (mirror r) (mirror l)
```

镜像可以直接定义，无需借助于某些追加 (append) 操作。因此，关于 mirror 的推理比关于 reverse 的推理更简单，如下所示：

```
theorem mirror_mirror {α : Type} (t : Tree α) :
  mirror (mirror t) = t :=
by
  induction t with
  | nil           => rfl
  | node a l r ih_l ih_r => simp [mirror, ih_l, ih_r]
```

Informal

更详细的**非形式**证明如下：

证明采用对 t 的结构归纳法。

情况 Tree.nil : 我们必须证明 $\text{mirror}(\text{mirror } \text{Tree.nil}) = \text{Tree.nil}$ 。这直接遵循 mirror 的定义。

情况 $\text{Tree.node } a \ l \ r$: 归纳假设为

$$(ih_l) \text{ mirror}(\text{mirror } l) = l \quad (ih_r) \text{ mirror}(\text{mirror } r) = r$$

我们必须证明 $\text{mirror}(\text{mirror}(\text{Tree.node } a \ l \ r)) = \text{Tree.node } a \ l \ r$ 。我们有

$$\begin{aligned} & \text{mirror}(\text{mirror}(\text{Tree.node } a \ l \ r)) \\ &= \text{mirror}(\text{Tree.node } a \ (\text{mirror } r) \ (\text{mirror } l)) \quad (\text{通过 mirror} \\ & \quad \text{的定义}) \\ &= \text{Tree.node } a \ (\text{mirror}(\text{mirror } l)) \ (\text{mirror}(\text{mirror } r)) \quad (\text{同} \\ & \quad \text{上}) \\ &= \text{Tree.node } a \ l \ (\text{mirror}(\text{mirror } r)) \quad (\text{通过 } ih_l) \\ &= \text{Tree.node } a \ l \ r \quad (\text{通过 } ih_r) \end{aligned}$$

□

为了在 Lean 证明中达到同样的详细程度，我们可以使用计算块（第 4.4 节）而不是 `simp`：

```
theorem mirror_mirror_calc {α : Type} :
  ∀t : Tree α, mirror (mirror t) = t
| Tree.nil           => by rfl
| Tree.node a l r =>
  calc
    mirror (mirror (Tree.node a l r))
  = mirror (Tree.node a (mirror r) (mirror l)) :=
    by rfl
  _ = Tree.node a (mirror (mirror l))
    (mirror (mirror r)) :=
    by rfl
  _ = Tree.node a l (mirror (mirror r)) :=
    by rw [mirror_mirror_calc l]
  _ = Tree.node a l r :=
    by rw [mirror_mirror_calc r]
```

我们以以下定理结束本节，该定理在第 6 章中会很有用：

```
theorem mirror_Eq_nil_Iff {α : Type} :
  ∀t : Tree α, mirror t = Tree.nil ↔ t = Tree.nil
| Tree.nil           => by simp [mirror]
| Tree.node _ _ _ => by simp [mirror]
```

5.9 Cases 策略

cases

```
cases 项 [with
| 构造子1 名称列表1 => 策略1
  :
| 构造子n 名称列表n => 策略n]
```

`cases` 策略对指定的项进行分情况讨论。这会产生与该项类型定义中的构造子数量相同的子目标。该策略与 `induction` 类似，不同之处在于它不产生归纳假设，并且会自动排除不可能的情况。可选名称 名称列表₁, ..., 名称列表_n 用于任何出现的变量或假设。

`cases` 形如 `-l- 等于 -r- 的假设`

`cases` 策略也可以用于形如 $l = r$ 的假设 h 。它将 r 与 l 进行匹配，并在目标的各处将所有出现的 r 中的变量替换为 l 中相应的项。如果需要，可以使用 `clear h` 删除剩余的假设 $l = l$ 。

```
cases Classical.em (命题) with
| inl 为真时的名称 => 为真时的策略
| inr 为假时的名称 => 为假时的策略
```

`cases` 策略还可以用于对命题进行分情况讨论。会出现两种情况：一种是命题为真，另一种是命题为假。可选名称 `为真时的名称` 和 `为假时的名称` 分别用于真和假情况下的假设。

5.10 依值归纳类型

归纳类型 `List  $\alpha$` 和 `Tree  $\alpha$` 属于 Lean 的简单类型部分。归纳类型也可以依赖于（非类型的）项。一个典型的例子是长度为 n 的列表，或者称为^{Vectors}向量：

```
inductive Vec ( $\alpha$  : Type) :  $\mathbb{N} \rightarrow$  Type where
| nil : Vec  $\alpha$  0
| cons (a :  $\alpha$ ) {n :  $\mathbb{N}$ } (v : Vec  $\alpha$  n) : Vec  $\alpha$  (n + 1)
```

因此，项 `Vec.cons 3 (Vec.cons 1 Vec.nil)` 的类型为 `Vec  $\mathbb{N}$  2`。通过在类型中编码向量长度，我们可以提供有关函数结果的更精确信息。例如 `Vec.reverse` 函数反转向量，它会将值 `Vec  $\alpha$  n` 映射到具有相同 n 的相同类型的另一个值。而 `Vec.zip` 可以要求其两个参数具有相同的长度。固定长度的向量和矩阵在计算机科学和数学中都很有用。

不幸的是，这种更精确的信息是有代价的。当依值归纳类型所依赖的项是可证明相等但在计算上句法相等（例如 $m + n$ 与 $n + m$ ）时，它们会引起困难。在本指南中，我们通常会避免使用依值归纳类型。本节简要介绍它们是为了完整性。明确地说：这不属于考试内容。

下面的定义引入了列表和向量之间的转换：

```
def listOfVec { $\alpha$  : Type} :  $\forall$ {n :  $\mathbb{N}$ }, Vec  $\alpha$  n  $\rightarrow$  List  $\alpha$ 
| _, Vec.nil      => []
| _, Vec.cons a v => a :: listOfVec v

def vecOfList { $\alpha$  : Type} :
   $\forall$ xs : List  $\alpha$ , Vec  $\alpha$  (List.length xs)
| []      => Vec.nil
| x :: xs => Vec.cons x (vecOfList xs)
```

`listOfVec` 转换接受类型 α 、长度 n 以及 α 上长度为 n 的向量作为参数，并返回 α 上的列表。虽然我们关心长度 n ，但它仍然需要作为一个参数，因为它出现在第三个参数的类型中。我们将前两个参数 α 和 n 设置为隐式的，因为它们可以从第三个参数的类型中推断出来。

`vecOfList` 转换接受类型 α 和 α 上的列表作为参数，并返回与列表长度相同的向量。Lean 的类型检查器足够强大，可以确定两个右侧具有所需的类型。

根据 PAT 原理，证明类似于函数定义。让我们验证将向量转换为列表是否保持其长度：

```
theorem length_listOfVec {α : Type} :
  ∀(n : ℕ) (v : Vec α n), List.length (listOfVec v) =
    n
| _, Vec.nil      => by rfl
| _, Vec.cons a v =>
  by simp [listOfVec, length_listOfVec _ v]
```

为了通过对 v 进行结构归纳来证明目标 $v : \text{Vec } \alpha \ n \vdash P[v]$ ，我们可能会天真地认为展示以下两个子目标就足够了：

$$\vdash P[\text{Vec.nil}]$$

$$m : \mathbb{N}, a : \alpha, u : \text{Vec } \alpha \ m, ih : P[u] \vdash P[\text{Vec.cons } a \ u]$$

这是天真的，因为子目标甚至不是类型正确的： $P[\]$ 中的洞具有类型 $\text{Vec } \alpha \ n$ (其原始^{Inhabitant}居留元 v 的类型)，所以我们不能简单地将类型为 $\text{Vec } \alpha \ 0$ 、 $\text{Vec } \alpha \ m$ 和 $\text{Vec } \alpha \ (m + 1)$ 的 Vec.nil 、 u 或 $\text{Vec.cons } a \ u$ 填入那个洞中。我们必须每次都调整 P ，将 n 替换为 0 、 m 或 $m + 1$ 。使用符号 $P_t[\]$ 表示 $P[\]$ 的变体，其中所有出现的 n 都被项 t 替换，我们得到：

$$\vdash P_0[\text{Vec.nil}]$$

$$m : \mathbb{N}, a : \alpha, u : \text{Vec } \alpha \ m, ih : P_m[u] \vdash P_{m+1}[\text{Vec.cons } a \ u]$$

使用 `cases` 策略的分情况讨论证明与之类似，但不含归纳假设。通常，长度 n 不会是一个变量，而是一个复杂的项。那么将 $P[\]$ 中的 n 替换可能并不具有直观意义。使用 `cases`，相应的子目标会被默默地消除。因此，对类型为 $\text{Vec } \alpha \ 0$ 的值进行分情况讨论将只产生一个子目标，形式为 $\vdash P[\text{Vec.nil}]$ ，因为 0 永远不可能等于形式为 $m + 1$ 的项。

依值类型的模式匹配是微妙的，因为我们要匹配的值的类型可能会根据构造子的不同而改变。给定 $v : \text{Vec } \alpha \ n$ ，我们可能会尝试编写：

```
match v with
| Vec.nil      => ...
| Vec.cons a u => ...
```

但这与我们上面第一次尝试归纳证明时一样天真。由于类型 $\text{Vec } \alpha \ n$ 中的项 n 可能会根据构造子的不同而改变，我们必须对 n 也进行模式匹配：

```
match n, v with
| 0,      Vec.nil      => ...
| m + 1, Vec.cons a u => ...
```

从本质上讲，这等同于

```
match n, v with
| 0,      @Vec.nil α      => ...
| m + 1, @Vec.cons α a m u => ...
```

通常，在第一列放置^{Placeholders}占位符就足够了：

```
match n, v with
| _, Vec.nil      => ...
| _, Vec.cons a u => ...
```

对 n 进行模式匹配却忽略其结果可能看起来有些自相矛盾，但如果没有它，Lean 就无法推断 Vec.cons 的第二个隐式参数。在这方面，`cases` 比 `match` 更用户友好。

5.11 新引入的 Lean 结构小结

声明

<code>class</code>	将结构体类型声明为类型类
<code>instance</code>	将结构体值声明为类型类实例
<code>structure</code>	引入结构体类型及其选择器

表达式

<code>if ... then ... else</code>	^{Decidable} 对可判定命题进行分情况讨论
<code>match ... with</code>	进行模式匹配

策略

<code>cases</code>	执行分情况讨论
--------------------	---------

归纳谓词

Inductive Predicates

归纳谓词，或归纳定义的命题，是指定类型为 $\dots \rightarrow \text{Prop}$ 的函数的便捷方式。它们让人联想到形式系统（第 1.3 节）和 Prolog 风格的逻辑编程。但 Lean 比 Prolog 的表达能力强得多，因此我们需要做一些工作来建立定理，而不仅仅是运行 Prolog 解释器。对 Lean 的一个可能的看法：

Lean = 函数式编程 + 逻辑编程 + 更多逻辑

6.1 入门示例

除非你接触过 Prolog 或逻辑编程，否则你可能会好奇归纳谓词是什么以及它们为什么有用。我们首先回顾三个展示其多种用途的示例：偶数、网球比赛和自反传递闭包。

6.1.1 偶数

数学家经常将特定集合定义为满足某些标准的最小集合。考虑这个定义：

Even Natural Numbers

偶自然数的集合 E 被定义为在以下规则下封闭的最小集合 S ：

- (1) $0 \in S$;
- (2) 对于每个 $k \in \mathbb{N}$ ，如果 $k \in S$ ，那么 $k + 2 \in S$ 。

根据 Knaster–Tarski 定理，这样的集合是存在的。

（最后一句话通常被省略。）我们很容易相信 E 包含了所有的偶数，且仅包含这些数。让我们带上数学家的帽子，证明 4 是偶数：

根据规则 (1)，我们有 $0 \in E$ 。

因此，根据规则 (2) (取 $k := 0$)，我们有 $2 \in E$ 。

由此，根据规则 (2) (取 $k := 2$)，我们有 $4 \in E$ ，证毕。 $\square$

相比之下，计算机科学家可能会使用一个由两个^{Derivation Rules}**推导规则**组成的形式系统来指定相同的集合：

$$\frac{}{0 \in E} \text{ZERO} \qquad \frac{k \in E}{k + 2 \in E} \text{ADDTWO}_k$$

那么证明就是一个^{Derivation Tree}**推导树**：

$$\frac{\frac{\frac{}{0 \in E} \text{ZERO}}{2 \in E} \text{ADDTWO}_0}{4 \in E} \text{ADDTWO}_2$$

如果我们向下阅读，证明是正向的；如果我们向上阅读，证明是逆向的。

归纳谓词是逻辑学家实现相同结果的方式。在 Lean 中，我们不会定义一个集合，而是归纳地定义一个^{Characteristic Predicate}**特征谓词**：

```
inductive Even : ℕ → Prop where
| zero      : Even 0
| add_two   : ∀k : ℕ, Even k → Even (k + 2)
```

这看起来应该很熟悉。除了使用 Prop 代替 Type 之外，我们使用了相同的语法来定义归纳类型。根据 PAT 原理，在 Lean 中，归纳类型和归纳谓词是由同一种机制提供的。

上面的命令定义了一个一元谓词 Even 以及两个^{Introduction Rules}**引入规则**Even.zero 和 Even.add_two，它们可用于证明形如 $\vdash \text{Even } \dots$ 的目标。回想一下，一个符号（例如 Even）的引入规则是一个其结论包含该符号的定理（第 4.3 节）。根据 PAT 原理，Even n 可以被视为像 $\text{Vec } \alpha \ n$ （第 5.10 节）那样的依值归纳类型，而 Even.zero 和 Even.add_two 则是像 Vec.nil 和 Vec.cons 那样的构造子。

如果我们将 Lean 定义翻译回英语，我们会得到类似于上面的 Knaster-Tarski 风格的定义：

以下条款定义了偶数。

- (1) 0 是偶数；
- (2) 假设 k 是偶数，任何形如 $k + 2$ 的数都是偶数。

任何其他数字都不是偶数。

值得注意的是，归纳定义的符号没有定义。因此，我们不能使用 `simp` 展开 (`unfold`) 它们的定义。唯一可用的推理原则是引入和消去 (`elimination`)。

作为一个热身练习，这里是 `Even 4` 的证明：

```
theorem Even_4 :
  Even 4 :=
  have Even_0 : Even 0 :=
    Even.zero
  have Even_2 : Even 2 :=
    Even.add_two _ Even_0
  show Even 4 from
    Even.add_two _ Even_2
```

例如，证明项 `Even.add_two _ Even_0` 的类型为 `Even (0 + 2)`，它在计算意义下句法等价于 `Even 2`，因此在类型检查器看来是相等的。下划线代表 `0`。

得益于归纳定义的「无杂质 (no junk)」保证，`Even.zero` 和 `Even.add_two` 是构造 $\vdash \text{Even } \dots$ 证明的仅有的两种方式。通过检查结论 `Even 0` 和 `Even (k + 2)`，我们发现永远不存在证明 `1` 是偶数的风险。

看待上述归纳定义的另一种方式如下：第一行引入了一个谓词，而第二行和第三行引入了我们希望该谓词满足的公理。相应地，我们可以写成：

```
opaque Even :  $\mathbb{N} \rightarrow \text{Prop}$ 
axiom Even.zero : Even 0
axiom Even.add_two :  $\forall k : \mathbb{N}, \text{Even } k \rightarrow \text{Even } (k + 2)$ 
```

将 `inductive` 替换为 `opaque`，将 `|` 替换为 `axiom`。

但这个公理性版本没有告诉我们 `Even` 什么时候为假。我们无法用它来证明 $\neg \text{Even } 1$ 或 $\neg \text{Even } 17$ 。据我们所知，`Even` 对所有自然数都可能为真。相比之下，归纳定义保证我们获得满足引入规则 `Even.zero` 和 `Even.add_two` 的最小谓词（即「最假」的谓词），并提供消去和归纳原则，允许我们证明 $\neg \text{Even } 1$ 、 $\neg \text{Even } 17$ 或 $\neg \text{Even } (2 * n + 1)$ 。

既然我们可以定义递归函数，为什么还要麻烦地使用归纳谓词呢？事实上，以下定义是完全合法的：

```
def evenRec :  $\mathbb{N} \rightarrow \text{Bool}$ 
| 0      => true
| 1      => false
| k + 2 => evenRec k
```

每种风格都有其优缺点。递归版本迫使我们指定一个假的情况（第二个等式），并且迫使我们担心停机问题。另一方面，因为它具有等式性和计算性，它能很好地与 `rfl`、`simp`、`#eval` 和 `#reduce` 配合使用。归纳版本可以说更加抽象和优雅。每个引入规则都是独立声明的。我们可以添加或删除规则，而无需考虑停机或可执行性。

定义 `Even` 的另一种方式是使用取模运算符（`%`）进行非递归定义：

```
def evenNonrec (k : ℕ) : Prop :=
  k % 2 = 0
```

数学家可能更喜欢这个版本。但归纳版本是一个方便的「Hello, World!」示例，类似于许多现实的归纳定义。它可能只是个玩具，但它是个有用的玩具。

6.1.2 网球比赛

Transition Systems

迁移系统由连接「前」状态和「后」状态的迁移规则组成。作为迁移系统的范例，我们考虑网球比赛中从 `o-o`（双方零分，「Love all」）开始的可能迁移。网球比赛也是一个玩具示例，但在第 ?? 章中，我们将以类似的风格将命令式编程语言的语义定义为迁移系统。

以下摘录自国际网球联合会的《网球规则》，介绍了网球的计分规则。

标准局的计分方式如下，发球方的得分先报：

```
零分    - 「Love」
第一分  - 「15」
第二分  - 「30」
第三分  - 「40」
第四分  - 「Game」（局）
```

但如果每位球员/每对球员都赢得了三分，比分就是「Deuce」（平分）。「Deuce」之后，赢得下一分的球员/球员对获得「Advantage」（占先）。如果该球员/球员对也赢得了下一分，则该球员/球员对赢得该「Game」（局）；如果对方球员/球员对赢得了下一分，比分再次变为「Deuce」。球员/球员对需要在「Deuce」之后立即连赢两分才能赢得该「Game」。

我们首先定义一个归纳类型来表示比分：

```
inductive Score : Type where
  | vs      : ℕ → ℕ → Score
  | advServ : Score
  | advRecv : Score
  | gameServ : Score
  | gameRecv : Score
```

诸如 30-15 之类的比分表示为 `Score.vs 30 15`，我们也将使用中缀记法将其记作 `30 - 15`。我们忽略了计分规则中一些最繁琐的方面，将「Love」记作 0，将「Deuce」记作 40 - 40。如果愿意，我们可以引入诸如 `def love : N := Score.vs 0 0` 之类的别名。

下一阶段是引入一个二元谓词 `Step`，它确定迁移是否可能：

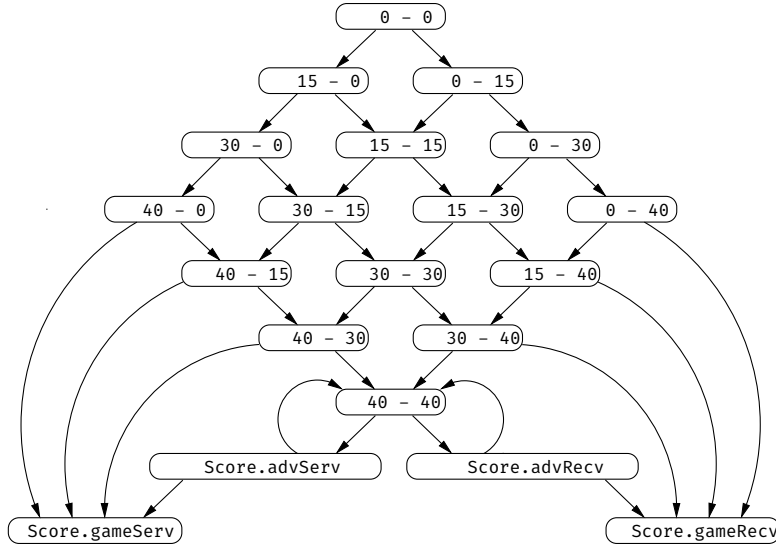
```
inductive Step : Score → Score → Prop where
  | serv_0_15      : ∀n, Step (0 - n) (15 - n)
  | serv_15_30     : ∀n, Step (15 - n) (30 - n)
  | serv_30_40     : ∀n, Step (30 - n) (40 - n)
  | serv_40_game   : ∀n, n < 40 → Step (40 - n)
    Score.gameServ
  | serv_40_adv    : Step (40 - 40) Score.advServ
  | serv_adv_40    : Step Score.advServ (40 - 40)
  | serv_adv_game  : Step Score.advServ Score.gameServ
  | recv_0_15      : ∀n, Step (n - 0) (n - 15)
  | recv_15_30     : ∀n, Step (n - 15) (n - 30)
  | recv_30_40     : ∀n, Step (n - 30) (n - 40)
  | recv_40_game   : ∀n, n < 40 → Step (n - 40)
    Score.gameRecv
  | recv_40_adv    : Step (40 - 40) Score.advRecv
  | recv_adv_40    : Step Score.advRecv (40 - 40)
  | recv_adv_game  : Step Score.advRecv Score.gameRecv
```

令 $s \rightsquigarrow t$ 作为 `Step s t` 的简写。一场比赛是一条链 $s_0 \rightsquigarrow s_1 \rightsquigarrow s_2 \rightsquigarrow \dots \rightsquigarrow s_n$ 其中 $s_0 = 0 - 0$ ，且从 s_n 无法进行任何迁移。该谓词允许诸如 $15 - 99 \rightsquigarrow 30 - 99$ 之类的无意义迁移，但由于比分 15 - 99 无法从「双方零分」(0-0) 达到，因此这些迁移是无害的。

有了形式化定义，我们就可以提出并正式回答以下问题：比赛是否有最大长度？有多少种不同的最终比分？从「双方零分」(Love all) 开始是否能达到 15-99？比分是否可能回到 0-0？让我们使用 Lean 来反驳最后一项断言：

```
theorem no_Step_to_0_0 (s : Score) :
  ¬ s ~ 0 - 0 :=
by
  intro h
  cases h
```

下图总结了哪些比分可以从哪些比分到达：



6.1.3 自反传递闭包

归纳谓词一个特别有用的应用是^{Binary Relation}二元关系 r 的^{Reflexive Transitive Closure}自反传递闭包 r^* 。^{Informally}非形式地, r^* 是表示 r 的零次或多次步骤的关系。例如, 如果 r 对应于在迁移系统中 (如自动机) 执行一次迁移, 那么 r^* 则对应于执行任意步数 (包括零步)。作为一个更具体的例子, 请考虑以下情况:

$$r = \{(1, 2), (2, 4), (4, 8)\}$$

$$r^* = \{(n, n) \mid n \in \mathbb{N}\} \cup \{(1, 2), (1, 4), (1, 8), (2, 4), (2, 8), (4, 8)\}$$

星号 (*) 算子通常被严格地定义为一个形式系统:

$$\frac{(a, b) \in r}{(a, b) \in r^*} \text{BASE} \quad \frac{}{(a, a) \in r^*} \text{REFL} \quad \frac{(a, b) \in r^* \quad (b, c) \in r^*}{(a, c) \in r^*} \text{TRANS}$$

这些规则将 r^* 定义为包含 r (通过 BASE) 且是自反的 (通过 REFL) 和传递的 (通过 TRANS) 的^{Smallest Relation}最小关系。如果我们想定义^{Transitive Closure}传递闭包 r^+ , 只需省略 REFL 规则即可。如果我们想要^{Reflexive Symmetric Closure}自反对称闭包, 我们会将 TRANS 规则替换为 SYMM 规则。在形式系统中, 我们只需声明我们希望为真的属性, 而无需考虑停机或可执行性。

将上述推导规则直接翻译为归纳谓词的引入规则也很简单:

```
inductive Star {α : Type} (R : α → α → Prop) : α → α
  → Prop
where
  | base (a b : α)      : R a b → Star R a b
  | refl (a : α)        : Star R a a
  | trans (a b c : α) : Star R a b → Star R b c → Star
    R a c
```

我们将^{Relations}关系表示为^{Binary Predicates}二元谓词而非^{Pair}偶对的集合，将 $(a, b) \in R$ 写作 $R\ a\ b$ 。关系^{Predicates Interchangeable}和谓词是可互换的，但在证明助手中，谓词通常更方便使用。^{Reflexive Transitive Closure} R 的自反传递闭包记作 $\text{Star } R$ 。^{Introduction Rules}请注意， a 、 b 和 c 在冒号左侧被声明为引入规则的参数。我们也可以写成：

```
| base : ∀ a b : α, R a b → Star R a b
```

或者在另一个极端写成：

```
| base (a b : α) (hab : R a b) : Star R a b
```

所有这些形式都在^{Logically and Operationally Equivalent}逻辑和操作上等价。

归纳谓词的一般格式如下：

```
inductive 谓词名称 (参数1 : 类型1) ... (参数k : 类型k) :
  类型k+1 → ... → 类型k+p → Prop where
  | 规则名称1 (参数11 : 类型11) ... (参数1m1 : 类型1m1) :
    命题1
    ⋮
  | 规则名称n (参数n1 : 类型n1) ... (参数nmn : 类型nmn) :
    命题n
```

其中每个命题_j 的结论必须是已定义谓词谓词名称在某些^{Arguments Application}实参上的应用。这些参数可以是任意项；它们不需要是^{Constructor Patterns}构造子模式。如果我们想让对应于参数的实参变成^{Implicit}隐式的，也可以使用花括号 $\{ \}$ 而非圆括号 $()$ 。

上述 Star 的定义确实非常^{Elegant}优雅。如果你仍然对此表示怀疑，请尝试将其实现为^{Recursive Function}递归函数：

```
def starRec {α : Type} (R : α → α → Bool) :
  α → α → Bool :=
```

总结一下，归纳谓词 P 的每个引入规则由以下部分组成（从左到右）：

- 一个^{Name}名称；
- 可能出现在规则中的^{Variables}变量；
- 必须满足的零个或多个^{Conditions}条件，这些条件可能会^{Recursively}递归地应用 P ；
- 将 P 应用于某些参数，形成一个^{Pattern}模式。

因此，对于规则 Star.base ，模式是 $\text{Star } R\ a\ b$ ，条件是 $R\ a\ b$ ，变量是 a 和 b 。

6.1.4 反例

并非所有的归纳定义都允许一个^{Least Solution}最小解。最简单的反例是：

```
-- fails
inductive Illegal : Prop where
  | intro : ¬ Illegal → Illegal
```

如果 Lean 接受了这个定义,我们就可以用它来证明^{Equivalence}等价性 $\text{Illegal} \leftrightarrow \neg \text{Illegal}$, 从而很容易推导出 False。幸运的是, Lean 拒绝了这个定义:

'Illegal.intro' 的第 1 个参数具有正在声明的数据类型的非正出现 (non positive occurrence)

它所抱怨的^{Nonpositive Occurrence}非正出现是指在^{Negation}否定下的 Illegal 出现。数学家会根据^{Knaster-Tarski Theorem}Knaster-Tarski 定理的^{Monotonicity Condition}单调性条件不满足而拒绝该定义。

6.2 逻辑符号

虽然 Even 是本指南中第一个公开的归纳谓词,但前面的章节已经秘密地介绍了其他归纳谓词。其中第一个是^{Equality}相等(=),在第 2 章中引入,随后是^{Logical Symbols}逻辑符号 $\wedge$ 、 $\vee$ 、 $\leftrightarrow$ 、 $\exists$ 、True 和 False。它们的定义值得仔细研究:

```
inductive And (a b : Prop) : Prop where
  | intro : a → b → And a b
```

```
inductive Or (a b : Prop) : Prop where
  | inl : a → Or a b
  | inr : b → Or a b
```

```
inductive Iff (a b : Prop) : Prop where
  | intro : (a → b) → (b → a) → Iff a b
```

```
inductive Exists {α : Type} (P : α → Prop) : Prop where
  | intro : ∀ a : α, P a → Exists P
```

```
inductive True : Prop where
  | intro : True
```

```
inductive False : Prop where
```

```
inductive Eq {α : Type} : α → α → Prop where
  | refl : ∀ a : α, Eq a a
```

(严格来说，在 Lean 中，上述一些定义实际上是^{Structures}结构而非归纳谓词，但区别只是表面的。正如我们在第 5.5 节中看到的，结构本质上是具有^{Syntactic Sugar}一些语法糖的单构造子归纳谓词。)

传统记法 $\exists x : \alpha, P$ 和 $x = y$ 是 $\text{Exists}(\text{fun } x : \alpha \mapsto P)$ 和 $\text{Eq } x \ y$ 的语法糖。请注意 fun 是如何充当全能^{Binder}绑定子的。另请注意， False 没有构造子。不存在 False 的证明，就像不存在 $\text{Even } 1$ 的证明一样。对于归纳谓词，我们只陈述我们希望为真的规则。

符号 $\forall$ (包括其特例 $\rightarrow$) 直接内置于逻辑中，并未定义为归纳谓词。它也没有显式的引入或消去规则。^{Introduction Principle}引入原理是^{Anonymous Function}匿名函数 $\text{fun } x \mapsto _$ ，而^{Elimination Principle}消去原理是函数应用 $_ u$ 。

对于任何归纳谓词，只需指定引入规则。第 3.3 节和第 3.4 节中介绍的消去规则必须手动推导。

6.3 规则归纳

正如我们可以在归纳类型的项上进行归纳一样，我们也可以在归纳谓词的证明上进行归纳。例如，给定目标 $h : \text{Even } n \vdash P \ n$ ，我们可以调用 $\text{induction } h$ 并得到两个子目标，分别对应 Even.zero 和 Even.add_two 。这被称为「在 h 的推导结构上归纳」或简称为「规则归纳」，因为归纳是在谓词的引入规则（即证明项的构造子）上进行的。

看待规则归纳有两种方式：「满足.....的最小谓词」视角和「PAT」视角。要理解「满足.....的最小谓词」视角，请回想一下，归纳定义将一个符号引入为满足引入规则的最小（即最假）谓词。相应地， Even 是满足性质 $Q \ 0$ 和 $\forall k, Q \ k \rightarrow Q \ (k + 2)$ 的最小谓词 Q 。因此，如果我们能证明对于某个谓词 P ，性质 $P \ 0$ 和 $\forall k, P \ k \rightarrow P \ (k + 2)$ 成立，那么 P 要么是 Even 本身，要么比 Even 更大（即更真）。结果是， $\text{Even } n$ 蕴含 $P \ n$ ，这正是我们证明目标 $h : \text{Even } n \vdash P \ n$ 所需要的。

^{Informal Arguments}「满足.....的最小谓词」视角为规则归纳提供了一个很好的直观解释，可用于非形式论证中，例如以下关于对于所有 n ， $\text{Even } n$ 蕴含 $n \% 2 = 0$ 的证明：

证明对假设 $\text{Even } n$ 使用规则归纳。

****情况**** Even.zero ：我们必须证明 $0 \% 2 = 0$ 。这可以通过计算得出。

****情况**** $\text{Even.add_two } k$ ：归纳假设是 $k \% 2 = 0$ 。我们必须证明 $(k + 2) \% 2 = 0$ 。这可以通过基本算术推理得出。 $\square$

Lean 证明具有相同的结构：

```

theorem mod_two_Eq_zero_of_Even (n : ℕ) (h : Even n) :
  n % 2 = 0 :=
by
  induction h with
  | zero          => rfl
  | add_two k hk ih => simp [ih]

```

PAT Principle

PAT 原理为我们提供了另一种看待规则归纳的有效方式。其核心思想是，在形如 $h : \text{Even } n \vdash P[h]$ 的目标中对 h 进行规则归纳，完全类似于在^{Dependent Inductive Type}**依值归纳类型**（例如 $\text{Vec } \alpha \ n$ ，见第 5.10 节）的值上进行结构归纳。将 $P[\]$ 中 n 被替换为项 u 的变体记作 $P_u[\]$ ，我们得到以下子目标：

$$\vdash P_0[\text{Even.zero} : \text{Even } 0]$$

$$k : \mathbb{N}, hk : \text{Even } k, ih : P_k[hk] \vdash P_{k+2}[\text{Even.add_two } k \ hk : \text{Even } (k + 2)]$$

这些实际上就是 `induction` 策略产生的子目标。

无论归纳谓词 Q 为何，计算子目标的步骤总是相同的：

1. 在 $P[h]$ 中，将 h 替换为应用在一些新变量上的每一个可能的引入规则（例如 $\text{Even.add_two } k \ hk$ ），并在 $P[\]$ 中对 n 进行实例化以使 $P[\]$ 类型正确。这会产生与引入规则数量相等的子目标。
2. 将这些新变量（例如 k 、 hk ）添加到本地^{Context}**语境**中。
3. 为所有断言 $Q \dots$ 的新假设添加归纳假设。

注意上述假设中出现了 $hk : \text{Even } k$ 。在「满足……的最小谓词」视角中，它是缺失的，而且它并不是必不可少的，因为总是可以通过将 P 强化为 $\text{Even } n \wedge \dots$ 的形式来恢复它。

在几乎所有的实际情况中， h 不会出现在 $P[h]$ 中。我们可以简单地写作：

$$\vdash P_0 \qquad k : \mathbb{N}, hk : \text{Even } k, ih : P_k \vdash P_{k+2}$$

在极少数情况下， h 会出现在 $P[h]$ 中。正如我们在第 5.7 节中尝试提取列表头部时看到的那样，证明可能作为子项出现在任意项中。

接下来，我们考虑^{Reflexive Transitive Closure}**自反传递闭包** $\text{Star } R$ 。给定目标 $h : \text{Star } R \ x \ y \vdash P$ ，对 h 进行规则归纳会产生以下子目标，其中 $P_{t,u}$ 表示将 P 中的 x 和 y 分别替换为 t 和 u 的变体：

$$\begin{aligned}
 & a \ b : \alpha, hab : R \ a \ b \vdash P_{a,b} \\
 & a : \alpha \vdash P_{a,a} \\
 & a \ b \ c : \alpha, hab : \text{Star } R \ a \ b, hbc : \text{Star } R \ b \ c, ihab : P_{a,b}, ihbc : P_{b,c} \vdash P_{a,c}
 \end{aligned}$$

这就是「证明助手」中「助手」这一方面发挥作用的地方。Star 的关键性质之一是^{Idempotence}**幂等性**——对 Star R 应用 Star 不会有任何效果。这可以在 Lean 中如下证明，在等价性的 $\rightarrow$ 方向上使用规则归纳：

```
theorem Star_Star_Iff_Star {α : Type} (R : α → α →
  Prop)
  (a b : α) :
  Star (Star R) a b ↔ Star R a b :=
by
  apply Iff.intro
  • intro h
    induction h with
    | base a b hab          => exact hab
    | refl a                => apply Star.refl
    | trans a b c hab hbc ihab ihbc =>
      apply Star.trans a b
      • exact ihab
      • exact ihbc
  • intro h
    apply Star.base
    exact h
```

我们小心地为新出现的变量赋予直观的名称。在包含长的、自动生成的名称的目标中很容易迷失。第 3.8 节中介绍的清理策略在面对大型目标时也很有帮助。

我们可以用相等性而不是等价性更常规地陈述幂等性质：

```
@[simp] theorem Star_Star_Eq_Star {α : Type}
  (R : α → α → Prop) :
  Star (Star R) = Star R :=
by
  apply funext
  intro a
  apply funext
  intro b
  apply propext
  apply Star_Star_Iff_Star
```

由于 Lean 的逻辑是^{Classical}**经典逻辑**，证明中使用了以下两个定理：

```
funext : (∀x, ?f x = ?g x) → ?f = ?g
propext : (?a ↔ ?b) → ?a = ?b
```

Functional Extensionality
函数外延性(funext) 指出, 如果两个函数对所有输入都产生相等的结果, 那么这两个函数必须相等。Propositional Extensionality
命题外延性(propext) 指出, 命题的等价性与相等性是一致的。在这些短语中, 外延性意味着类似于「所见即所得」的意思。虽然这些性质看起来显而易见, 但有些证明助手是建立在较弱的、Intuitionistic Logics
直觉主义逻辑之上的, 在这些逻辑中, 这些性质通常不成立。

我们将定理 `Star_Star_Eq_Star` 注册为 `simp` 规则, 因为从左到右的重写规则来看, 它确实用一个更简单的项替换了一个复杂的项。很难想象会有我们不希望 `simp` 将 `Star (Star ...)` 重写为 `Star ...` 的情况。

对于规则归纳, 我们使用 `induction` 策略。Pattern Matching Recursion 由于一些微妙的逻辑原因 (这在第 ?? 章中会变得更加清晰), 不允许通过**模式匹配**和**递归**进行规则归纳。

在第 5.4 节中, 我们看到了一张并排描述 `Bool` 和 `Prop` 解释的图表。该图表暗示存在无限数量的命题, 但我们现在知道, 恰好有两个命题: `False` 和 `True`。以下是修订后的图表:



我们看到, 只有两个命题, 一个没有证明, 另一个有若干证明。图中显示了三个证明。我们将在第 ?? 章中再次完善这幅图。

6.4 线性算术策略

linarith

Linear Arithmetic
`linarith` 策略可用于证明涉及**线性算术**等式 ($=$)、不等式 ($<$ 、 $>$ 、 $\leq$ 和 $\geq$) 以及非等式 ($\neq$) 的目标。「线性」意味着不出现乘法和除法, 或者如果出现, 则其中一个操作数必须是数字常量。例如, $2 * x < y$ 是一个线性约束 (可以改写为 $x + x < y$), 而 $x * y < y$ 是非线性的。

6.5 消去

给定一个归纳谓词 Q , 其引入规则通常具有 $\forall \dots, \dots \rightarrow Q \dots$ 的形式, 且可用于证明 $\vdash Q \dots$ 形式的目标。消去则以相反的方式工作: 它从定理或假设 $h : Q \dots$ 中提取信息。消去有多种形式: `cases` 和 `induction` 策略、模式匹配以及消去规则 (例如 `And.left`)。

在 $h : Q \dots$ 上调用的 `cases h` 策略执行的规则归纳大致与 `induction h` 相同, 但不产生任何归纳假设。我们在第 5 章中遇到了两个惯用法, 现在终于可以分析它们了。

第一个惯用法是当 h 的形式为 $l = r$ 时——即 $\text{Eq } l \ r$ (第 6.2 节)。假设目标是 $h : l = r \vdash P[h]$ 。第 6.3 节中介绍的步骤会产生子目标

$$a : \alpha \vdash P_{a,a}[\text{Eq.refl } a : a = a]$$

其中 $P_{t,u}[\]$ 代表 $P[\]$ 的变体, 其中 l 和 r 分别被 t 和 u 替换。(严格来说, 无用的假设 $h : a = a$ 也会出现在子目标中。)在实践中, $P[h]$ 可能并不依赖于 h 。此外, `cases` 会重用名称 l , 而不是使用像 a 这样不同的名称。因此, 我们会得到

$$l : \alpha \vdash P_{l,l}$$

换句话说, 原始目标中所有出现的 r 都被替换为了 l 。这对应于我们在第 5.7 节中观察到的行为。

第二个惯用法是策略 `cases Classical.em Q`, 其中 Q 是一个命题。`Classical.em Q` 部分是 $Q \vee \neg Q$ (即 $\text{Or } Q (\neg Q)$, 见第 6.2 节) 的一个证明项。然后应用 `cases` 来消去 $\vee$ 联结词。假设目标是 $\vdash P[\text{Classical.em } Q]$ 。根据 `Or` 谓词的定义, 新的子目标是

$$hQ : Q \vdash P[\text{Or.inl } hQ : Q \vee \neg Q]$$

$$hnQ : \neg Q \vdash P[\text{Or.inr } hnQ : Q \vee \neg Q]$$

不需要修改 P , 因为 `Or.inl hQ` 和 `Or.inr hnQ` 具有与 `Classical.em Q` 相同的类型——即 $Q \vee \neg Q$ 。在实践中, 目标通常不包含 `Classical.em Q`, 而仅仅是 $\vdash P$, 这样我们就得到子目标

$$hQ : Q \vdash P$$

$$hnQ : \neg Q \vdash P$$

同样, 这也是我们在第 5.7 节中观察到的行为。

在结构化证明中, 我们可以使用 `match` 表达式 (第 5.4 节) 来实现与 `cases` 相同的效果。这对于逻辑符号非常有效。然而, 对于像 `Even` 和 `Star` 这样参数会随归纳而变化的谓词, 我们最终会遇到**依值类型模式匹配**^{Dependently Typed Pattern Matching}, 这是很微妙的 (见第 5.10 节)。通常, 让 `cases` 来确定子目标的样式, 比我们自己进行模式匹配要容易得多。我们将在下面回顾这两种风格的示例。

展开形如 $Q(c \dots)$ 的假设通常很有用, 其中 c 是一个**构造子**^{Constructor}或某些其他常量。我们可以陈述并证明一个**反转规则**^{Inversion Rule}来支持这种消去推理。典型的反转规则具有以下形式:

$$\forall x_1 \dots x_n, Q(c \ x_1 \dots x_n) \rightarrow (\exists \dots, \dots \wedge \dots) \vee \dots \vee (\exists \dots, \dots \wedge \dots)$$

将引入和消去合并为一个定理也很有用, 这可以用于重写假设和子目标的目的。除了中间的联结词 $\leftrightarrow$ 之外, 格式是相同的:

$$\forall x_1 \dots x_n, Q(c \ x_1 \dots x_n) \leftrightarrow (\exists \dots, \dots \wedge \dots) \vee \dots \vee (\exists \dots, \dots \wedge \dots)$$

`Even` 的反转规则如下所示:

```

theorem Even_Iff (n : ℕ) :
  Even n ↔ n = 0 ∨ (∃m : ℕ, n = m + 2 ∧ Even m) :=
by
  apply Iff.intro
  • intro hn
    cases hn with
    | zero      => simp
    | add_two k hk =>
      apply Or.inr
      apply Exists.intro k
      simp [hk]
  • intro hor
    cases hor with
    | inl heq => simp [heq, Even.zero]
    | inr hex =>
      cases hex with
      | intro k hand =>
        cases hand with
        | intro heq hk =>
          simp [heq, Even.add_two _ hk]

```

照例，策略式证明并不是特别易读，但我们可以看到引入规则和消去式的 cases 策略在逻辑符号和 Even 谓词中都起着重要作用。simp 策略则进行了最后的收尾工作。

如果你更喜欢结构化证明，这里有一个对 $hn : \text{Even } n$ 使用 ^{Dependently Typed Pattern Matching} 依值类型模式匹配的证明版本：

```

theorem Even_Iff_struct (n : ℕ) :
  Even n ↔ n = 0 ∨ (∃m : ℕ, n = m + 2 ∧ Even m) :=
Iff.intro
  (assume hn : Even n
   match hn with
   | Even.zero      =>
     show 0 = 0 ∨ _ from
       by simp
   | Even.add_two k hk =>
     show _ ∨ (∃m, k + 2 = m + 2 ∧ Even m) from
       Or.inr (Exists.intro k (by simp [*])))
  (assume hor : n = 0 ∨ (∃m, n = m + 2 ∧ Even m)

```

```

match hor with
| Or.inl heq =>
  show Even n from
    by simp [heq, Even.zero]
| Or.inr hex =>
  match hex with
  | Exists.intro m hand =>
    match hand with
    | And.intro heq hm =>
      show Even n from
        by simp [heq, Even.add_two _ hm])

```

6.6 更多示例

有了对归纳谓词更好的理解，我们现在准备依次回顾四个进一步的应用。

6.6.1 有序列表

我们的第一个例子是一个检查自然数列表是否按升序排列的谓词：

```

inductive Sorted : List N → Prop where
| nil : Sorted []
| single (x : N) : Sorted [x]
| two_or_more (x y : N) {zs : List N} (hle : x ≤ y)
  (hsorted : Sorted (y :: zs)) :
  Sorted (x :: y :: zs)

```

该定义捕捉了以下数学直觉：

有序列表的集合被定义为满足下列规则的最小闭集：

- (1) 列表 [] 是有序的；
- (2) 给定一个数字 x ，列表 $[x]$ 是有序的；
- (3) 给定两个数字 x 、 y 和一个列表 zs ，如果 $x \leq y$ 且 $y :: zs$ 是有序的，那么 $x :: y :: zs$ 是有序的。

通过尝试一些小例子来测试我们的定义总是一个好主意。列表 $[3, 5]$ 是有序的吗？看起来是的：

```

theorem Sorted_3_5 :
  Sorted [3, 5] :=

```

```

by
  apply Sorted.two_or_more
    • simp
    • exact Sorted.single _

```

该示例使用了 `sorted` 的两个引入规则。下面是一个更紧凑的证明，使用了证明项：

```

theorem Sorted_3_5_raw :
  Sorted [3, 5] :=
  Sorted.two_or_more _ _ (by simp) (Sorted.single _)

```

同样的想法可以用来证明 `[7, 9, 9, 11]` 是有序的：

```

theorem sorted_7_9_9_11 :
  Sorted [7, 9, 9, 11] :=
  Sorted.two_or_more _ _ (by simp)
    (Sorted.two_or_more _ _ (by simp)
      (Sorted.two_or_more _ _ (by simp)
        (Sorted.single _)))

```

相反，我们可以证明某些列表不是有序的。为此，我们需要消去：

```

theorem Not_Sorted_17_13 :
  ¬ Sorted [17, 13] :=
  by
    intro h
    cases h with
    | two_or_more _ _ hlet hsorted => simp at hlet

```

从列表 `[17, 13]` 是有序的这一假设中，我们提取出不等式 $17 \leq 13$ 。cases 策略静默地排除了 `nil` 和 `single` 情况，因为它们无法与双元素列表匹配。然后我们调用 `simp` 来利用不可能成立的假设 $17 \leq 13$ 。

6.6.2 回文

回文是从左向右读和从右向左读都相同的列表。例如，`[a, b, b, a]` 和 `[a, h, a]` 都是回文。当且仅当作为参数传递的列表是回文时，以下归纳谓词为 `True`：

```

inductive Palindrome {α : Type} : List α → Prop where
  | nil : Palindrome []
  | single (x : α) : Palindrome [x]
  | sandwich (x : α) (xs : List α) (hxs : Palindrome xs)
  :

```

```
Palindrome ([x] ++ xs ++ [x])
```

该定义区分了三种情况：(1) `[]` 是回文；(2) 对于任何元素 x ，^{Singleton List} **单元素列表** `[x]` 是回文；(3) 对于任何元素 x 和任何回文 $[y_1, \dots, y_n]$ ，列表 `[x, y_1, \dots, y_n, x]` 是回文。

回文是归纳谓词大显身手的另一个例子。以下^{Naive Recursive Definition} **朴素递归定义**无法工作，因为 `[x] ++ xs ++ [x]` 不是一个^{Constructor Pattern} **构造子模式**，且变量 x 被重复了：

```
-- fails
def palindromeRec {α : Type} : List α → Bool
| []           => true
| [_]          => true
| ([x] ++ xs ++ [x]) => palindromeRec xs
| _            => false
```

正确的递归定义是可能的，但超出了本指南的范围。

自然地，回文的反转也是回文。这是一个很好的练习：

```
theorem Palindrome_reverse {α : Type} (xs : List α)
  (hxs : Palindrome xs) :
  Palindrome (reverse xs) :=
by
  induction hxs with
  | nil           => exact Palindrome.nil
  | single x      => exact Palindrome.single x
  | sandwich x xs hxs ih =>
    • simp [reverse, reverse_append]
      exact Palindrome.sandwich _ _ ih
```

非形式地：

证明通过对假设 hxs 进行^{Rule Induction} **规则归纳**。

CASE `Palindrome.nil`: 我们必须证明 `Palindrome (reverse [])`。利用 `reverse [] = []`，这由 `Palindrome.nil` 得出。

CASE `Palindrome.single x`: 我们必须证明 `Palindrome (reverse [x])`。利用 `reverse [x] = [x]`，这由 `Palindrome.single x` 得出。

CASE `Palindrome.sandwich x xs hxs`: 在假设 (hxs) `Palindrome xs` 下，我们必须证明 `Palindrome (reverse ([x] ++ xs ++ [x]))`。归纳假设是 `Palindrome (reverse xs)`。通过简化，只需证明 `Palindrome ([x] ++ reverse xs ++ [x])`。根据

Palindrome.sandwich, 只需证明 $\text{Palindrome}(\text{reverse xs})$, 这正是归纳假设。 $\square$

6.6.3 满二叉树

我们的第三个例子基于第 5.8 节中介绍的二叉树类型。如果二叉树的所有节点都有零个或两个子节点, 则称该二叉树是^{Full}满的。这可以编码为一个归纳谓词:

```
inductive IsFull {α : Type} : Tree α → Prop where
| nil : IsFull Tree.nil
| node (a : α) (l r : Tree α)
  (hl : IsFull l) (hr : IsFull r)
  (hiff : l = Tree.nil ↔ r = Tree.nil) :
  IsFull (Tree.node a l r)
```

第一种情况指出空树是满树。第二种情况指出, 如果一个非空树有两个本身也是满的子树, 且这两个子树要么都是空的, 要么都是非空的, 那么该树就是满树。这两种情况整齐地遵循了归纳类型的结构, 因此重用名称 nil 和 node 是很自然的。

由以空树为子节点的节点组成的树是满树。这是一个简单的证明:

```
theorem IsFull_singleton {α : Type} (a : α) :
  IsFull (Tree.node a Tree.nil Tree.nil) :=
by
  apply IsFull.node
  · exact IsFull.nil
  · exact IsFull.nil
  · rfl
```

满树的一个更有趣的性质是, 满树性在^{Mirror Operation}镜像操作下是保持不变的。我们的第一个证明是通过对 $ht : \text{IsFull } t$ 进行^{Rule Induction}规则归纳:

```
theorem IsFull_mirror {α : Type} (t : Tree α)
  (ht : IsFull t) :
  IsFull (mirror t) :=
by
  induction ht with
  | nil                                     => exact
    IsFull.nil
  | node a l r hl hr hiff ih_l ih_r =>
    · rw [mirror]
      apply IsFull.node
```

- `exact ih_r`
- `exact ih_l`
- `simp [mirror_Eq_nil_Iff, *]`

由于 `IsFull` 的定义遵循了 `Tree` 的定义，因此对树 `t` 进行^{Structural Induction}**结构归纳**也是合理的：

```
theorem IsFull_mirror_struct_induct {α : Type} (t : Tree
  α) :
  IsFull t → IsFull (mirror t) :=
by
  induction t with
  | nil                                     =>
    • intro ht
      exact ht
  | node a l r ih_l ih_r =>
    • intro ht
      cases ht with
      | node _ _ _ hl hr hiff =>
        • rw [mirror]
          apply IsFull.node
          • exact ih_r hr
          • apply ih_l hl
          • simp [mirror_Eq_nil_Iff, *]
```

这里的关键是对假设 `ht : IsFull (Tree.node a l r)` 进行的^{Case Distinction}**分类讨论**。`cases` ^{Tactic}**策略**注意到 `IsFull.nil` 引入规则不可能被用来推导出 `ht`，因此它只产生一种情况，对应于 `IsFull.node`。照例，如果你在 Visual Studio Code 中通过移动光标来检查策略式证明，它会更有意义。

6.6.4 一阶项

我们的最后一个例子基于^{First-Order Terms}**一阶项**的归纳类型：

```
inductive Term (α β : Type) : Type where
  | var : β → Term α β
  | fn  : α → List (Term α β) → Term α β
```

一阶项要么是一个变量 x ，要么是一个应用于参数列表的函数符号 f ： $f(t_1, \dots, t_n)$ ，其中数学变量 $t_1, \dots, t_n$ 代表参数，它们本身也是项。因此， $\sin(\max(x, y))$ 是一个一阶项。参数 α 和 β 分别是函数符号和变量的类型。

并非所有项都是合法的。例如，项 $\min(\cos(a), \cos(a, b))$ 被认为是^{Ill-formed}不合式的，因为函数 $\cos$ 调用时的参数数量不一致（1 个对比 2 个）。除了 α 和 β ，我们还考虑了^{Arity}元数，由函数 $\text{arity} : \alpha \rightarrow \mathbb{N}$ 表示，指示每个函数符号接受多少个参数。例如，二元符号的元数为 2。

`WellFormed` 谓词接着检查给定的项是否仅包含具有指定数量参数的函数符号应用：

```
inductive WellFormed {α β : Type} (arity : α → ℕ) :
  Term α β → Prop where
  | var (x : β) : WellFormed arity (Term.var x)
  | fn (f : α) (ts : List (Term α β))
    (hargs : ∀t ∈ ts, WellFormed arity t)
    (hlen : length ts = arity f) :
    WellFormed arity (Term.fn f ts)
```

`var` 情况指出变量总是合式的。`fn` 情况检查参数 `ts` 是否递归地合式，并且 `ts` 的长度是否等于所讨论函数符号 `f` 的指定元数。

一阶项的另一个有趣性质是它们是否包含变量。这可以使用归纳谓词轻松检查：

```
inductive VariableFree {α β : Type} : Term α β → Prop
  where
  | fn (f : α) (ts : List (Term α β))
    (hargs : ∀t ∈ ts, VariableFree t) :
    VariableFree (Term.fn f ts)
```

没有对应于 `Term.var` 的引入规则，因为变量永远不是无变量的。

6.7 归纳中的陷阱

对归纳谓词调用 `induction` 时需要小心。归纳谓词的参数通常在归纳过程中演变。这些细节在非正式证明中通常被一带而过，但证明助手要求我们保持精确。

回想一下偶数的定义：

```
inductive Even : ℕ → Prop where
  | zero      : Even 0
  | add_two : ∀k : ℕ, Even k → Even (k + 2)
```

如果目标的格式为 $h : \text{Even } n \vdash P n$ ，对 h 应用 `induction` 将产生以下子目标：

$$\vdash P 0 \qquad k : \mathbb{N}, h k : P k \vdash P (k + 2)$$

这符合预期。

问题出现在 Even 的参数不是变量时。在目标 $\text{hev} : \text{Even} (2 * n + 1) \vdash \text{False}$ 中对假设 hev 应用 induction 会失败并报错：

index in target's type is not a variable (consider using the `cases` tactic instead) (目标类型中的索引不是变量 (考虑改用 `cases` 策略))

为了解决这个问题，我们需要用变量 m 替换 $2 * n + 1$ ，并添加等式 $m = 2 * n + 1$ 作为假设：

$$m : 2 * n + 1, \text{hev} : \text{Even } m \vdash \text{False}$$

这个目标在逻辑上是等价的，但现在 induction 会产生两个子目标：

$$\begin{aligned} & m \ n : \mathbb{N}, \text{hm} : 0 = 2 * n + 1 \vdash \text{False} \\ & m : 2 * n + 1, \text{ih} : m = 2 * n + 1 \rightarrow \text{False}, \text{hm} : m + 2 = 2 * n + 1 \vdash \text{False} \end{aligned}$$

遗憾的是，第二个子目标是不可证明的。问题在于我们想要用 $n - 1$ 来实例化归纳假设 ih 中的 n ，但 n 是不可实例化的。

解决方案是什么？我们需要对 n 进行全称量化，以便能够在归纳假设中实例化它。这可以通过在 induction h 之后指定 generalizing n 来实现。现在我们得到如下子目标：

$$\begin{aligned} & m \ n : \mathbb{N}, \text{hm} : 0 = 2 * n + 1 \vdash \text{False} \\ & m : 2 * n + 1, \text{ih} : \forall n, m = 2 * n + 1 \rightarrow \text{False}, \text{hm} : m + 2 = 2 * n + 1 \vdash \text{False} \end{aligned}$$

现在归纳假设中的变量 n 与目标其余部分中的变量 n 断开了联系，我们可以将归纳假设中的 n 实例化为 $n - 1$ 。

综上所述，我们得到：

```
theorem Not_Even_two_mul_add_one (m n : ℕ)
  (hm : m = 2 * n + 1) :
  ¬ Even m :=
by
  intro h
  induction h generalizing n with
  | zero          => linarith
  | add_two k hk ih =>
    apply ih (n - 1)
    cases n with
    | zero      => simp at *
    | succ n' =>
```

```
simp at *  
linarith
```

我们使用定理 `Nat.succ_eq_add_one` 将 `Nat.succ n` 形式的项重写为 `n + 1`，并使用 `linarith` 策略进行简单的算术推理。当我们面对 `Nat.succ` 和加法的混合时，该定理非常有用。

6.8 新引入的 Lean 结构小结

定理

<code>funext</code>	函数外延性
<code>propext</code>	命题外延性

策略

<code>linarith</code>	调用线性算术过程
-----------------------	----------

带作用的编程

纯函数式编程有时会让人感到过于受限。^{Effectful Functional Programming}带作用的函数式编程提供了一些习语来缓解这些限制，让我们能够以带有副作用、异常、非确定性和其他^{Effects}作用的方式进行编程。

其底层的抽象被称为^{Monad}单子。单子推广了带有作用的程序。它们在 Haskell 中非常流行，用于编写指令式程序。在 Lean 中，它们被用于表达指令式程序并对其进行推理。它们甚至对于 Lean 自身的编程也非常有用，正如我们将在第 8 章中看到的那样。

本笔记受到 ^{Programming in Lean}《Lean 语言编程》[2] 第 7 章的启发。我们也参考了 ^{Real World Haskell}《实战 Haskell》[22] 的第 14 章，以获得关于^{Effectful Functional Programming}带作用的函数式编程的一般性介绍。

7.1 引入示例

考虑以下编程任务：

实现一个函数 `sum257 ns`，对自然数列表 `ns` 的第二个、第五个和第七个元素求和。结果使用 `Option N`，以便在列表元素太少时返回 `Option.none`。

一个直接的解决方案如下：

```
def nth {α : Type} : List α → Nat → Option α
| [], _ => Option.none
| x :: _, 0 => Option.some x
| _ :: xs, n + 1 => nth xs n

def sum257 (ns : List N) : Option N :=
```

```

match nth ns 1 with
| Option.none      => Option.none
| Option.some n2 =>
  match nth ns 4 with
  | Option.none      => Option.none
  | Option.some n5 =>
    match nth ns 6 with
    | Option.none      => Option.none
    | Option.some n7 => Option.some (n2 + n5 + n7)

```

(令人困惑的是, `nth` 从 0 开始计数元素。) 这段代码相当不够优雅, 因为其中充斥着对 `Option` 的^{Pattern Matching}模式匹配。虽然这个编程任务是人为构造的, 但我们都能回想起编写带有嵌套错误处理和不断增加的缩进层级的代码时的经历。

我们可以做得更好, 方法是将这些丑陋之处集中在一个函数中:

```

def connect {α : Type} {β : Type} :
  Option α → (α → Option β) → Option β
| Option.none, _ => Option.none
| Option.some a, f => f a

```

`connect` 函数作用于 `Option`。如果其值为 `Option.none`, 我们保持原样。这对应于一种错误情况, 而错误是「具有粘性的」。否则, 其值的形式为 `Option.some a`, 我们对 `a` 应用操作 `f` —— 或者说将 `f` 的参数^{Bind}绑定到 `a`。现在我们可以使用 `connect` 来编写求和函数:

```

def sum257Connect (ns : List ℕ) : Option ℕ :=
  connect (nth ns 1)
    (fun n2 ↦ connect (nth ns 4)
      (fun n5 ↦ connect (nth ns 6)
        (fun n7 ↦ Option.some (n2 + n5 + n7))))

```

直观上, 该程序执行以下步骤:

1. 使用 `nth` 从列表中提取第二个元素。如果没有该元素, `nth` 返回 `Option.none`; 直接返回此值即可。否则, 将 `n2` 绑定到该元素, 并继续下一步。
2. 对第五个和第七个元素执行相同的操作, 同理推导。
3. 返回 `n2`、`n5` 和 `n7` 的和, 并包装在 `Option.some` 中。

在数学上, 我们的新函数 `sum257Connect` 等于原来的函数 `sum257`。

我们不必自己定义 `connect`, 而可以使用 Lean 预定义的通用^{Bind Operation}绑定操作。它接受相同的参数和顺序。以下是新代码:

```

def sum257Bind (ns : List ℕ) : Option ℕ :=

```

```
bind (nth ns 1)
  (fun n2 ↦ bind (nth ns 4)
    (fun n5 ↦ bind (nth ns 6)
      (fun n7 ↦ pure (n2 + n5 + n7))))
```

我们也使用了预定义的^{Pure Function}纯函数 `pure`，而非使用 `Option.some` 来将一个纯 α 值转换为 `Option  $\alpha$` 。

使用预定义的 `bind` 的优点之一是它提供了^{Syntactic Sugar}语法糖，即 `>>=` 运算符的形式：

```
def sum257Op (ns : List N) : Option N :=
  nth ns 1 >>=
    fun n2 ↦ nth ns 4 >>=
      fun n5 ↦ nth ns 6 >>=
        fun n7 ↦ pure (n2 + n5 + n7)
```

语法 `oa >>= f` 展开为 `bind oa f`，其中 `oa` 的类型为 `Option  $\alpha$` 。

求和程序的倒数第二个版本使用了更重的语法糖：

```
def sum257Dos (ns : List N) : Option N :=
  do
    let n2 ← nth ns 1
  do
    let n5 ← nth ns 4
  do
    let n7 ← nth ns 6
    pure (n2 + n5 + n7)
```

^{do Notation}

do 记法为带作用的程序提供了一种方便的语法。程序 `do let a ← oa ...` 等价于 `oa >>= (fun a ↦ ...)`。如果我们对 `oa` 的计算结果不感兴趣，可以省略 `let a ←` 绑定并编写 `do oa ...`，它展开为 `oa >>= (fun _ ↦ ...)`。

`do` 记法可以方便地在单个块中允许重复多次 `let` 绑定。这引出了程序的最终版本：

```
def sum257Do (ns : List N) : Option N :=
  do
    let n2 ← nth ns 1
    let n5 ← nth ns 4
    let n7 ← nth ns 6
    pure (n2 + n5 + n7)
```

带有箭头 `←` 的每一行都尝试读取一个值。在失败的情况下，整个程序求值为 `Option.none`。

上述函数可以被读作一个指令式程序，其中每个 `nth` 调用都可以抛出一个异常。但是，即使这种表示法具有指令式风格，该函数仍然是一个纯函数式程序。

7.2 两个操作和三个定律

`Option` 类型构造子是单子的一个例子，被称为^{Option Monad}**选项单子**。通常，单子是一个一元类型构造子 $m : \text{Type} \rightarrow \text{Type}$ ，并配有两个特殊的操作：

$$\begin{aligned} \text{pure } \{\alpha : \text{Type}\} &: \alpha \rightarrow m \alpha \\ \text{bind } \{\alpha \beta : \text{Type}\} &: m \alpha \rightarrow (\alpha \rightarrow m \beta) \rightarrow m \beta \end{aligned}$$

通常，大括号表示隐式参数。对于选项，`pure` 操作仅是 `Option.some`，而 `bind` 则等同于我们的 `connect`。

类型 $m \alpha$ 的值是一个带作用的程序。`pure` 操作将类型为 α 的纯、无作用的程序嵌入到 $m \alpha$ 中。`bind` 操作组合了两个带作用的程序，其类型分别为 $m \alpha$ 和 $m \beta$ 。第一个程序的输出（类型为 α ）被传递给第二个程序。第二个程序的输出也是组合程序的输出。

我们可以将单子想象成一个包含某些数据的盒子。这个盒子捕捉了一些特殊的作用（例如异常、可变状态）。`pure` 操作将数据放入盒子中，而 `bind` 允许我们访问盒子中的数据并修改它——甚至可能改变它的类型，因为结果类型是 $m \beta$ ，而不是 $m \alpha$ 。然而，没有通用的方法可以从盒子中提取数据——即从 $m \alpha$ 获得一个 α 。盒子里可能没有任何 α 值，也可能有多个。

总而言之，`pure a` 是一个包含值 a 的盒子，没有作用，而 `bind ma f`（也写作 `ma >>= f` 或 `do a ← ma; f a`）执行 `ma`，然后使用 `ma` 的拆箱结果 a 来执行 f 。为类型为 $m \alpha$ 或 $m \beta$ 的「装箱值」使用 `ma` 或 `mb` 这样的名称，并为类型为 α 或 β 的数据使用 a 或 b 是很方便的。

单子是一个具有许多应用场景的抽象概念。选项类型只是众多实例中的一个。下表概述了一些单子实例及其作用。

类型	作用
<code>id</code>	无作用
<code>Option</code>	简单异常
$\text{fun } \alpha \mapsto \sigma \rightarrow \alpha \times \sigma$	传递类型为 σ 的状态
<code>Set</code>	返回 α 值的非确定性计算
$\text{fun } \alpha \mapsto t \rightarrow \alpha$	读取类型为 t 的元素（例如配置）
$\text{fun } \alpha \mapsto \mathbb{N} \times \alpha$	附加运行时间（例如建模时间复杂度）
$\text{fun } \alpha \mapsto \text{String} \times \alpha$	附加文本输出（例如用于日志记录）
<code>IO</code>	与操作系统的交互
<code>TacticM</code>	与证明助手的交互

以上所有这些都是一元类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 。一些作用可以组合（例如 $\text{fun } \alpha \mapsto \text{Option } (t \rightarrow \alpha)$ ）。有些作用是不可执行的（例如 Set ）；尽管如此，它们对于抽象地建模程序非常有用。特定的类型构造子 m 可能会提供 pure 和 bind 之外的更多运算符。例如，它们可能提供提取装箱值的方法。

单子有几个好处。它们提供了高度可读的 do 记法。它们支持通用操作，例如 $\text{List.mmap } \{\alpha \beta : \text{Type}\} : (\alpha \rightarrow m \beta) \rightarrow \text{List } \alpha \rightarrow m (\text{List } \beta)$ ，这些操作对所有单子 m 都是统一的。引用《**Lean 语言编程**》[2] 的话：

这种抽象的力量不仅在于它提供了可以用于所有这些不同实例化的通用函数和表示法，还在于它提供了一种思考它们共同点的有益方式。

单子除了是一个有用的计算机科学概念外，还提供了一个 Axiomatic Reasoning **公理化推理** 的好例子。

通常要求 bind 和 pure 操作遵守三条定律。 bind 操作组合了两个程序。如果其中任何一个程序是纯程序，我们可以将其内联并消除 bind 。这引出了前两条定律：

```
do
  let a' ← pure a      =    f a
  f a'
```

以及

```
do
  let a ← ma           =    ma
  pure a
```

第三条定律是 bind 的结合律。它允许我们展平一个嵌套计算：

```
do
  let b ←
    do
      let a ← ma
      f a
    g b
  =
do
  let a ← ma
  let b ← f a
  g b
```

之前我们将单子比作一个盒子。更具体地说，将盒子想象成一个瑞士银行账户可能会有所帮助，其中 α 是钱。第一条定律意味着如果你把一些钱存入账户，你就可以把它取出来。第二条定律意味着如果你取走一些钱然后又把它放回去，没有人会注意到。第三条定律意味着将两项银行操作结合在一起然后再进行第三项操作，与先单独执行第一项操作然后再执行另外两项操作是一样的。鉴于瑞士银行在保密方面的声誉，这三条定律似乎都是合理的。

7.3 一种类型类

单子是一种数学结构，因此我们在 Lean 中使用类型类来指定它们。回想一下，类型类是一个参数化的结构体类型，通常是以类型作为参数，但在这里是以类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 作为参数。每当我们对具体的 m 使用类型类中的字段时，类型类推导机制就会检索相关的结构体值——即类型类实例。

下面是一个可能的单子 Lean 定义，连同三条定律：

```
class LawfulMonad (m : Type → Type)
  extends Pure m, Bind m where
  pure_bind {α β : Type} (a : α) (f : α → m β) :
    (pure a >>= f) = f a
  bind_pure {α : Type} (ma : m α) :
    (ma >>= pure) = ma
  bind_assoc {α β γ : Type} (f : α → m β) (g : β → m γ)

    (ma : m α) :
    ((ma >>= f) >>= g) = (ma >>= (fun a ↦ f a >>= g))
```

让我们逐步研究这个定义：

- 我们正在创建一个由一元类型构造子 m 参数化的结构体——即一个类型为 $\text{Type} \rightarrow \text{Type}$ 的值。
- 该结构体从名为 Pure 和 Bind 的结构体中继承了字段和任何语法糖。这些提供了 m 上的 pure 和 bind 操作，具有预期的类型和语法糖。
- 最后，在 Pure 和 Bind 已经提供的字段基础上，增加了三个字段 (pure_bind、bind_pure 和 bind_assoc)。每个字段都是三条定律之一的证明。

我们将我们的类型类称为 LawfulMonad，因为要求这三条定律成立。要实例化此定义，我们必须提供类型构造子 m 、合适的 bind 和 pure 运算符，以及定律的证明。

Lean 包含了它自己的单子概念，也称为 LawfulMonad。它与我们的定义大致等价，但分布在多个类型类中。

7.4 无作用

在本节以及第 7.5 节到第 7.7 节中，我们将回顾由特定单子提供的各种作用。我们从 ^{Identity Monad}恒等单子开始，它根本不提供任何特殊作用。

Lean 的常量 $\text{id} \{ \alpha : \text{Type} \} : \alpha \rightarrow \alpha$ 被定义为恒等函数 $\text{fun } x \mapsto x$ 。恒等类型构造子是通过取 $\alpha := \text{Type}$ 获得的。我们可以将其注册为单子：

```

def id.pure {α : Type} : α → id α
  | a => a

def id.bind {α β : Type} : id α → (α → id β) → id β
  | a, f => f a

instance id.LawfulMonad : LawfulMonad id :=
{ pure      := id.pure
  bind      := id.bind
  pure_bind :=
    by
      intro α β a f
      rfl
  bind_pure :=
    by
      intro α ma
      rfl
  bind_assoc :=
    by
      intro α β γ f g ma
      rfl }

```

注册过程要求我们提供五个组件：pure 和 bind 操作以及三条定律的证明。

Identity Monad

恒等单子是可能的最简单的单子。它提供了一个简单的盒子，其中只有一个值，没有任何作用。它在加法算术中扮演着与 `o` 类似的角色。我们可以将其他单子看作是它的变体；例如，选项单子就是一个增加了一个代表错误状态的特殊 `Option.none` 值的恒等单子。

7.5 基本异常

正如我们在上面看到的，选项类型提供了一个基本的异常机制。下面的代码展示了如何将 `Option : Type → Type` 注册为一个合法单子：

```

def Option.pure {α : Type} : α → Option α :=
  Option.some

def Option.bind {α β : Type} :
  Option α → (α → Option β) → Option β
  | Option.none, _ => Option.none

```

```

| Option.some a, f => f a

instance Option.LawfulMonad : LawfulMonad Option :=
{ pure      := Option.pure
  bind      := Option.bind
  pure_bind :=
    by
      intro  $\alpha$   $\beta$  a f
      rfl
  bind_pure :=
    by
      intro  $\alpha$  ma
      cases ma with
      | none   => rfl
      | some _ => rfl
  bind_assoc :=
    by
      intro  $\alpha$   $\beta$   $\gamma$  f g ma
      cases ma with
      | none   => rfl
      | some _ => rfl }

```

这三个证明都很简单直接。

除了标准操作之外，抛出和捕获异常也很有用。这可以实现如下：

```

def Option.throw { $\alpha$  : Type} : Option  $\alpha$  :=
  Option.none

def Option.catch { $\alpha$  : Type} : Option  $\alpha$  → Option  $\alpha$  →
  Option  $\alpha$ 
| Option.none,   ma' => ma'
| Option.some a, _  => Option.some a

```

`Option.throw` 操作会抛出一个异常，使程序处于错误状态(`Option.none`)。`Option.catch` 操作可用于从之前的异常中恢复。如果程序当前处于错误状态，`Option.catch` 会调用一些异常处理代码（其第二个参数）。这段代码可能会依次抛出一个新的异常。如果 `Option.catch` 应用于正常状态（形式为 `Option.some a`），则什么也不会发生。

作为 `Option.catch ma ma'` 的一种便捷替代方案，Lean 支持语法 `ma.catch ma'`。这里有一个示意性例子，演示了使用此语法进行抛出和捕获：

```

do
  ...
  if ... then
    Option.throw
  else
    ...
.catch do
  ...

```

相应的 Java 代码如下所示：

```

try {
  ...
  if (...) {
    throw new UnknownException();
  } else {
    ...
  }
} catch (UnknownException e) {
  ...
}

```

^{Option}**选项**只应对一种错误状态。一种更通用的抽象被称为^{Error Monad}**错误单子**，它支持不同的错误，就像 Java 和其他编程语言中的异常一样。

7.6 可变状态

^{State Monad}**状态单子**提供了一种对应于可变状态的抽象。对于某些编程语言，编译器可以检测到状态单子的使用，并将使用它们的程序翻译成更高效的指令式程序。

无可否认，「对应于可变状态的抽象」听起来可能有些抽象，所以让我们考虑一个半具体的例子。如果你有一些函数式编程的经验，你可能在某个时候写过看起来非常像下面这个片段的代码：

```

def welcomeNewUser (userName : String) (ctxt : Context)
:
  (N × String) × Context :=
let
  (userId, ctxt') := createUser userName ctxt
  (password, ctxt'') := generateTemporaryPassword
  userId ctxt'

```

```

      (ok, ctxt'') := sendUnencryptedEmail user password
    ctxt''
  in
    ((userId, password), ctxt'')

```

(通过未加密的邮件发送密码, 并忽略函数的 ok 状态是非常重要的。) 该函数接受一些全局状态或^{Context}语境作为输入, 并依次调用三个函数, 其中每个函数都接受一个语境值, 并返回一些数据和新的语境。语境实际上是在程序中「^{Threaded Through}穿针引线」。这使我们能够在没有副作用的编程语言中拥有可变状态。

上述方法容易出错——很容易忘记一个撇号 (') 并将错误的语境传递给函数。代码也被所有的语境变量弄得杂乱无章。如果我们能像下面这样写呢?

```

def welcomeNewUserDo (userName : String) :
  Context → (ℕ × String) × Context :=
do
  let user ← createUser userName
  let password ← generateTemporaryPassword user
  let ok ← sendUnencryptedEmail user password
  pure (user, password)

```

这正是^{State Monad}状态单子所提供的。

状态单子建立在^{Binary Type Constructor}二元类型构造子 Action 之上, 它捕捉了在类型为 σ 的^{State}状态上进行^{Computation Action}计算或操作的概念, 其^{Return Value}返回值类型为 α 。在 Lean 中, $\text{Action } \sigma \alpha$ 被定义为等同于 $\sigma \rightarrow \alpha \times \sigma$ ¹。

```

def Action (σ α : Type) : Type :=
  σ → α × σ

```

(由于类型也是项, 我们也可以使用 def 来定义类型缩写。) 对于给定的类型 σ , 我们有 $\text{Action } \sigma : \text{Type} \rightarrow \text{Type}$ 是一个单子。类型 σ 对确切的内存布局进行了抽象。例如, 我们可以使用偶对或列表来表示内存, 并相应地实例化抽象的状态 σ 。

一个^{Stateful Action}带状态的操作是一个函数, 它接收某个状态并返回一个值和一些新的状态。Action 的定义中的 $\sigma \rightarrow$ 部分给出了旧状态; 笛卡尔积的左分量 α 给出了计算的结果; 而乘积的右分量 σ 给出了新的状态。因此, 状态被隐式地贯穿于程序之中。与其他带作用的程序一样, do 记法只暴露数据——类型为 α 的值——并隐藏了作用——旧的和新的 σ 状态。

当 $\sigma := \text{Unit}$ 时, 这是一个^{Cardinality}基数为一的类型 (类似于 C 或 Java 中的 void), 其唯一的值写为 (), 这对应于恒等单子: 类型 $\text{Unit} \rightarrow \alpha \times \text{Unit}$ 与 α 是^{Isomorphic}同构的。这种直觉可以指导我们定义 pure 和 bind。

¹这个类型构造子在传统上被称为 State, 但这个名称非常容易引起混淆。

我们首先定义基本操作：两个操作 `read` 和 `write` 用于访问内存，以及标准操作 `bind` 和 `pure`：

```
def Action.read {σ : Type} : Action σ σ
  | s => (s, s)

def Action.write {σ : Type} (s : σ) : Action σ Unit
  | _ => ((), s)

def Action.pure {σ α : Type} (a : α) : Action σ α
  | s => (a, s)

def Action.bind {σ : Type} {α β : Type} (ma : Action σ α)
  )
    (f : α → Action σ β) :
    Action σ β
  | s =>
    match ma s with
    | (a, s') => f a s'
```

`read` 操作简单地返回当前状态 `s` (在第一个^{Pair}偶对分量中)，并保持状态不变 (在第二个偶对分量中)。`write` 操作将当前状态替换为 `s` 并返回 `()`。`pure` 操作返回未改变的当前状态 `s`，并与给定的值 `a` 组成^{Tuple}元组。`bind` 操作将初始状态传递给 `ma` 参数，产生一个结果 `a` 和一个新的状态 `s'`。这些被传递给 `f`，它返回一个新的结果和新的状态。

为了将 `Action` 类型构造子注册为一个^{Lawful Monad}合法单子，我们需要像以前一样证明这三条定律：

```
instance Action.LawfulMonad {σ : Type} :
  LawfulMonad (Action σ) :=
  { pure      := Action.pure
    bind      := Action.bind
    pure_bind :=
      by
        intro α β a f
        rfl
    bind_pure :=
      by
        intro α ma
```

```

      rfl
bind_assoc :=
  by
    intro  $\alpha$   $\beta$   $\gamma$  f g ma
    rfl }

```

作为一个具体的例子，下面的程序删除了列表中所有小于前一个元素的元素，留下一个递增元素的列表。最大元素存储为状态 σ 。请注意状态是如何通过 read 和 write 访问的。

```

def increasingly : List N → Action N (List N)
| []      => pure []
| (n :: ns) =>
  do
    let prev ← Action.read
    if n < prev then
      increasingly ns
    else
      do
        Action.write n
        let ns' ← increasingly ns
        pure (n :: ns')

```

要执行该程序，我们必须提供一个初始状态。最后一个状态连同结果列表一起返回。它对应于列表中遇到的最大元素或起始状态。因此，命令

```

#eval increasingly [1, 2, 3, 2] 0
#eval increasingly [1, 2, 3, 2, 4, 5, 2] 0

```

产生如下输出：

```

([1, 2, 3], 3)
([1, 2, 3, 4, 5], 5)

```

7.7 非确定性

选项单子存储零个或一个 α 值，恒等单子和状态单子存储恰好一个值，而集合单子存储可能无限数量的值。这^{Set Monad}对于建模^{Nondeterminism}非确定性很有用，即作为一组可能的行为。

Lean 的类型 $\text{Set } \alpha$ 定义为 $\alpha \rightarrow \text{Prop}$ 。换句话说，集合是通过其^{Characteristic Predicate}特征谓词来标识的。支持熟悉的运算符，例如空集 ($\emptyset$)、全集 (Set.univ)、并集 ($\cup$)、交集 ($\cap$) 和属于 ($\in$)，以及传统的花括号表示法，例如 $\{a\}$ 、 $\{a, b\}$ 和 $\{x \mid P x\}$ 。许多集合构造可以通过 simp 来简化。

Set 类型构造子可以按如下方式注册为合法单子：

```
def Set.pure {α : Type} : α → Set α
  | a => {a}

def Set.bind {α β : Type} : Set α → (α → Set β) → Set
  β
  | A, f => {b | ∃a, a ∈ A ∧ b ∈ f a}

instance Set.LawfulMonad : LawfulMonad Set :=
{ pure      := Set.pure
  bind      := Set.bind
  pure_bind :=
    by
      intro α β a f
      simp [Set.pure, Set.bind]
  bind_pure :=
    by
      intro α ma
      simp [Set.pure, Set.bind]
  bind_assoc :=
    by
      intro α β γ f g ma
      simp [Set.bind]
      aesop }
```

pure 操作简单地将给定的值 a 放入单元素集合 $\{a\}$ 中。bind 操作对集合 A 中的所有值调用 f ，并返回所有结果的并集。例如，如果 $A := \{3, 8\}$ 且 $f := (\text{fun } a \mapsto \{a + 1, a + 2\})$ ，那么 $\text{Set.bind } A \ f$ 等于 $\{4, 5, 9, 10\}$ 。

请注意，在这三个证明中，我们都展开了通用 pure 和 bind 常量的定义，随后是 Set.pure 和 Set.bind 的定义。

最后一个证明依赖于 aesop 策略。目标的目的是：

$$\begin{aligned} & \forall x, \{b \mid \exists a, (\exists a_1 \in ma, a \in f \ a_1) \wedge b \in g \ a\} \\ & = \{b \mid \exists a \in ma, \exists a_1 \in f \ a, b \in g \ a_1\} \end{aligned}$$

等式的两侧除了存在量词的位置——以及令人困惑的绑定变量名称——之外都是相同的。这个丑陋的命题可以通过繁琐的引入和消去序列来证明，但我们应该获得更多的自动化。

Satisfiability Modulo Theories

上述 aesop 的一个替代方案是调用 grind, 这是一个灵感来自所谓的**可满足性模理论 (SMT)** 求解器的策略。

7.8 通用证明策略

aesop

Automated Extensible Search For Obvious Proofs

aesop 策略 [17], 其名称代表「**用于明显证明的自动化可扩展搜索**」, 是一个通用的证明搜索策略。此外, 它还可以对假设中的逻辑符号 $\wedge$ 、 $\vee$ 、 $\leftrightarrow$ 和 $\exists$ 进行消去, 并在目标中引入 $\wedge$ 、 $\leftrightarrow$ 和 $\exists$, 并且它还会定期调用 simp。它可以成功证明目标、失败或部分成功, 从而给用户留下一些未完成的子目标。

grind

grind 策略的灵感来自现代 SMT 求解器。它结合了多种推理引擎共同协作, 以解决涉及等值推理、分类讨论和线性算术等目标。与 simp 不同, grind 以无向 (即非从左到右) 的方式对等式进行推理。它系统地从目标中存在的等式中推导出新的等式。例如, 如果目标包含 $b = a$ 和 $f\ b \neq f\ a$ 作为假设, grind 将从 $b = a$ 推导出 $f\ b = f\ a$, 并发现与 $f\ b \neq f\ a$ 的矛盾。

有些目标可以用 grind 解决而不能用 aesop 解决, 反之亦然。很难预测哪种策略对给定的目标会取得成功。

7.9 一个通用算法：在列表上迭代

Effectful

假设我们使用 map 在列表的所有元素上应用一个**带作用的函数** f 。然后我们得到一个普通的带作用值的列表。例如：

```
def nthxFine {α : Type} (xss : List (List α)) (n : ℕ) :
  List (Option α) :=
  List.map (fun xs ↦ nth xs n) xss
```

函数 nthxFine xss n 试图提取 xss 中每个列表的第 $n + 1$ 个元素。运行

```
#eval nthxFine [[11, 12, 13, 14], [21, 22, 23]] 2
```

返回 [Option.some 13, Option.some 23]。

Constructor

这些 Option.some **构造子** 可能会很不方便。通常, 我们只关心是否出现了错误。这引出了我们的最后一个例子：一个通用的带作用程序 mmap, 它在列表上迭代, 并将带作用函数 f 应用于每个元素。该定义是递归的：

```
def mmap {m : Type → Type} [LawfulMonad m] {α β : Type}
  (f : α → m β) :
```

```

List α → m (List β)
| []      => pure []
| a :: as =>
  do
    let b ← f a
    let bs ← mmap f as
    pure (b :: bs)

```

请注意，该函数返回一个包含列表的单个 m 值，而不是一个 m 值的列表。试着弄清楚为什么它是良类型的，并且具有预期的行为。

我们现在可以尝试使用 `mmap` 而不是 `List.map`：

```

def nthCoarse {α : Type} (xss : List (List α)) (n : ℕ)
:
  Option (List α) :=
  mmap (fun xs ↦ nth xs n) xss

```

运行

```
#eval nthCoarse [[11, 12, 13, 14], [21, 22, 23]] 2
```

返回 `Option.some [13, 23]`，在纯列表周围只有一个 `Option.some`。

`mmap` 函数在追加运算符 `++` 上具有分配性。`do` 记法不仅对定义函数有用，而且对陈述它们的性质也很有用：

```

theorem mmap_append {m : Type → Type} [LawfulMonad m]
  {α β : Type} (f : α → m β) :
  ∀ as as' : List α, mmap f (as ++ as') =
  do
    let bs ← mmap f as
    let bs' ← mmap f as'
    pure (bs ++ bs')
| [], _ =>
  by simp [mmap, LawfulMonad.bind_pure,
    LawfulMonad.pure_bind]
| a :: as, as' =>
  by simp [mmap, mmap_append _ as as',
    LawfulMonad.pure_bind,
    LawfulMonad.bind_assoc]

```

7.10 新引入的 Lean 结构小结

记法

<code>do</code>	表示一个带作用程序的开始
<code>let ... ← ...</code>	在带作用程序中分配变量
<code>>>=</code>	组合带作用计算

定理

<code>Set.ext</code>	集合外延性
----------------------	-------

策略

<code>aesop</code>	使用通用搜索过程证明命题
<code>grind</code>	使用 SMT 风格的推理证明命题

元编程

像大多数证明助手一样，Lean 可以通过自定义策略和其他功能进行扩展。对 Lean 自身进行编程，而不仅仅是使用它，被称为^{Metaprogramming}**元编程**。Lean 的元编程框架大多使用与 Lean 输入语言相同的概念和语法，因此我们不需要学习不同的语言来对 Lean 进行编程。单子被用来访问 Lean 的状态。

以归纳类型呈现的^{Abstract Syntax Tree}**抽象语法树**反映了内部数据结构。证明助手的内部结构通过 Lean 函数公开，我们可以使用这些函数来访问当前目标、统一项、查询和修改全局^{Context}**语境**，以及设置属性（例如 `q[simp]`）。

下面是元编程的一些示例应用：

- 目标转换（例如：应用安全引入规则，将目标置于^{Negation Normal Form}**否定范式**）；
- 启发式证明搜索（例如：结合^{Backtracking}**回溯**应用非安全引入规则）；
- ^{Decision Procedures}**判定过程**（例如：针对线性算术、命题逻辑）；
- 定义生成器（例如：归纳类型的 Haskell 风格的 `deriving`）；
- 建议工具（例如：定理查找器、反例生成器）；
- 导出器（例如：文档生成器）；
- 即时证明自动化（为了避免^{Boilerplate}**样板代码**或重复）。

正如数学家及 Lean 用户 Kevin Buzzard 所写的：¹

如果你发现自己正在^{Grinding}「**刷怪**」（借用电脑游戏的用语），一遍又一遍地做着同样的事情，因为你需要这样做才能取得进展，那么你可以试着说服一位计算机科学家为你编写一个策略（或者如果你足够勇敢，可以编写^{meta}**元** Lean 代码，甚至可以自己编写策略）。

¹<https://xenaproject.wordpress.com/2020/02/09/lean-is-better-for-proper-maths-than-all-the-other-theorem-provers/>

8.1 策略组合子

首先，是一些术语：如果应用一个策略产生了错误，则该策略^{Fail}**失败**；否则，它^{Succeed}**成功**。策略成功的一种方式是完全证明目标。另一种方式是产生替换当前目标的新子目标。有些策略通过什么都不做来取得成功。

在编写我们自己的策略时，我们经常需要在多个目标上重复某些操作，或者在策略失败时进行恢复。在这种情况下，^{Tactic Combinators}**策略组合子**会很有帮助。

最实用的策略组合子之一是 `repeat' tactic`。它在所有目标上重复调用 `tactic`，然后在产生的子目标上调用，再在产生的子子目标上调用，依此类推，直到 `tactic` 在所有可用目标上都失败为止。下面是一个涉及第 6 章中介绍的 `Even` 谓词的 `repeat'` 示例：

```
theorem repeat'_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  repeat' apply Even.add_two
```

在第一个 `repeat'` 行之后，证明状态包含四个目标：

$\vdash \text{Even } 4 \quad \vdash \text{Even } 7 \quad \vdash \text{Even } 3 \quad \vdash \text{Even } 0$

请注意，所有的合取项都消失了。第二个 `repeat'` 重复应用定理 `Even.add_two`： $\forall k, \text{Even } k \rightarrow \text{Even } (k + 2)$ ，留下了这些目标：

$\vdash \text{Even } 0 \quad \vdash \text{Even } 1 \quad \vdash \text{Even } 1 \quad \vdash \text{Even } 0$

第一个和最后一个目标很烦人，因为它们对应于定理 `Even.zero`。我们可以在应用 `Even.add_two` 失败时尝试应用 `Even.zero` 来证明它们。这可以通过以下方式实现：

```
theorem repeat'_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  repeat'
    first
    | apply Even.add_two
    | apply Even.zero
```

策略组合子 `first | tactic1 | ... | tacticn` 首先尝试执行其第一个参数 `tactic1`。如果失败，则尝试 `tactic2`，依此类推。如果所有指定的策略都失败，则整个组合子也会失败。在上面的示例中，我们还剩两个无法证明的目标：

$\vdash \text{Even } 1$ $\vdash \text{Even } 1$

下一个组合子 `all_goals tactic` 在每个目标上恰好调用一次策略。该组合子仅在 `tactic` 在「所有」目标上都成功时才成功。在下面的示例中它会失败，因为 `Even.add_two` 无法应用于目标 $\vdash \text{Even } 0$ ：

```
theorem all_goals_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  all_goals apply Even.add_two -- fails
```

为了忽略 `tactic` 的失败，我们可以将其包装在 `try` 组合子中：

```
theorem all_goals_try_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  all_goals try apply Even.add_two
```

结果状态为

$\vdash \text{Even } 2$ $\vdash \text{Even } 5$ $\vdash \text{Even } 1$ $\vdash \text{Even } 0$

结构 `try tactic` 等价于 `first | tactic | skip`，其中 `skip` 是一个不做任何事情就成功的策略。因此 `try tactic` 总是成功。一个相关的策略是 `done`：如果没有剩余目标则成功，否则失败。

另一个变体是 `any_goals tactic` 组合子。它尝试在每个目标上调用一次 `tactic`，但与 `all_goals` 不同，只要 `tactic` 在**任何**目标上成功，它就会成功。示例

```
theorem any_goals_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  any_goals apply Even.add_two
```

结果状态为

$\vdash \text{Even } 2$ $\vdash \text{Even } 5$ $\vdash \text{Even } 1$ $\vdash \text{Even } 0$

这与前一个示例中的状态相同。通常，区别在于 `any_goals tactic` 可能会失败，而 `all_goals try tactic` 总是成功。

有时我们希望除非能完全证明一个目标，否则不去管它。组合子 `solve | tactic1 | ... | tacticn` 首先尝试执行其第一个参数 `tactic1`。如果这未能证明目标，则尝试 `tactic2`，依此类推。如果所有指定的策略都未能证明目标，则整个组合子失败。（将此行为与 `first | tactic1 | ... | tacticn` 的行为进行比较，后者只要求指定的策略之一成功，而不要求证明目标。）考虑这个示例：

```
theorem any_goals_solve_repeat_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  any_goals
    solve
    | repeat'
      first
      | apply Even.add_two
      | apply Even.zero
```

第一个和第四个目标已被证明，剩下的两个无法证明的目标与定理陈述中的完全一致：

$\vdash \text{Even } 7$ $\vdash \text{Even } 3$

请注意，`repeat'` 组合子可能会导致无限循环。考虑这个示例：

```
theorem repeat'_Not_example :
  ¬ Even 1 :=
by repeat' apply Not.intro
```

`Not.intro` 规则是 $(?a \rightarrow \text{False}) \rightarrow \neg ?a$ ，因此它应用一次将目标转换为 $\vdash \text{Even } 1 \rightarrow \text{False}$ 。由于 $\neg ?a$ 被定义为 $?a \rightarrow \text{False}$ ，该规则再次应用，再次产生相同的目标。该策略会发生死循环。

最后，^{And Then}「继而」运算符 `<;>` 可用于连接两个策略。左侧在第一个目标上执行。右侧在产生的每个子目标上执行（但不在原始的第二个、第三个等目标上执行）。因此，我们可以这样写

```
by
  induction n <;>
  aesop
```

instead of the more verbose

```
by
  induction n with
  | zero      => aesop
  | succ n' ih => aesop
```

8.2 宏

让我们通过将自定义策略编码为^{Macro}宏，来进行一些实际的元编程。该策略体现了我们在上面的 solve 示例中硬编码的行为：

```
macro "intro_and_even" : tactic =>
  '(tactic|
    (repeat' apply And.intro
      any_goals
      solve
      | repeat'
        first
        | apply Even.add_two
        | apply Even.zero))
```

第一行将 intro_and_even 声明为属于 tactic ^{Syntactic Category}语法范畴的宏。在其余行中，'(tactic| tactic) 结构将指定的策略 tactic 嵌入到宏中。策略本身是使用标准语法指定的。

一旦我们定义了自定义策略，就可以在证明中调用它：

```
theorem intro_and_even_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    intro_and_even
```

这会产生子目标

⊢ Even 4

⊢ Even 3

8.3 元编程单子

宏是一种可用于编写简单证明自动化的机制。然而，对于大多数元编程任务，我们需要使用元编程单子：MetaM 和 TacticM。

MetaM 结合了几种单子的属性：

- 它是一个状态单子，提供对全局^{Context}语境（包括所有定义和归纳类型）、记法和属性（例如 `@[simp]` 定理列表）等的访问。
- 它的行为类似于选项单子。元程序 `failure` 表示策略已失败。
- 它支持追踪（tracing），因此我们可以使用程序 `logInfo` 来显示消息。
- 它支持指令式结构，例如 `for-in` 循环、`continue` 语句和 `return` 语句。

TacticM 通过目标管理扩展了 MetaM：它提供对目标列表的访问。它还允许我们运行 Lean 来填充表达式中的隐式 `{ }` 和类型类 `[ ]` 参数、展开宏等等。

在 Lean 内部，每个目标都表示为一个^{Metavariable}元变量 `?m`，代表一个缺失的项（通常是一个证明项）。每个元变量都有一个类型（通常是一个命题）和一个本地语境，指定了可以用来证明与该元变量相关联的目标的变量和假设。

让我们通过定义一个利用 TacticM 追踪功能的策略，来实际使用一下 TacticM。该策略将显示 Lean 版本号、目标列表以及第一个目标的^{Target}目的（Lean 称之为^{Main Goal}主目标）：

```
def traceGoals : TacticM Unit :=
  do
    logInfo m!"Lean version {Lean.versionString}"
    logInfo "All goals:"
    let goals ← getUnsolvedGoals
    logInfo m!"{goals}"
    match goals with
    | [] => return
    | _ :: _ =>
      logInfo "First goal's target:"
      let target ← getMainTarget
      logInfo m!"{target}"

elab "trace_goals" : tactic =>
  traceGoals
```

这段代码有许多有趣的特性：

- 第一行声明了一个类型为 `TacticM Unit` 的函数 `traceGoals` —— 这是返回 `Unit`（一个基数为 1 的平凡类型）的策略类型。注意，元编程函数的定义使用了与任何 Lean 函数相同的语法。

- 第二行进入了单子。其余行是访问 Lean 内部结构的带作用的操作。
- `m! "..."` 语法指定了一个字符串，其中每个出现的 `{term}`（其中 `term` 是一个 Lean 项）都会被求值并序列化为字符串。例如，如果 `Lean.versionString` 是 `"v4.24.0"`，那么 `m!"Lean version {Lean.versionString}"` 会求值为字符串 `"Lean version v4.24.0"`。
- 最后两行使用 `elab` 命令，告诉 Lean 的解析器将字符串 `"trace_goals"` 识别为一个策略。当 Lean 遇到这个策略时，它会运行 `elab` 命令主体中的元程序——在这里是 `traceGoals`。（当我们之前使用更高层级的 `macro` 命令时，`elab` 是在底层被调用的。）

下面是一个展示如何使用新策略的示例：

```
theorem Even_18_and_Even_20 (α : Type) (a : α) :
  Even 18 ∧ Even 20 :=
  by
    apply And.intro
    trace_goals
    intro_and_even
```

将鼠标悬停在 `trace_goals` 上可见的输出如下：

```
Lean version v4.24.0
All goals:
[case left
α : Type
a : α
⊢ Even 18
,
case right
α : Type
a : α
⊢ Even 20]
First goal's target:
Even 18
```

虽然 Lean 使用熟悉的 `C ⊢ P` 语法显示目标，但它们实际上是元变量。

上述程序中使用的常量具有以下类型：

```
logInfo : MessageData → TacticM Unit
getUnsolvedGoals : TacticM (List MVarId)
getMainTarget : TacticM Expr
```

其中 `MessageData` 代表消息, `MVarId` 代表元变量标识符, 而 `Expr` 代表项。

8.4 第一个示例：一个 Assumption 策略

我们的第一个较大的示例实现了一个 `hypothesis` 策略, 它像预定义的 `assumption` 策略一样, 寻找正确类型 (即正确的命题) 的假设, 并应用它来证明目标:

```
def hypothesis : TacticM Unit :=
  withMainContext
    (do
      let target ← getMainTarget
      let lctx ← getLCtx
      for ldecl in lctx do
        if ! LocalDecl.isImplementationDetail ldecl
      then
        let eq ← isDefEq (LocalDecl.type ldecl)
        target
          if eq then
            let goal ← getMainGoal
            MVarId.assign goal (LocalDecl.toExpr ldecl)
            return
          failure)

elab "hypothesis" : tactic =>
  hypothesis
```

在 `hypothesis` 函数中, 我们首先提取第一个目标的目的和本地语境。为了确保 `getLCtx` 给出的是当前第一个目标的本地语境, 我们将整个 `do` 块传递给 `withMainContext` 函数。通常, 任何 `TacticM` 计算都是在环境本地语境中执行的, 该语境为表达式中出现的自由变量赋予了意义。`withMainContext` 函数将此本地语境设置为当前第一个目标的本地语境。

在 `do` 块内部, 我们使用方便的单子结构 `for-in` 遍历本地语境中的所有声明。对于每个不是所谓“实现细节” (即 Lean 插入的对用户不可见的假设) 的本地变量或假设 `h`, 我们检查其类型 (通常是其命题) 在计算和元变量实例化后是否等于目标。如果是, 我们获取与第一个目标关联的元变量, 并将 `h` 分配给它, 从而证明该目标。最后, 我们 `return`。

由于目标由元变量表示, 将一个项分配给元变量 `?m` 是 Lean 证明目标的底层方式。该项中出现的新元变量对应于必须证明的新子目标。

下面是 `hypothesis` 的一个简单调用:

```

theorem hypothesis_example {α : Type} {p : α → Prop} {a
  : α}
  (hpa : p a) :
  p a :=
  by hypothesis

```

如果我们添加追踪，我们可以看到在找到匹配的假设 hpa 并成功应用之前，会依次尝试 α、p 和 a。

该示例使用了以下新常量：

```

getLCtx : TacticM LocalContext
LocalDecl.isImplementationDetail : LocalDecl → Bool
isDefEq : Expr → Expr → TacticM Bool
LocalDecl.type : LocalDecl → Expr
getMainGoal : TacticM MVarId
MVarId.assign : MVarId → Expr → TacticM Unit
LocalDecl.toExpr : LocalDecl → Expr
failure {α : Type} : TacticM α

```

8.5 表达式

元编程框架围绕着^{Expression}表达式(或^{Term}项)的类型 Expr 展开。表达式的一个重要组成部分是类型为 Name 的^{Name}名称。我们从它们开始。

名称可以使用单反引号指定。例如，'x 代表名称 x，它可以赋予变量或常量。在引用常量时，我们必须指定包括命名空间在内的完整名称；因此，要引用第 6 章中的 Even 谓词，我们必须编写 'LoVe.Even 而不是 'Even。

如果我们想引用一个现有的常量，Lean 提供了双反引号语法，它使用 Lean 通常的名称^{Elaboration}阐释规则查找名称，并将其扩展为完整名称。因此，''Even 和 ''LoVe.Even 都指向名称 LoVe.Even，而如果我们编写一些未声明的名称（如 ''EvenIf），Lean 会报错。

类型 Expr 定义如下：

```

inductive Expr : Type where
  | const    : Name → List Level → Expr
  | sort     : Level → Expr
  | fvar     : FVarId → Expr
  | mvar     : MVarId → Expr
  | app      : Expr → Expr → Expr
  | lam      : Name → Expr → Expr → BinderInfo → Expr

```

```

| bvar      : Nat → Expr
| forallE   : Name → Expr → Expr → BinderInfo → Expr
| letE      : Name → Expr → Expr → Expr → Bool →
  Expr
| lit       : Literal → Expr
| mdata     : MData → Expr → Expr
| proj      : Name → Nat → Expr → Expr

```

让我们来看看主要的构造子：

- `Expr.const` 名字 层级 代表名为 名字 的常量，例如 `Nat.add` 或 `N`。层级 参数代表^{Universe Levels}**宇宙层级**，这一概念将在第 ?? 章中进行解释。例如，`Expr.const 'Nat.add []` 代表 `Nat.add`，而 `Expr.const 'Nat []` 代表 `Nat`（即 `N`）。
- `Expr.sort level` 用于表示类型的类型。例如，`Expr.sort Level.zero` 代表 `Prop`，而 `Expr.sort (Level.succ Level.zero)` 代表 `Type`。
- `Expr.fvar id` 代表本地语境中的自由变量（例如 `a`、`h`）。`id` 参数是该变量的唯一标识符。
- `Expr.mvar id` 代表元变量，即带问号的变量 `?m`。`id` 参数是该元变量的唯一标识符。
- `Expr.app t u` 代表函数 `t` 应用于参数 `u`。例如，`Expr.app (Expr.const 'Nat.succ []) (Expr.const 'Nat.zero [])` 代表 `Nat.succ Nat.zero`。
- `Expr.lam name σ t bi` 代表匿名函数（或 λ -表达式）。`name` 参数是绑定变量的名称，`σ` 参数是绑定变量的类型，`t` 参数是函数体，而 `bi` 参数存储该变量是显式 `()`、隐式 `{ }` 还是类型类 `[]` 参数。
- `Expr.bvar i` 代表绑定变量，使用的是一种被称为^{De Bruijn Index}**De Bruijn 指数**的记法。`Expr.var 0` 指代由最近的^{Binder}**绑定子**绑定的变量，`Expr.var 1` 指代由次近的绑定子绑定的变量，依此类推。

因此，

```

Expr.lam 'x (Expr.const 'Nat []) (Expr.bvar 0)
  BinderInfo.default

```

代表 $\text{fun } x : \mathbb{N} \mapsto x$ ，而

```

Expr.lam 'x (Expr.const 'Nat [])
  (Expr.lam 'y (Expr.const 'Nat []) (Expr.bvar 1)
    BinderInfo.default)
  BinderInfo.default

```

代表 $\text{fun } x \ y : \mathbb{N} \mapsto x$ 。

- `Expr.forallE name  $\sigma$   $\tau$  bi` 代表可能^{Dependent}依值的函数类型。`name` 参数是绑定变量的名称, σ 参数是定义域类型, τ 参数是结果类型, 而 `bi` 与上述 `Expr.lam` 相同。例如,

```
Expr.forallE 'n (Expr.const 'Nat [])
  (Expr.app (Expr.const 'Even []) (Expr.bvar 0))
  BinderInfo.default
```

代表 $(n : \mathbb{N}) \rightarrow \text{Even } n$ (也写成 $\forall n : \mathbb{N}, \text{Even } n$), 而

```
Expr.forallE 'dummy (Expr.const 'Nat [])
  (Expr.const 'Bool []) BinderInfo.default
```

代表 $\mathbb{N} \rightarrow \text{Bool}$ 。

8.6 第二个示例：一个合取式分解策略

在本节和下一节中, 我们定义了另外两个完成明确任务的策略。这两个策略中的第一个被称为 `destruct_and`, 它自动消除前提中的合取式。我们的目标是使如下证明自动化:

```
theorem abc_a (a b c : Prop) (h : a ∧ b ∧ c) :
  a :=
  And.left h
```

```
theorem abc_b (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  And.left (And.right h)
```

```
theorem abc_bc (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  And.right h
```

```
theorem abc_c (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  And.right (And.right h)
```

在每种情况下, 我们都希望只需编写 `by destruct_and h` 作为证明。

我们的策略依赖于一个辅助函数, 该函数以证明项 `hP` (最初是假设 `h`) 作为参数, 我们从中提取^{Conjuncts}合取项:

```

partial def destructAndExpr (hP : Expr) : TacticM Bool
:=
withMainContext
  (do
    let target ← getMainTarget
    let P ← inferType hP
    let eq ← isDefEq P target
    if eq then
      let goal ← getMainGoal
      MVarId.assign goal hP
      return true
    else
      match Expr.and? P with
      | Option.none          => return false
      | Option.some (Q, R) =>
        let hQ ← mkAppM 'And.left #[hP]
        let success ← destructAndExpr hQ
        if success then
          return true
        else
          let hR ← mkAppM 'And.right #[hP]
          destructAndExpr hR)

```

与 hypothesis 一样，我们将整个 do 块传递给 withMainContext 函数。这确保了 inferType 和 isDefEq 在正确的本地语境中运行。

在 do 块内部，我们首先提取第一个目标的目的和 hP 的类型（通常是其命题）P。如果它们在计算和元变量实例化后相等，我们就通过分配其元变量来关闭目标，就像我们在 hypothesis 示例中所做的那样，并返回 true 表示成功。否则，我们检查 hP 的命题是否具有 $Q \wedge R$ 的形式。如果是，我们使用证明项 $hQ := \text{And.left } hP$ （它是 Q 的证明）递归调用辅助函数。如果成功，则完成；否则，我们尝试证明项 $hR := \text{And.right } hP$ （它是 R 的证明）。

注意函数定义开始处的关键字 partial。这里需要它是因为 Lean 无法证明该函数总是终止。由于该函数仅用作元程序，而不是在命题内部，因此终止是可选的，我们可以通过指定 partial 来禁用终止检查。

同样值得注意的是 mkAppM 函数，它用于构造常量对参数数组的^{Curried}柯里化应用。数组类似于列表，但书写时带有前缀符号 #（例如 $\#[1, 2, 3]$ ）。使用 mkAppM 比多次应用 Expr.app 构造子更方便。此外，mkAppM 允许我们省略隐式参数（例如命题 Q 和 R），否则我们必须将它们作为参数提供给 And.left 和 And.right。

主函数剩下的工作很少了：

```
def destructAnd (name : Name) : TacticM Unit :=
  withMainContext
    (do
      let h ← getFVarFromUserName name
      let success ← destructAndExpr h
      if ! success then
        failure)

elab "destruct_and" h:ident : tactic =>
  destructAnd (getId h)
```

该函数使用 `getFVarFromUserName` 获取假设 `h`，并调用辅助函数。如果辅助函数返回 `false`，则策略失败。

我们现在可以在这些激励性示例中使用我们的新工具了：

```
theorem abc_a_again (a b c : Prop) (h : a ∧ b ∧ c) :
  a :=
  by destruct_and h

theorem abc_b_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  by destruct_and h

theorem abc_bc_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  by destruct_and h

theorem abc_c_again (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  by destruct_and h
```

上述元程序中使用了以下新常量：

```
inferType : Expr → TacticM Expr
Expr.and? : Expr → Option (Expr × Expr)
mkAppM : Name → Array Expr → TacticM Expr
getFVarFromUserName : Name → TacticM Expr
```

8.7 第三个示例：一个直接证明查找器

有时我们陈述了一个定理，证明了它，后来才意识到该定理已经存在。这可以通过使用 `prove_direct` 来防止，该策略会遍历所有可用的定理，并检查其中是否有一个可以证明当前目标。我们将分步审阅其代码。

第一步是一个函数 `isTheorem`，如果声明是公理或定理，它返回 `true`，否则返回 `false`：

```
def isTheorem : ConstantInfo → Bool
  | ConstantInfo.axiomInfo _ => true
  | ConstantInfo.thmInfo _   => true
  | _                        => false
```

我们将使用此函数来过滤掉我们不感兴趣的声明。

下一个函数将名为 `name` 的定理应用于当前目标：

```
def applyConstant (name : Name) : TacticM Unit :=
do
  let cst ← mkConstWithFreshMVarLevels name
  liftMetaTactic (fun goal ↦ MVarId.apply goal cst)
```

给定一个名称，`mkConstWithFreshMVarLevels` 函数创建一个代表该常量的表达式 `cst`。函数名称中提到的“新鲜元变量层级”将在第 ?? 章中变得更加清晰。`MVarId.apply`（不要与 `MVarId.assign` 混淆）然后将该常量应用于当前目标，设置 `?m := cst ?m1 ... ?mn`，并返回代表 `cst` 前提的新鲜元变量 `?mj`。

`liftMetaTactic` 函数检索第一个目标的标识符，在底层 `MetaM` 单子中对该目标运行给定的函数，并将该目标替换为函数返回的子目标。

下一个函数实现了一个其行为类似于 `<;>` 但可以从元程序中使用的组合子：

```
def andThenOnSubgoals (tac1 tac2 : TacticM Unit) :
  TacticM Unit :=
do
  let origGoals ← getGoals
  let mainGoal ← getMainGoal
  setGoals [mainGoal]
  tac1
  let subgoals1 ← getUnsolvedGoals
  let mut newGoals := []
  for subgoal in subgoals1 do
    let assigned ← MVarId.isAssigned subgoal
    if ! assigned then
```

```

    setGoals [subgoal]
  tac₂
  let subgoals₂ ← getUnsolvedGoals
  newGoals := newGoals ++ subgoals₂
  setGoals (newGoals ++ List.tail origGoals)

```

TacticM 单子跟踪当前要证明的目标。我们可以使用 `getGoals` 获取列表，并使用 `setGoals` 设置列表。如果我们希望策略暂时专注于特定的子目标，设置子目标列表是非常有用的。

在这里，我们首先专注于第一个目标 (`setGoals [mainGoal]`) 并调用第一个策略。对于产生的每个子目标，我们专注于它 (`setGoals [subgoal]`) 并调用第二个策略。从第二个策略产生的所有未解决的子子目标 (`subsubgoals`) 都收集在可变量 `newGoals` 中。由于证明一个目标有时会实例化另一个元变量，我们在每次迭代中检查当前子目标的元变量是否已被分配，如果是，则跳过该子目标。最后，我们更新目标以包含所有待处理的目标：`newGoals` 中的目标，以及 `origGoals` 中除第一个目标外所有我们尚未考虑的目标。

通常，在策略结束时，我们应该确保目标列表由所有待证明的目标组成。否则，我们可能会遇到一些令人费解的错误，例如：

```
declaration has metavariables
```

我们还需要一个策略，它尝试使用由其名称指定的定理来证明目标，并调用 `hypothesis` 来证明任何产生的子目标：

```

def proveUsingTheorem (name : Name) : TacticM Unit :=
  andThenOnSubgoals (applyConstant name) hypothesis

```

这在程序上等同于证明 `apply name <;> hypothesis`。

最后，我们准备好审阅主函数了：

```

def proveDirect : TacticM Unit :=
  do
    let origGoals ← getUnsolvedGoals
    let goal ← getMainGoal
    setGoals [goal]
    let env ← getEnv
    for (name, info)
      in SMap.toList (Environment.constants env) do
      if isTheorem info && ! ConstantInfo.isUnsafe info
    then
      try

```

```

    proveUsingTheorem name
    logInfo m!"Proved directly by {name}"
    setGoals (List.tail origGoals)
    return
  catch _ =>
    continue
failure

elab "prove_direct" : tactic =>
  proveDirect

```

我们专注于第一个目标，然后遍历环境中声明的所有常量。如果该常量是一个定理，并且不是“unsafe”（一个与我们的不安全规则和策略概念无关的 Lean 概念），我们尝试使用我们的辅助函数 `proveUsingTheorem` 来应用它。如果成功，我们打印“Proved directly by name”，其中 `name` 是该定理的名称，然后返回。如果失败，我们继续遍历。如果整个遍历都结束了，我们就报告失败。

下面是该策略的实际应用：

```

theorem Nat.symm (x y : ℕ) (h : x = y) :
  y = x :=
  by prove_direct

```

这将打印「Proved directly by symm」。

该消息很有帮助，因为我们可以直接应用指定的定理，而无需依赖相对较慢的 `prove_direct` 策略。具体来说，我们可以结合 `hypothesis` 来应用定理 `symm`，如下所示：

```

theorem Nat.symm_manual (x y : ℕ) (h : x = y) :
  y = x :=
  by
    apply symm
    hypothesis

```

以下是本示例中出现的新常量列表：

```

mkConstWithFreshMVarLevels : Name → TacticM Expr
  liftMetaTactic : (MVarId → MetaM (List MVarId)) →
    TacticM Unit
  MVarId.apply : MVarId → Expr → MetaM (List MVarId)
  getGoals : TacticM (List MVarId)
  setGoals : List MVarId → TacticM Unit
  MVarId.isAssigned : MVarId → TacticM Bool
  getEnv : TacticM Environment
  SMap.toList : ConstMap → List (Name × ConstantInfo)
  Environment.constants : Environment → ConstMap
  ConstantInfo.isUnsafe : ConstantInfo → Bool

```

至此，我们对 `prove_direct` 的审阅就结束了。在 `mathlib` 中也有一个类似的策略，名为 `apply?`。

8.8 杂项策略

虽然本章的重点是开发新策略，但我们也遇到了三个预定义的策略。

skip

`skip` 策略在不执行任何操作的情况下成功。在我们开发自定义策略时，它有时可用作构建基块。

done

如果还有一些目标剩余，`done` 策略会产生失败；否则，它在不执行任何操作的情况下成功。与 `skip` 一样，它可以用作构建基块。

apply?

`apply?` 策略在加载的库中搜索能够精确证明目标的定理。成功时，它会建议一种形式为 `exact ...` 的策略调用，该调用可以插入到形式化（formalization）中。

8.9 新引入的 Lean 结构小结

声明

`partial`

作为可能不终止的元程序声明的前缀

引号

<code>'n</code>	引用一个字面名称
<code>''n</code>	引用一个经过阐释和检查的字面名称

策略

<code>apply?</code>	搜索能够证明当前目标的定理
<code>done</code>	如果还有目标剩余，则失败
<code>skip</code>	不执行任何操作

策略组合子

<code><;></code>	在第一策略产生的所有子目标上调用第二策略
<code>all_goals</code>	在每个目标上调用一次策略，并期望全部成功
<code>any_goals</code>	在每个目标上调用一次策略，并期望至少一个成功
<code>first </code>	依次尝试这些策略，直到其中一个成功为止
<code>repeat'</code>	在所有目标和子目标上重复调用策略，直到失败为止
<code>solve </code>	尝试通过依次尝试这些策略来完全证明当前目标
<code>try</code>	尝试调用一个策略；如果失败，则不执行任何操作

第三部分

程序语义

第四部分

数学

参考文献

- [1] *The Lean 4 Manual*. 2021.
<https://leanprover.github.io/lean4/doc/>.
- [2] J. Avigad, L. de Moura, and J. Roesch. *Programming in Lean*. 2016.
- [3] H. P. Barendregt, W. Dekkers, and R. Statman. *Lambda Calculus with Types*. Perspectives in logic. Cambridge University Press, 2013.
- [4] K. Buzzard, J. Commelin, and P. Massot. Formalising perfectoid spaces. In J. Blanchette and C. Hritcu, editors, *CPP 2020*, pages 299–312. ACM, 2020.
- [5] G. Gonthier. Formal proof—the Four-Color Theorem. *Notices AMS*, 55(11):1382–1393, 2008.
<https://www.ams.org/notices/200811/tx081101382p.pdf>.
- [6] G. Gonthier, A. Asperti, J. Avigad, Y. Bertot, C. Cohen, F. Garillot, S. L. Roux, A. Mahboubi, R. O’Connor, S. O. Biha, I. Pasca, L. Rideau, A. Solovyev, E. Tassi, and L. Théry. A machine-checked proof of the Odd Order Theorem. In S. Blazy, C. Paulin-Mohring, and D. Pichardie, editors, *ITP 2013*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013.
<https://hal.inria.fr/hal-00816699/document>.
- [7] K. Gopinathan and I. Sergey. Certifying certainty and uncertainty in approximate membership query structures. In S. K. Lahiri and C. Wang, editors, *CAV 2020, Part II*, volume 12225 of *Lecture Notes in Computer Science*, pages 279–303. Springer, 2020. <https://arxiv.org/abs/2004.13312>.
- [8] R. Gu, Z. Shao, H. Chen, X. N. Wu, J. Kim, V. Sjöberg, and D. Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In K. Keeton and T. Roscoe, editors, *OSDI 2016*, pages 653–669. USENIX Association, 2016. <https://www.usenix.org/system/files/conference/osdi16/osdi16-gu.pdf>.

- [9] T. C. Hales, M. Adams, G. Bauer, D. T. Dang, J. Harrison, T. L. Hoang, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, T. Q. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, A. H. T. Ta, T. N. Tran, D. T. Trieu, J. Urban, K. K. Vu, and R. Zumkeller. A formal proof of the Kepler conjecture. *CoRR*, abs/1501.02155, 2015. <http://arxiv.org/abs/1501.02155>.
- [10] J. Harrison. Formal verification at Intel. In *LICS 2003*, pages 45–54. IEEE Computer Society, 2003. <https://ieeexplore.ieee.org/document/1210044>.
- [11] J. Harrison, J. Urban, and F. Wiedijk. History of interactive theorem proving. In J. H. Siekmann, editor, *Computational Logic*, volume 9 of *Handbook of the History of Logic*, pages 135–214. Elsevier, 2014. <https://www.cl.cam.ac.uk/~jrh13/papers/joerg.pdf>.
- [12] S. K. Jeremy Avigad, Leonardo de Moura and S. Ullrich. *Theorem Proving in Lean 4*. 2021. https://leanprover.github.io/theorem_proving_in_lean4/.
- [13] G. Klein, J. Andronick, K. Elphinstone, G. Heiser, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal verification of an operating-system kernel. *Commun. ACM*, 53(6):107–115, 2010. https://www.researchgate.net/profile/Gernot_Heiser/publication/220910193_SeL4_Formal_verification_of_an_OS_kernel/links/09e4150f00292a032900000000/SeL4-Formal-verification-of-an-OS-kernel.pdf.
- [14] D. E. Knuth. *The TeXbook*. Addison-Wesley, 1986.
- [15] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A verified implementation of ML. In S. Jagannathan and P. Sewell, editors, *POPL 2014*, pages 179–192. ACM, 2014. <https://cakeml.org/popl14.pdf>.
- [16] X. Leroy. A formally verified compiler back-end. *J. Automated Reasoning*, 43(4):363–446, 2009. <https://arxiv.org/pdf/0902.2137.pdf>.
- [17] J. Limperg and A. H. From. Aesop: White-box best-first proof search for Lean. In B. Pientka and S. Zdancewic, editors, *CPP '23*. ACM, 2023. <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [18] A. Lochbihler. Verifying a compiler for Java threads. In A. D. Gordon, editor, *ESOP 2010*, volume 6012 of *Lecture Notes in Computer Science*, pages 427–447.

- Springer, 2010.
https://link.springer.com/content/pdf/10.1007/978-3-642-11957-6_23.pdf.
- [19] The mathlib Community. The Lean mathematical library. In J. Blanchette and C. Hrițcu, editors, *CPP 2020*, pages 367–381. ACM, 2020.
<https://arxiv.org/pdf/1910.09336.pdf>.
- [20] R. Nederpelt and H. Geuvers. *Type Theory and Formal Proof: An Introduction*. Cambridge University Press, 2014.
- [21] T. Nipkow and G. Klein. *Concrete Semantics: With Isabelle/HOL*. Springer, 2014.
<http://www.concrete-semantics.org/concrete-semantics.pdf>.
- [22] B. O’Sullivan, D. Stewart, and J. Goerzen. *Real World Haskell*. O’Reilly, 2008.
<http://book.realworldhaskell.org/read/>.
- [23] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [24] D. M. Russinoff. A mechanically checked proof of correctness of the AMD K5 floating point square root microcode. *Formal Methods in System Design*, 14(1):75–125, 1999.
<https://link.springer.com/content/pdf/10.1023/A:1008669628911.pdf>.